Universidad Politécnica de Madrid

Escuela Técnica Superior de Ingenieros de Telecomunicación



Grado en Ingeniería de Tecnologías y Servicios de Telecomunicación

Trabajo Fin de Grado

Diseño e implementación del motor cinemático y navegación de un robot hexápodo en ROS 2

Josué Bohórquez Cristóbal

Tutor: D. Álvaro Gutiérrez Martín

2026

Universidad Politécnica de Madrid

Escuela Técnica Superior de Ingenieros de Telecomunicación



Grado en Ingeniería de Tecnologías y Servicios de Telecomunicación

Trabajo Fin de Grado

Diseño e implementación del motor cinemático y navegación de un robot hexápodo en ROS 2

Josué Bohórquez Cristóbal

Tutor: D. Álvaro Gutiérrez Martín

2026

Resumen

Número máximo de palabras: 500

...

Summary

Maximum number of words: 500

...

Palabras Clave

Deben reflejar el contenido del trabajo, deberían servir para localizar el TFG mediante búsqueda bibliográfica...

Keywords

...

Agradecimientos

...

Índice

Resumen y Palabras Clave II

Agradecimientos III

Lista de acrónimos VIII

1. Introducción y objetivos **1**
1.1. Introducción . . . 1
1.2. Objetivos . . . 2
1.3. Estructura del documento . . . 2

2. Estado del arte **3**
2.1. Origen y evolución de los robots hexápodos . . . 3
2.2. Plataformas hexápodo en la investigación actual . . . 5
2.3. Servomotores . . . 6
2.4. ROS2 como arquitectura de integración robótica . . . 7
2.5. Trabajos previos en control de hexápodos . . . 8

3. Marco Teórico y Fundamentos **9**
3.1. Descripción espacial y transformaciones . . . 9
3.1.1. Posición y orientación . . . 9
3.1.2. Definición de sistema de coordenadas y transformaciones . . . 10
3.2. Cinemática directa del cuerpo . . . 12
3.2.1. Sistemas de coordenadas y notación . . . 12
3.2.2. Matriz de posición del cuerpo y actualización incremental . . . 14
3.2.3. Actualización de los efectores finales . . . 15
3.3. Modelado cinemático inverso . . . 17
3.3.1. Concepto de cinemática inversa . . . 17
3.3.2. Parámetros geométricos . . . 17
3.3.3. Resolución de la cinemática inversa (IK) . . . 18
3.4. Algoritmos de marcha y locomoción . . . 21
3.4.1. Estabilidad de la marcha . . . 21
3.4.2. Parámetros fundamentales de la marcha . . . 22
3.4.3. Patrones de marcha periódica . . . 23
3.5. Planificación de trayectorias . . . 24
3.5.1. Perfil de trayectoria de vuelo (swing phase) . . . 24
3.5.2. Suavizado de postura . . . 26

4. Plataforma hardware e implementación **28**
4.1. Arquitectura general del sistema . . . 28
4.2. Hardware de bajo nivel y control de actuadores . . . 30
4.2.1. Servomotores DYNAMIXEL AX-12A . . . 30
4.2.2. Bus DYNAMIXEL y protocolo de comunicación . . . 31

4.2.3. Mapa de memoria del AX-12A y registros utilizados 32
4.2.4. Electrónica de adaptación al bus *half-duplex* 34
4.2.5. Módulo de acceso a los servomotores (`Firmware AX12A`) 36
4.3. Implementación del firmware cinemático 37
4.3.1. Biblioteca de transformaciones espaciales (`matrix_math`) 38
4.3.2. Configuración de las patas (`leg_config`) 38
4.3.3. Cinemática del cuerpo (`body_kinematics/posture_engine`) 39
4.3.4. Generador de marcha (`gait_engine`) 40
4.3.5. Cinemática inversa de las patas (`leg_kinematics`) 41
4.3.6. Estado y comunicación (`robot_controller/serial_bridge`) 41
4.3.7. Locomoción y diagnóstico (`locomotion_task/diagnostic`) 42
4.4. Subsistema de alto nivel e integración ROS 2 43
4.4.1. Interfaz ROS 2 del puente serie 43
4.4.2. Panel web como cliente de la interfaz del hexápodo 45
4.4.3. Percepción y navegación reactiva 46

5. Validación, conclusiones y líneas futuras **48**
5.1. Validación funcional 48
5.2. Conclusiones 48
5.3. Líneas futuras 48

Bibliografía **49**

Anexo A: Aspectos éticos, económicos, sociales y ambientales **53**

Anexo B: Presupuesto económico **54**

Anexo C: Documentación técnica de la PCB principal **55**

Anexo D: Manual de puesta en marcha del sistema **57**

Anexo E: Capturas de la interfaz web **65**

Índice de figuras

1.1. PhantomX Mark III 1

2.1. Caracterización del patrón en insectos con marcha trípode 3
2.2. Robot ODEX I 4
2.3. Adaptive Suspension Vehicle (ASV) 4
2.4. Robot Genghis 4
2.5. Plataforma educativa Lynxmotion 5
2.6. PhantomX Hexapod Mark IV 5
2.7. Proyecto ZeroBug 5
2.8. Control PWM de servomotores 6
2.9. Arquitectura de servomotor digital 6
2.10. Esquema de comunicación Half-Duplex 6
2.11. Modelo de comunicación de ROS 7

3.1. Ejes de roll, pitch y yaw 9
3.2. Representación geométrica de un sistema de coordenadas 10
3.3. Concatenación de transformaciones 11
3.4. Sistemas de coordenadas y notación 12
3.5. Coordenadas de anclaje de las coxas (x_i, y_i) 13
3.6. Esquema de continuidad del ciclo de paso 16
3.7. Sistema local y estructura de la pata 17
3.8. Cálculo del ángulo de coxa 18
3.9. Reducción al mecanismo plano fémur–tibia 19
3.10. Esquema de ajuste y conversión a grados 20
3.11. Polígono de soporte y estabilidad estática 21
3.12. Trayectoria de la pata en un ciclo de marcha. 22
3.13. Marcha trípode: polígono y diagrama 23
3.14. Marcha de onda: polígono y diagrama 23
3.15. Marcha tetrapódica: polígono y diagrama 24
3.16. Perfil vertical sinusoidal cuadrático 25
3.17. Perfil horizontal cúbico 26
3.18. Perfil quíntico de suavizado 27

4.1. Arquitectura general de la plataforma hardware. 28
4.2. Montaje hardware del hexápodo: vista general y detalle de la electrónica integrada. 29
4.3. Actuador DYNAMIXEL AX-12A. 30
4.4. Topología de conexión en cadena (*daisy-chain*). 30
4.5. Distribución de bits del byte de error del DYNAMIXEL AX-12A. 32
4.6. Rango del AX-12A. 33
4.7. Perfil de par de salida en función del error de posición. 33
4.8. Esquema de la interfaz de adaptación al bus *half-duplex*. 34
4.9. PCB en 3D. 35

4.10. Encaminado en las capas externas de la PCB. 35
4.11. Captura de instrucción Read. 35
4.12. Posición y responsabilidades del módulo `ax12` 36
4.13. Organización del firmware del hexápodo 37
4.14. Organización de la biblioteca `matrix_math` 38
4.15. Organización del módulo `leg_config` 38
4.16. Suavizado de postura y cinemática del cuerpo 39
4.17. Actualización de cada pata en `gait_engine` 40
4.18. Cadena de conversión de `leg_kinematics` 41
4.19. Intercambio entre el puente serie y el estado compartido 41
4.20. Ciclo de locomoción como transacción 42
4.21. Planificación de la tarea de diagnóstico 42
4.22. Teleoperación mediante una interfaz ROS 2 estándar 44
4.23. Panel web como cliente de la interfaz ROS 2 45
4.24. LiDAR RPLIDAR A1M8 y su sistema de referencia. 46
4.25. Barrido del RPLIDAR A1M8 visualizado en RViz 2 46
4.26. Arquitectura en dos etapas del evitador de obstáculos. 46
4.27. Zonas y reglas del evitador reactivo 47

E.1. Vista general de la interfaz web 65
E.2. Mapa de diagnóstico de los servomotores 66
E.3. Acceso a los registros AX-12A 66

Índice de tablas

3.1. Coordenadas de la postura neutral (${}^{B}\boldsymbol{p}_{neu}$) configuradas para cada extremidad en el sistema de referencia del cuerpo. 15
3.2. Parámetros geométricos y de montaje 20

4.1. Mapa de asignación de IDs de los actuadores. 30
4.2. Instrucciones principales del protocolo DYNAMIXEL 1.0. 31
4.3. Mapa de memoria del DYNAMIXEL AX-12A (registros principales). . . . 32
4.4. Lógica de salidas del buffer. 34
4.5. Interfaz ROS 2 expuesta por el nodo `/esp32_bridge`. 43

B.1. Presupuesto económico 54

D.1. Identificadores de los AX-12A utilizados por el firmware. 57
D.2. Interfaces serie empleadas en el ESP32. 58
D.3. Comandos aceptados por la UART1. 59
D.4. Mensajes transmitidos por el firmware. 60
D.5. Dependencias declaradas por los paquetes ROS 2 del hexápodo. 61
D.6. Argumentos de `hexapod.launch.py`. 63
D.7. Límites de postura configurados en `web_panel.yaml`. 64

Lista de acrónimos

API *Application Programming Interface.*

CCW *Counterclockwise.*

CPG *Central Pattern Generator.*

CW *Clockwise.*

DC *Direct Current.*

DDS *Data Distribution Service.*

EEPROM *Electrically Erasable Programmable Read-Only Memory.*

ESP-IDF *Espressif IoT Development Framework.*

GPIO *General Purpose Input/Output.*

IK *Inverse Kinematics.*

IMU *Inertial Measurement Unit.*

LiDAR *Light Detection and Ranging.*

LiPo *Lithium Polymer.*

NUKE *Nearly Universal Kinematics Engine.*

PCB *Printed Circuit Board.*

PWM *Pulse Width Modulation.*

RAM *Random Access Memory.*

ROS *Robot Operating System.*

TFG Trabajo Fin de Grado.

TTL *Transistor-Transistor Logic.*

UART *Universal Asynchronous Receiver-Transmitter.*

1. Introducción y objetivos

1.1. Introducción

La locomoción con patas constituye una de las líneas de mayor interés dentro de la robótica móvil, principalmente por su capacidad para desplazarse sobre superficies irregulares, salvar obstáculos discretos y mantener la estabilidad en escenarios donde las plataformas con ruedas presentan limitaciones significativas [1]. En particular, los robots hexápodos (una configuración de seis patas) ofrecen una combinación especialmente atractiva de estabilidad estática, redundancia de apoyos y versatilidad cinemática, por lo que son idóneos para la investigación, la docencia y la experimentación.

Aprovechar esta capacidad exige, sin embargo, resolver problemas de control que abarcan distintos niveles de abstracción dentro de la robótica. Por un lado, la locomoción coordinada requiere modelos precisos, tanto de cinemática directa como inversa, así como la generación de trayectorias suaves y una ejecución determinista en tiempo real. Además, la operación dentro de un ecosistema robótico demanda mecanismos de comunicación, estimación de estado y compatibilidad con plataformas como ROS 2 [2]. Por este motivo, además de la estructura mecánica y los materiales empleados, la calidad de la arquitectura de control y la integración de software resultan fundamentales para el rendimiento global de la plataforma.



Figura 1.1: PhantomX Mark III.
Fuente: elaboración propia.

En este contexto, este Trabajo Fin de Grado (TFG) toma como punto de partida el kit *PhantomX Mark III* de Trossen Robotics (Figura 1.1), un robot hexápodo con dieciocho grados de libertad y estructura de aluminio, cuyo sistema original se basaba en una tarjeta ArbotiX [3] y en un firmware heredado de la librería Nuke [4]. Aunque esta solución proporciona una base funcional para el control básico del robot, presenta limitaciones relevantes en cómputo, modularidad del software e integración con herramientas actuales.

Para superar estas limitaciones en este TFG, se aborda la modernización integral de la plataforma mediante la sustitución del controlador por un microcontrolador ESP32, el rediseño de la electrónica de interfaz para el bus de comunicación con los motores y la reestructuración completa de la arquitectura software. Sobre esta base se desarrolla un motor cinemático y de marcha ejecutable en tiempo real, y se sientan las bases para su integración con una Raspberry Pi y el ecosistema ROS 2. De esta manera, se consigue una plataforma experimental que permite la investigación y el desarrollo de algoritmos de control, navegación y planificación de trayectorias en trabajos futuros.

1.2. Objetivos

De acuerdo con lo anterior, el objetivo general de este trabajo es modernizar la plataforma *PhantomX Mark III* para dotarla de una arquitectura hardware-software modular, capaz de ejecutar locomoción en tiempo real e integrarse con ROS 2, dejando preparada la plataforma para futuras capacidades de navegación.

1. **Rediseñar la capa de control de actuadores:** desarrollar el driver de bajo nivel para los servomotores DYNAMIXEL AX-12A sobre bus serie *half-duplex*, junto con la electrónica de adaptación y el diseño de la placa de circuito impreso (PCB).
2. **Formalizar el modelo cinemático del robot:** definir la cinemática inversa de cada extremidad e implementar el conjunto de transformaciones que relacionan las patas, el cuerpo y el sistema de referencia global.
3. **Implementar el motor de locomoción:** diseñar los algoritmos de coordinación de marcha y generación de trayectorias necesarios para su ejecución determinista en tiempo real sobre el microcontrolador.
4. **Integrar la plataforma en el ecosistema ROS 2:** establecer la comunicación entre el ESP32 y una Raspberry Pi, de forma que el robot pueda intercambiar comandos, estado y odometría con nodos del ecosistema ROS 2.

1.3. Estructura del documento

Con la intención de ofrecer una visión organizada de la memoria, se ha adoptado una estructura que progresa desde los fundamentos matemáticos necesarios para comprender la locomoción de la plataforma hasta las decisiones de diseño e implementación que concretan el proyecto. En consecuencia, el documento se estructura de la siguiente manera:

- **Capítulo 2, Estado del arte:** repasa el contexto y los trabajos previos en robots hexápodos y en el control de plataformas multípedas.
- **Capítulo 3, Marco teórico:** desarrolla la cinemática, las transformaciones y las estrategias de marcha en las que se apoya el motor de locomoción.
- **Capítulo 4, Plataforma hardware e implementación:** describe la electrónica, el software y la integración con ROS 2 desarrollados en el proyecto.
- **Capítulo 5, Validación, conclusiones y líneas futuras:** recoge las pruebas realizadas sobre el sistema, valora el cumplimiento de los objetivos y las limitaciones encontradas, y propone las posibles líneas de trabajo futuro.

2. Estado del arte

2.1. Origen y evolución de los robots hexápodos

El desarrollo de sistemas de locomoción mediante extremidades articuladas ha estado ligado a la robótica bioinspirada, tomando como referencia la morfología y los patrones de movimiento de animales e insectos. Desde el punto de vista de la estabilidad, uno de los trabajos fundacionales fue el de McGhee y Frank [5], quienes estudiaron las secuencias de marcha posibles en robots cuadrúpedos y demostraron que solo un subconjunto reducido garantizaba estabilidad estática. En particular, identificaron seis marchas con al menos tres apoyos, de las cuales solo tres eran estáticamente estables, y solo una maximizaba el margen, coincidiendo con patrones observados en cuadrúpedos en la naturaleza.

Estos principios matemáticos sentaron el estándar para el análisis de cualquier plataforma multípeda. En el caso de los hexápodos, esta teoría explica por qué presentan ventajas en terrenos irregulares: su redundancia mecánica les permite mantener polígonos de soporte mucho más amplios y seguros, como el clásico apoyo en trípode, superando las limitaciones que enfrentan los sistemas de cuatro patas. En esta misma línea, y motivados por la alta eficiencia biomecánica producto de la evolución, numerosos estudios se han dedicado a caracterizar los patrones de locomoción de los artrópodos. Estas investigaciones han revelado que la naturaleza no emplea un único patrón, sino que los insectos se adaptan dinámicamente en función de la velocidad de avance y las características del terreno. Para lograrlo, el sistema nervioso del insecto modula la marcha y el desfase temporal entre el movimiento de sus extremidades, un parámetro conocido como retraso metacronal, tal y como se ilustra en la Figura 2.1: la fila superior muestra la alternancia entre los dos trípodes de apoyo, {I2, D1, D3} y {I1, D2, I3}, y la fila inferior representa, con barras de apoyo y de vuelo, la duración de ese desfase en cada periodo de marcha.

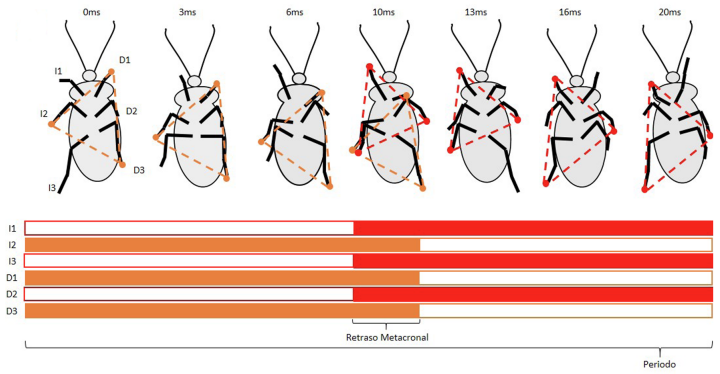


Figura 2.1: Caracterización del patrón en insectos con marcha trípode
Fuente: Fernández-Villalobos (2019) [6].

La traslación de estos principios de locomoción a plataformas electromecánicas supuso un desafío tecnológico. Durante la década de 1980, el desarrollo de robots multípedos experimentó avances fundamentales. Uno de los hitos fue el robot Odex I (ver Figura 2.2), presentado en 1983 [7]. Este sistema demostró empíricamente la superioridad de los mecanismos articulados de seis patas frente a los vehículos tradicionales de orugas o ruedas, especialmente en la superación de obstáculos geométricos complejos y la navegación en entornos no estructurados. No obstante, su viabilidad estaba restringida por una estructura física sumamente pesada.

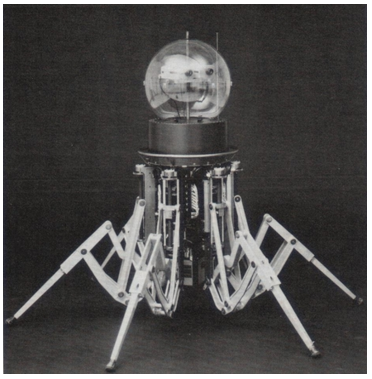


Figura 2.2: Robot ODEX I
Fuente: Russell (1983) [7].

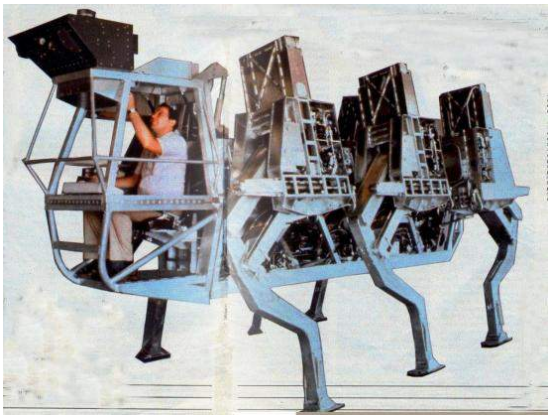


Figura 2.3: Adaptive Suspension Vehicle (ASV)
Fuente: Siciliano y Khatib (2016) [1].

Poco después, la aplicación a gran escala de las teorías de estabilidad estática se materializó con el *Adaptive Suspension Vehicle* (ASV) en 1986 (ver Figura 2.3). Resulta especialmente destacable que este proyecto estuviera coliderado por Robert McGhee, aplicando a una plataforma real los modelos matemáticos de locomoción formulados dos décadas atrás [1, 8]. Un sistema de tres toneladas y más de cinco metros de longitud, impulsado por motores de combustión y actuadores hidráulicos, que demostró la viabilidad de la marcha en trípode en un vehículo de escala humana, dependiente de sistemas de computación voluminosos.

A diferencia de los enfoques convencionales de la época, el MIT introdujo una aproximación alternativa con el robot *Genghis* (ver Figura 2.4), que demostró que era posible lograr una locomoción robusta y adaptativa mediante una red de reglas reactivas simples y descentralizadas [9]. En este modelo, cada extremidad operaba a partir de sensores básicos y reflejos locales, de forma análoga al procesamiento neuronal distribuido observado en los insectos. Así, cada pata podía interpretarse como una máquina de estados finitos local, capaz de coordinarse con las demás mediante eventos asíncronos.

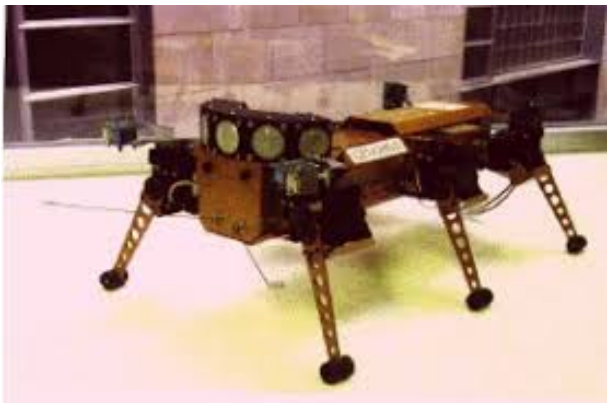


Figura 2.4: Robot *Genghis*
Fuente: Brooks (1989) [9].

A pesar de estos avances, el desarrollo de hexápodos seguía fuertemente condicionado por las restricciones del hardware disponible. Muchas de las primeras plataformas dependían de sistemas hidráulicos o neumáticos, lo que obligaba a operar conectadas a fuentes de alimentación o a incorporar pesados motores de combustión. Estas limitaciones reducían de forma significativa su autonomía, agilidad y viabilidad fuera de entornos experimentales.

2.2. Plataformas hexápodo en la investigación actual

El desarrollo de plataformas hexápodo en las últimas décadas ha trascendido los altos presupuestos de los años ochenta para consolidarse en un ecosistema accesible. La aparición de las impresoras 3D y el abaratamiento de la electrónica han revolucionado el DIY (Do It Yourself). Además, la resolución continua de los cálculos cinemáticos ya no constituye el principal impedimento, ya que los microcontroladores actuales pueden realizar esta tarea con solvencia. El reto radica en disponer de capacidad computacional suficiente para integrar sistemas de navegación autónoma.



Figura 2.5: Lynxmotion
Fuente: Lynxmotion [10].

Dentro del ámbito educativo destacan plataformas basadas en chasis acrílico, o impresión 3D, como los modelos ofrecidos por Freenove [11] o Lynxmotion [10] (ver Figura 2.5), que emplean servomotores. Estas soluciones son accesibles, sencillas de montar y adecuadas para introducir conceptos de locomoción multípeda. Sin embargo, no comunican información sensorial al controlador principal y carecen de la telemetría necesaria para implementar estrategias de control reactivo o diagnóstico avanzado. Para superar estas limitaciones, las plataformas orientadas a investigación suelen adoptar actuadores digitales y una arquitectura electrónica más rica.

En este ámbito destaca la familia *PhantomX* de Trossen Robotics, que ha evolucionado hasta el *PhantomX Mark IV* (ver Figura 2.6), una actualización orientada a investigación avanzada. Esta versión incorpora una Raspberry Pi para facilitar la integración con ROS 2 y actualiza los actuadores a nuevas generaciones de la serie X de DYNAMIXEL [13]. Aun así, numerosos trabajos continúan empleando el *PhantomX Mark III*, mencionado en la introducción, por su disponibilidad y por la amplia base de software y documentación.



Figura 2.6: PhantomX Mark IV
Fuente: Trossen Robotics [12].

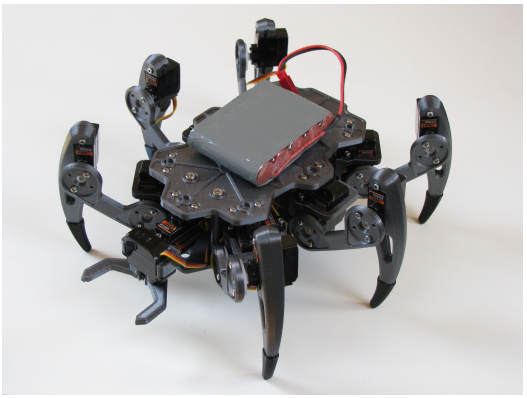


Figura 2.7: ZeroBug
Fuente: Hackaday.io [14].

También resultan relevantes algunos proyectos como ZeroBug [14] (ver Figura 2.7) y los hexápodos de Hiwonder [15], que exploran arquitecturas basadas en microcontroladores STM32/ESP32 y enfoques abiertos de desarrollo. En conjunto, estas iniciativas muestran cómo la frontera entre plataformas didácticas y sistemas de investigación se está reduciendo, permitiendo seleccionar una base mecánica accesible y complementarla con electrónica, software y capacidad de cómputo adaptadas a los objetivos del proyecto.

2.3. Servomotores

El control de los grados de libertad en plataformas robóticas requiere de servomotores que ofrezcan un equilibrio entre precisión, par mecánico y capacidad de integración. Los servomotores más populares son los controlados mediante una señal de modulación por ancho de pulsos (PWM), ya que son económicos y su implementación electrónica es sencilla.

Sin embargo, desde el punto de vista del controlador principal, estos presentan una limitación arquitectónica, ya que, aunque internamente disponen de un lazo cerrado de control de posición, dicha realimentación no sale del propio servomotor. El controlador se limita a enviar una onda cuadrada de alto nivel donde la duración del pulso fija la referencia angular, como se ilustra en la Figura 2.8, sin recibir a cambio ninguna información sensorial que confirme que el movimiento se haya realizado ni la posición real del eje. Esta ausencia de información sensorial hacia el controlador impide desarrollar un control de alto nivel o implementar rutinas de compensación dinámica que adapten la marcha del robot a las irregularidades del entorno. Para superar esta limitación, se recurre a los servomotores digitales. Estos dispositivos integran en un único encapsulado el motor de corriente continua, la electrónica de control, el sensor de posición y la interfaz de comunicación, como se muestra en la Figura 2.9.

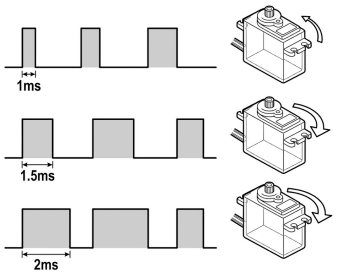


Figura 2.8: Control PWM de servomotores.
Fuente: Elaboración propia.

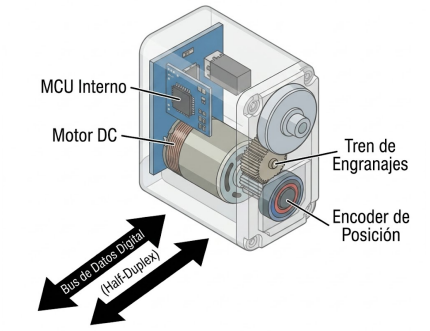


Figura 2.9: Arquitectura de servomotor digital.
Fuente: Elaboración propia.

Esta arquitectura, basada en un control digital de posición/velocidad, les permite operar mediante paquetes de comunicación digital bidireccional, sustituyendo las señales analógicas unidireccionales por un protocolo serie asíncrono que, en muchos casos comerciales, se implementa en configuración *half-duplex* [16]. Además, se pueden consultar variables críticas como la posición del eje, la carga mecánica aplicada, la temperatura interna, el voltaje de alimentación y posibles alarmas de error. Otro aspecto relevante es la simplificación del cableado. El protocolo *half-duplex* permite encadenar múltiples actuadores sobre una única línea de datos en topología *daisy-chain*, tal y como se ilustra en la Figura 2.10.

Esto elimina la necesidad de cablear individualmente cada motor al controlador, reduciendo drásticamente la complejidad y el peso del cableado, un factor especialmente crítico en robots móviles donde cada gramo de masa no útil penaliza la autonomía y la dinámica del sistema.



Figura 2.10: Comunicación *half-duplex*: transmisión bidireccional no simultánea sobre una única línea.
Fuente: Elaboración propia.

En el contexto de este proyecto, se han seleccionado los actuadores DYNAMIXEL AX-12A [17] como base motriz del hexápodo. Estos servomotores poseen todas las virtudes descritas y se comunican a través de un bus serie compartido que coordina de forma eficiente los 18 motores del sistema, tres por extremidad. Los detalles técnicos de sus capacidades mecánicas, protocolo de comunicación e integración hardware se abordan en profundidad en el Capítulo 4.

2.4. ROS2 como arquitectura de integración robótica

Robot Operating System (ROS) surgió como middleware de código abierto orientado a la robótica [18]. Se ha consolidado como estándar del sector por su modelo de comunicación basado en publicadores y suscriptores, su ecosistema de paquetes y su comunidad activa. ROS 1 presentaba limitaciones estructurales que se hicieron más evidentes a medida que los casos de uso evolucionaron. Destacaba su dependencia de un proceso centralizado llamado *rosmaster*, que actuaba como registro de nombres y punto de encuentro entre nodos. Esta arquitectura introducía un punto único de fallo para el descubrimiento y el registro. Si el proceso dejaba de estar disponible, las conexiones ya establecidas podían mantenerse, pero no era posible registrar nuevos nodos ni establecer nuevas conexiones. Además, utilizaba protocolos de transporte TCPROS/UDPROS [19] que no soportan de forma nativa la comunicación en tiempo real.

Para resolver estas limitaciones, ROS 2 sustituyó TCPROS/UDPROS por DDS (*Data Distribution Service*), un estándar del Object Management Group (OMG) [20]. ROS 2 elimina el *rosmaster* mediante un protocolo de descubrimiento descentralizado, de forma que cada nodo puede encontrar a los demás sin depender de un proceso central. Sin embargo, ROS 2 presenta una limitación importante: ejecutarse sobre un sistema operativo de propósito general como Linux no garantiza periodicidad estricta en el bucle de control, ya que el planificador puede introducir latencias no deterministas. Por ello, las arquitecturas habituales en robots combinan ROS 2 con un microcontrolador que ejecuta un sistema en tiempo real.

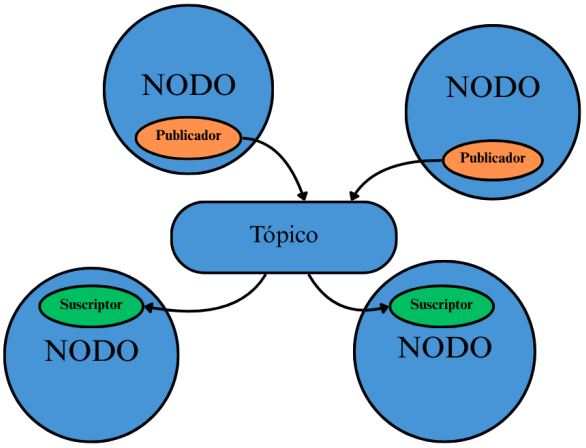


Figura 2.11: Modelo de comunicación de ROS basado en nodos, tópicos, publicadores y suscriptores.
Fuente: Elaboración propia.

El modelo de comunicación, basado en nodos, tópicos, publicadores y suscriptores, se resume en la Figura 2.11. ROS 2 ofrece un ecosistema completo para robots móviles, con paquetes de navegación como Nav2 [21], que proporciona planificación de rutas, evitación de obstáculos, localización y control de trayectorias, pudiendo emplearse con distintas cinemáticas siempre que el robot se abstraiga como una base móvil controlable. Para la percepción, `slam_toolbox` [22] permite construir mapas en tiempo real mediante LiDAR, mientras que `robot_localization` [23] fusiona fuentes de odometría e IMU. La visualización se realiza con RViz2 [24], que permite monitorizar el estado del robot, los tópicos activos y la nube de puntos del sensor en tiempo real. Para la simulación, Gazebo [25] integra físicas, sensores virtuales y modelos, permitiendo validar algoritmos antes de desplegarlos en hardware real. En conjunto, estas herramientas convierten a ROS 2 en la plataforma para integrar un hexápodo como nodo dentro de un sistema robótico completo, donde el control de bajo nivel del microcontrolador se expone al resto del ecosistema.

2.5. Trabajos previos en control de hexápodos

El punto de partida natural para controlar el *PhantomX* es el software que la propia empresa Trossen Robotics distribuye para la plataforma: el *PhantomX Hexapod NUKE Sketch*, construido sobre NUKE (*Nearly Universal Kinematics Engine*) dentro del ecosistema ArbotiX/PyPose [4]. Durante años ha sido la forma más rápida de poner un *PhantomX* a caminar sobre la placa ArbotiX, y buena parte de su éxito como robot educativo se apoya en él. El inconveniente es que su diseño ha envejecido: la cinemática del cuerpo se resuelve con relaciones escalares en lugar de transformaciones homogéneas, la fase de vuelo de la pata sigue un perfil triangular sin suavizado, y la generación de marcha, la cinemática y la comunicación con los motores conviven en un mismo bloque monolítico difícil de mantener o portar. Además, depende de la librería BioloidController [26], desarrollada por Vanadium Labs para la ArbotiX, que no tiene port oficial en ESP-IDF [27], el entorno de este proyecto.

Pero NUKE es solo una manera de resolver el problema, y el control de hexápodos se ha abordado desde enfoques muy distintos. Uno de ellos es el bioinspirado, que no parte de un modelo geométrico del robot, sino que deja que la marcha emerja de la interacción entre osciladores y reflejos locales. Su versión más conocida son los generadores centrales de patrones (*Central Pattern Generators*, CPG) [28], redes de osciladores que imitan los circuitos neuronales de la locomoción animal y producen marchas estables con muy pocos parámetros. El robot *Genghis* del MIT es el ejemplo clásico, con cada pata coordinándose con las demás sin un planificador central [9]. Son enfoques robustos y adaptables, aunque con ellos cuesta seguir trayectorias precisas o razonar sobre la postura del cuerpo.

El enfoque contrario, y el más habitual cuando se busca precisión, es el control basado en modelo. Aquí la posición de cada pata se calcula a partir de una descripción cinemática rigurosa, normalmente con transformaciones homogéneas que encadenan los sistemas de referencia del cuerpo, las caderas y los eslabones [29]. Sobre esa base se planifican el apoyo, el vuelo y la trayectoria del pie. Dentro de esta misma línea existen controladores generalistas, pensados para reutilizarse en muchas plataformas. El más representativo es OpenSHC [30], integrado en ROS y con una separación limpia entre cinemática, postura y marcha. Su punto débil es que se apoya en cinemática inversa numérica y en una capacidad de cómputo que no encaja con un microcontrolador sujeto a requisitos de tiempo real.

Ambas opciones presentan limitaciones evidentes para este proyecto: por un lado, NUKE presenta una arquitectura muy acoplada y depende de una librería sin port oficial para ESP-IDF, por otro, OpenSHC está orientado a equipos con mayor capacidad de cómputo y emplea cinemática inversa numérica. Ninguno de los dos reunía al mismo tiempo la formulación geométrica, la ejecución determinista en el ESP32 y la modularidad que se buscaban para el proyecto. En resumen, ante estas alternativas, en este TFG se optó por desarrollar un controlador propio en lugar de adaptar uno existente.

3. Marco Teórico y Fundamentos

3.1. Descripción espacial y transformaciones

La capacidad de definir la posición y orientación de objetos en el espacio tridimensional es esencial para controlar las extremidades y el cuerpo del robot. Para ello, se emplean sistemas de coordenadas y operaciones de álgebra lineal [29, 1].

3.1.1. Posición y orientación

La posición de un punto en el espacio, expresada respecto a un sistema de referencia $\{A\}$, se describe mediante un vector de posición de dimensión 3×1. Sus componentes corresponden a las proyecciones ortogonales del punto sobre los ejes principales del sistema:

$$ {}^{A}\boldsymbol{P} = \begin{bmatrix} {}^{A}P_x & {}^{A}P_y & {}^{A}P_z \end{bmatrix}^T \tag{3.1} $$

Sin embargo, para definir unívocamente el estado de un cuerpo rígido en el espacio, la posición no es suficiente; es necesario especificar también su orientación. Para ello se emplea la **matriz de rotación**, cuyos vectores columna representan los vectores unitarios de los ejes del sistema rotado $\{B\}$ expresados en la base del sistema de referencia $\{A\}$:

$$ {}^{A}\boldsymbol{R}_B = \begin{bmatrix} {}^{A}\hat{X}_B & {}^{A}\hat{Y}_B & {}^{A}\hat{Z}_B \end{bmatrix} = \begin{bmatrix} \hat{X}_B \cdot \hat{X}_A & \hat{Y}_B \cdot \hat{X}_A & \hat{Z}_B \cdot \hat{X}_A \\ \hat{X}_B \cdot \hat{Y}_A & \hat{Y}_B \cdot \hat{Y}_A & \hat{Z}_B \cdot \hat{Y}_A \\ \hat{X}_B \cdot \hat{Z}_A & \hat{Y}_B \cdot \hat{Z}_A & \hat{Z}_B \cdot \hat{Z}_A \end{bmatrix} \tag{3.2} $$

Dado que ${}^{A}\boldsymbol{R}_B$ contiene nueve elementos pero solo tres grados de libertad, resulta conveniente parametrizarla mediante tres ángulos. En este TFG se adopta la convención de ángulos fijos X-Y-Z (*roll-pitch-yaw*) de la Figura 3.1, que compone las rotaciones elementales en orden inverso ($Z \to Y \to X$) [29]:

$$ {}^{A}\boldsymbol{R}_B = R_z(\alpha)\, R_y(\beta)\, R_x(\gamma) \tag{3.3} $$

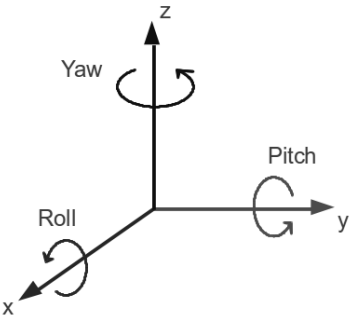


Figura 3.1: Ejes de rotación roll, pitch y yaw sobre un cuerpo rígido.
Fuente: adaptado de Aldebaran Robotics [31].

En la Ec. 3.3, α, β y γ son los ángulos de *yaw*, *pitch* y *roll*, es decir, los giros en torno a los ejes Z, Y y X, respectivamente. Desarrollando cada rotación elemental, la matriz resulta:

$$ {}^{A}\boldsymbol{R}_{B} = \underbrace{\begin{bmatrix} \cos\alpha & -\sin\alpha & 0 \\ \sin\alpha & \cos\alpha & 0 \\ 0 & 0 & 1 \end{bmatrix}}_{R_z(\alpha)} \underbrace{\begin{bmatrix} \cos\beta & 0 & \sin\beta \\ 0 & 1 & 0 \\ -\sin\beta & 0 & \cos\beta \end{bmatrix}}_{R_y(\beta)} \underbrace{\begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\gamma & -\sin\gamma \\ 0 & \sin\gamma & \cos\gamma \end{bmatrix}}_{R_x(\gamma)} \tag{3.4} $$

3.1.2. Definición de sistema de coordenadas y transformaciones

La combinación de posición y orientación permite definir el concepto de sistema de coordenadas, la entidad fundamental en robótica para describir la ubicación de un eslabón o cuerpo. Geométricamente, un sistema de coordenadas $\{B\}$ queda completamente definido respecto a $\{A\}$ mediante el par ordenado:

$$ \{B\} \text{ respecto a } \{A\} \equiv \left({}^{A}\boldsymbol{R}_{B},\ {}^{A}\boldsymbol{P}_{B} \right) \tag{3.5} $$

donde ${}^{A}\boldsymbol{R}_{B}$ es la orientación de $\{B\}$ respecto a $\{A\}$ y ${}^{A}\boldsymbol{P}_{B}$ es el vector de posición del origen de $\{B\}$ expresado en $\{A\}$. Esta estructura permite transformar las coordenadas de cualquier punto $\boldsymbol{P}$ descrito en el sistema local $\{B\}$ al sistema de referencia global $\{A\}$ mediante la ecuación fundamental de transformación de cuerpo rígido (Figura 3.2):

$$ {}^{A}\boldsymbol{P} = {}^{A}\boldsymbol{R}_{B}\,{}^{B}\boldsymbol{P} + {}^{A}\boldsymbol{P}_{B} \tag{3.6} $$

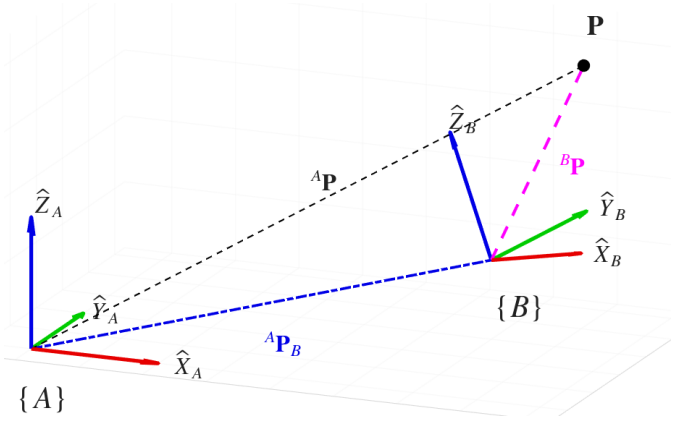


Figura 3.2: Relación espacial entre dos sistemas de coordenadas. La ecuación describe la transformación del punto P del sistema $\{B\}$ al sistema global $\{A\}$.
Fuente: Elaboración propia mediante MATLAB [32].

Esto se puede expresar de forma más compacta mediante el uso de la matriz de transformación homogénea que describe el sistema $\{B\}$ respecto a $\{A\}$ [29, 1] como:

$$ {}^{A}\mathbf{T}_{B} = \begin{bmatrix} {}^{A}\boldsymbol{R}_{B} & {}^{A}\boldsymbol{P}_{B} \\ \mathbf{0}_{1\times3} & 1 \end{bmatrix} \tag{3.7} $$

De esta forma, la transformación fundamental de la Ec. 3.6 se escribe en forma homogénea como:

$$ {}^{A}\tilde{\boldsymbol{P}} = {}^{A}\mathbf{T}_{B}\,{}^{B}\tilde{\boldsymbol{P}} \tag{3.8} $$

donde los vectores de posición se expresan en coordenadas homogéneas de dimensión 4×1, añadiendo una cuarta componente unitaria ($\tilde{\boldsymbol{P}} = [\boldsymbol{P}^T \;\; 1]^T$). De esta forma, al desarrollar la Ec. 3.8 se recupera directamente la expresión en $\mathbb{R}^3$:

$$ {}^{A}\tilde{\boldsymbol{P}} = \begin{bmatrix} {}^{A}\boldsymbol{P} \\ 1 \end{bmatrix} = \begin{bmatrix} {}^{A}\boldsymbol{R}_B \, {}^{B}\boldsymbol{P} + {}^{A}\boldsymbol{P}_B \\ 1 \end{bmatrix} \tag{3.9} $$

Propiedades algebraicas

La ventaja principal de esta representación es la capacidad de **concatenar** transformaciones relativas mediante multiplicación matricial [29, 1]. Si se conocen ${}^{A}\mathbf{T}_B$ y ${}^{B}\mathbf{T}_C$, entonces:

$$ {}^{A}\mathbf{T}_C = {}^{A}\mathbf{T}_B \, {}^{B}\mathbf{T}_C = \begin{bmatrix} {}^{A}\boldsymbol{R}_B \, {}^{B}\boldsymbol{R}_C & {}^{A}\boldsymbol{R}_B \, {}^{B}\boldsymbol{P}_C + {}^{A}\boldsymbol{P}_B \\ \mathbf{0}_{1\times3} & 1 \end{bmatrix} \tag{3.10} $$

Es importante observar que la traslación de la segunda transformación, ${}^{B}\boldsymbol{P}_C$, está expresada en $\{B\}$, por lo que debe rotarse mediante ${}^{A}\boldsymbol{R}_B$ para expresarla en $\{A\}$ antes de sumarle ${}^{A}\boldsymbol{P}_B$, tal y como se ilustra en la Figura 3.3.

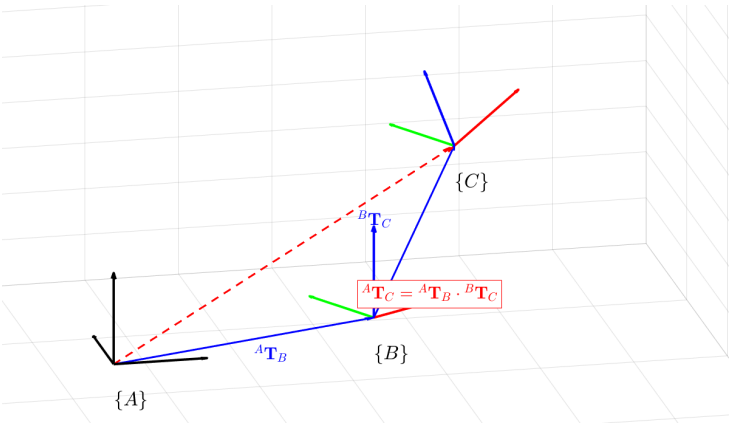


Figura 3.3: Concatenación de transformaciones espaciales. La transformación global ${}^{A}\mathbf{T}_C$ puede obtenerse mediante la composición de las transformaciones intermedias.
Fuente: Elaboración propia.

Esto es esencial en robótica, donde es habitual encadenar sistemas de coordenadas para obtener la posición final. También es frecuente transformar en sentido contrario (por ejemplo, de $\{A\}$ a $\{B\}$). La inversa de una transformación homogénea tiene forma cerrada:

$$ ({}^{A}\mathbf{T}_B)^{-1} = {}^{B}\mathbf{T}_A = \begin{bmatrix} ({}^{A}\boldsymbol{R}_B)^T & -({}^{A}\boldsymbol{R}_B)^T \, {}^{A}\boldsymbol{P}_B \\ \mathbf{0} & 1 \end{bmatrix} \tag{3.11} $$

donde se ha utilizado la propiedad de ortogonalidad de las matrices de rotación:

$$ ({}^{A}\boldsymbol{R}_B)^{-1} = ({}^{A}\boldsymbol{R}_B)^T \tag{3.12} $$

Esta propiedad simplifica la implementación: para invertir una rotación basta con trasponer la matriz, sin necesidad de inversión numérica, lo que reduce la carga computacional en cada iteración de control.

3.2. Cinemática directa del cuerpo

En un hexápodo controlado mediante posiciones de extremo (*endpoints*), la postura del cuerpo fija de forma directa la posición relativa de las patas y, por tanto, los objetivos cartesianos que cada una debe alcanzar. Por ello, la cinemática del cuerpo debe formularse de manera coherente con la cinemática de las patas, lo que exige modelar la posición del chasis respecto al sistema global y describir su evolución temporal.

3.2.1. Sistemas de coordenadas y notación

Se emplean tres sistemas de referencia principales:

- $\{G\}$: sistema global (GLOBAL), fijo al terreno, alineado verticalmente
- $\{B\}$: sistema del cuerpo (BODY), solidario al chasis.
- $\{L_i\}$: sistema local de la pata i (LEG), ubicado en la articulación de la coxa, el comienzo de la extremidad.

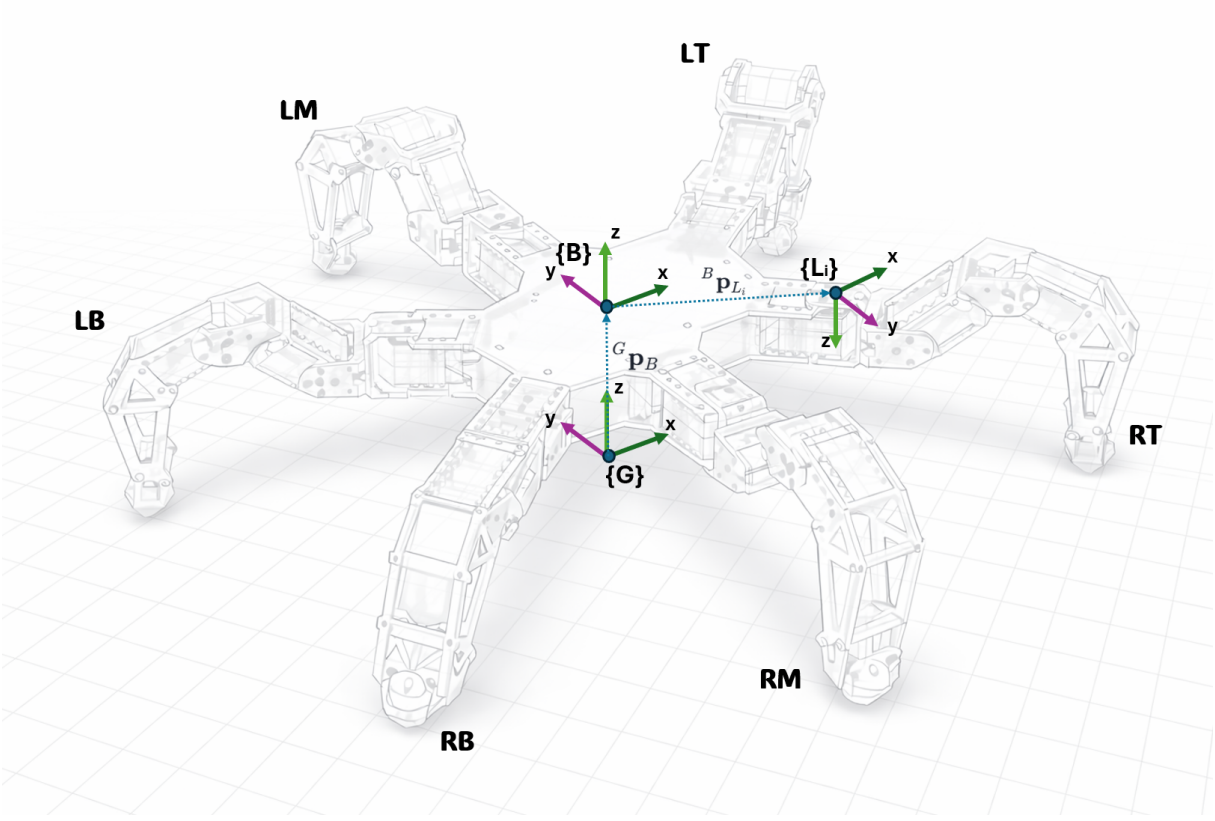


Figura 3.4: Descripción de los sistemas de coordenadas principales empleados en el modelado.
Fuente: Elaboración propia.

- ${}^{G}\boldsymbol{p}_{B}$: vector desde el origen de $\{G\}$ al origen de $\{B\}$, expresado en coordenadas de $\{G\}$.
- ${}^{B}\boldsymbol{p}_{L_i}$: vector desde el origen de $\{B\}$ al origen de $\{L_i\}$, expresado en coordenadas de $\{B\}$ (offset fijo del anclaje de la pata i).

Para expresar el punto objetivo desde $\{G\}$ en el sistema local de pata $\{L_i\}$ se aplican dos transformaciones consecutivas: (i) la transformación GLOBAL → BODY, y (ii) una transformación BODY → LEG específica de cada pata.

La transformación GLOBAL → BODY permite expresar un punto conocido en el sistema global $\{G\}$ en coordenadas del cuerpo $\{B\}$. Dado que la posición del cuerpo respecto al mundo se describe por ${}^{G}\mathbf{T}_{B}$, la transformación buscada se obtiene directamente como su inversa:

$$ {}^{B}\boldsymbol{P} = \left({}^{G}\mathbf{T}_{B}\right)^{-1} \, {}^{G}\boldsymbol{P}. \tag{3.13} $$

La transformación BODY → LEG se define mediante una traslación (x_j, y_j) y una inversión de ejes Y y Z (consistente con la orientación de los ejes mostrada en la Figura 3.4).

$$ {}^{L_i}\boldsymbol{P} = \underbrace{\begin{bmatrix} 1 & 0 & 0 & x_j \\ 0 & -1 & 0 & y_j \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}}_{{}^{L_i}\mathbf{T}_{B}} {}^{B}\boldsymbol{P}. \tag{3.14} $$

Los parámetros (x_j, y_j) son constantes para cada pata y se obtienen a partir de la posición del eje de la coxa (origen de $\{L_i\}$) respecto al centro del cuerpo, expresada en $\{B\}$. Esta convención se ha elegido por simetría y simplicidad. En la Figura 3.5 se ilustra junto con las medidas del cuerpo del hexápodo utilizado. En ella, cada pata se identifica con dos letras: la primera indica el lado (L, izquierda; R, derecha) y la segunda su posición en la vista superior (T, delantera; M, media; B, trasera, de *Top*, *Middle* y *Bottom*).

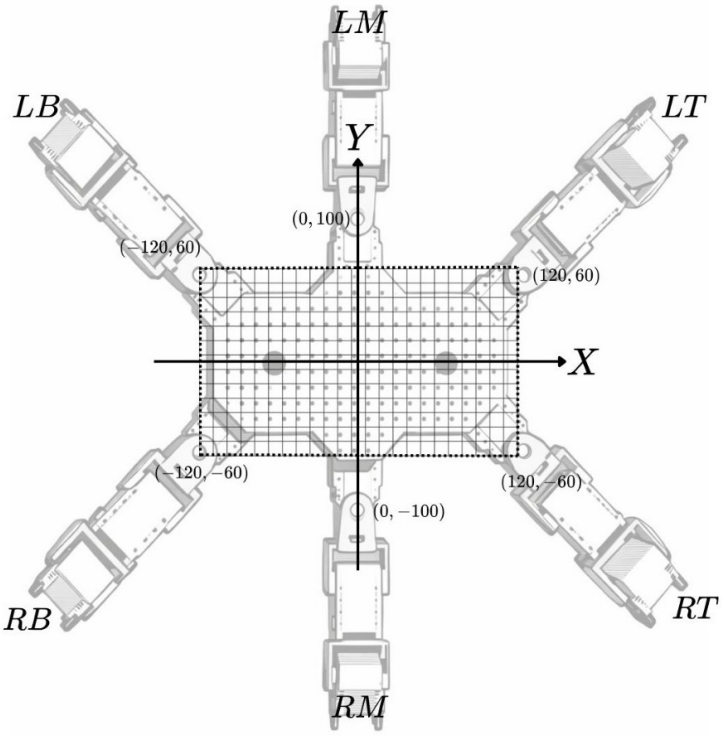


Figura 3.5: Ubicación de los puntos de anclaje de las coxas en el sistema de coordenadas del cuerpo $\{B\}$, expresada mediante las coordenadas planas (x_i, y_i).
Fuente: Elaboración propia.

Como ejemplo, si para la pata *Left-Top* (LT) se tiene:

$$ {}^{B}\boldsymbol{p}_{\text{coxa,LT}} = \begin{bmatrix} 120 \\ 60 \\ 0 \end{bmatrix} \tag{3.15} $$

en el sistema local debe cumplirse ${}^{L_{LT}}\boldsymbol{p}_{\text{coxa,LT}} = \mathbf{0}$. Aplicando la Ec. 3.14 a ese punto:

$$ \mathbf{0} = \begin{bmatrix} 120 + x_j \\ -60 + y_j \\ 0 \end{bmatrix} \Rightarrow \begin{array}{l} x_j = -120, \\ y_j = +60. \end{array} \tag{3.16} $$

De forma general, si la posición del eje de la coxa de la pata i en coordenadas del cuerpo es ${}^{B}\boldsymbol{p}_{\text{coxa},i} = [x_i \;\; y_i \;\; 0]^T$, imponer ${}^{L_i}\boldsymbol{p}_{\text{coxa},i} = \mathbf{0}$ en la Ec. 3.14 conduce a:

$$ \begin{aligned} x_j &= -x_i, \\ y_j &= +y_i. \end{aligned} \tag{3.17} $$

3.2.2. Matriz de posición del cuerpo y actualización incremental

La formulación cinemática adoptada en esta subsección sigue el enfoque de control por efectores finales propuesto por Arrigoni et al. [33], expresada aquí con la notación empleada en Craig [29]. El cuerpo en el espacio global se describe mediante la transformación homogénea ${}^{G}\mathbf{T}_{B}$, la cual relaciona las coordenadas del sistema del cuerpo $\{B\}$ con el sistema global $\{G\}$. Esta matriz se actualiza recursivamente en cada iteración i, integrando dos componentes: (i) el movimiento acumulado por el generador de marcha (Secciones 3.4 y 3.5), y (ii) las consignas de posición (traslación y rotación) definidas por el usuario.

En el instante inicial ($i = 0$), la transformación global-cuerpo se define como una traslación pura en el eje Z, situando al robot a su altura inicial SP_z:

$$ {}^{G}\mathbf{T}_{B,0} = \begin{bmatrix} \mathbf{I}_{3\times3} & {}^{G}\boldsymbol{p}_{B,0} \\ \mathbf{0}_{1\times3} & 1 \end{bmatrix}, \quad \text{con} \quad {}^{G}\boldsymbol{p}_{B,0} = \begin{bmatrix} 0 \\ 0 \\ SP_z \end{bmatrix}. \tag{3.18} $$

En cada iteración i, la matriz de estado ${}^{G}\mathbf{T}_{B,i}$ se construye mediante la composición de transformaciones:

$$ {}^{G}\mathbf{T}_{B,i} = \mathbf{T}_{tr,i} \cdot \mathbf{M}_{ov,i} \cdot (\mathbf{R}_{z,i}\,\mathbf{R}_{y,i}\,\mathbf{R}_{x,i})\,. \tag{3.19} $$

Aquí, $\mathbf{T}_{tr,i}$ representa la localización odométrica del robot en la iteración i: la posición en el plano, la orientación en *yaw* y la altura base que resultan de acumular, desde el instante inicial, los desplazamientos producidos por la marcha. Se calcula recursivamente aplicando el incremento de movimiento $\mathbf{Q}_{tr,i}$ calculado por el generador de marcha:

$$ \mathbf{T}_{tr,i} = \mathbf{T}_{tr,i-1}\,\mathbf{Q}_{tr,i}, \tag{3.20} $$

donde el incremento $\mathbf{Q}_{tr,i}$ se descompone en una traslación y una rotación diferencial en el eje vertical. Por tanto, $\mathbf{Q}_{tr,i}$ no representa la posición completa del cuerpo, sino únicamente la aportación incremental de la marcha a la trayectoria acumulada:

$$ \mathbf{Q}_{tr,i} = \underbrace{\begin{bmatrix} 1 & 0 & 0 & \delta x_{gait,i} \\ 0 & 1 & 0 & \delta y_{gait,i} \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}}_{\delta\mathbf{T}_{mov,i}} \cdot \underbrace{\begin{bmatrix} \cos(\delta rot_{z_i}) & -\sin(\delta rot_{z_i}) & 0 & 0 \\ \sin(\delta rot_{z_i}) & \cos(\delta rot_{z_i}) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}}_{\delta\mathbf{R}_{z,i}} \tag{3.21} $$

Por otra parte, la posición del cuerpo se gestiona componiendo la traslación relativa $\mathbf{M}_{ov,i}$ y la secuencia de rotaciones Euler ZYX:

$$ \mathbf{M}_{ov,i} \cdot (\mathbf{R}_{z,i}\,\mathbf{R}_{y,i}\,\mathbf{R}_{x,i}) = \left[\begin{array}{c|c} \mathbf{R}_z(\alpha_i)\,\mathbf{R}_y(\beta_i)\,\mathbf{R}_x(\gamma_i) & \begin{matrix} \text{bodyPos}_{x,i} \\ \text{bodyPos}_{y,i} \\ \text{bodyPos}_{z,i} \end{matrix} \\ \hline \mathbf{0}_{1\times3} & 1 \end{array}\right] \tag{3.22} $$

Sobre esta composición podría añadirse, además, una transformación de estabilización actualizada mediante retroalimentación sensorial.

Incremento de movimiento del cuerpo: $\mathbf{Q}_{body,i}$

Para actualizar la posición del efector final durante la fase de apoyo, no basta con conocer la posición absoluta del chasis; también es necesario cuantificar cómo cambia entre dos instantes de control consecutivos, $i-1$ y i. Siguiendo esta formulación, se define la transformación diferencial $\mathbf{Q}_{body,i}$, que expresa el sistema actual $\{B_i\}$ respecto al sistema anterior $\{B_{i-1}\}$ [33]:

$$\mathbf{Q}_{body,i} = \left({}^{G}\mathbf{T}_{B,i-1}\right)^{-1} \cdot {}^{G}\mathbf{T}_{B,i} \tag{3.23}$$

En otras palabras, $\mathbf{Q}_{body,i}$ representa el movimiento relativo completo del cuerpo entre dos iteraciones consecutivas. Esta magnitud es la que permite actualizar las posiciones de apoyo de forma recursiva e imponer la condición de no deslizamiento. Conviene distinguirla de $\mathbf{Q}_{tr,i}$: mientras esta última modela solamente el incremento odométrico de la marcha, $\mathbf{Q}_{body,i}$ recoge el cambio efectivo de la posición completa del chasis entre $i-1$ e i.

3.2.3. Actualización de los efectores finales

Fase de apoyo (*stance*)

Durante la fase de apoyo (*stance*), la hipótesis fundamental es que el pie no desliza sobre el terreno. Esto implica que la posición del efector final de la pata es constante en el sistema global $\{G\}$:

$${}^{G}\boldsymbol{p}_{foot,i} = {}^{G}\boldsymbol{p}_{foot,i-1} = \text{cte} \tag{3.24}$$

Sin embargo, el chasis sí se desplaza y rota entre iteraciones. Por tanto, aun cuando el efector final de la pata permanezca inmóvil en el sistema global $\{G\}$, sus coordenadas expresadas en el sistema del cuerpo $\{B\}$ cambian, porque el propio sistema $\{B\}$ se ha movido. Bajo esta hipótesis, la actualización recursiva de los efectores finales en apoyo se expresa como [33]:

$${}^{B}\boldsymbol{p}_{foot,i} = \left(\mathbf{Q}_{body,i}\right)^{-1} \cdot {}^{B}\boldsymbol{p}_{foot,i-1} \tag{3.25}$$

Al tratarse de una relación recursiva, es necesario definir la condición inicial ${}^{B}\boldsymbol{p}_{foot,0}$ para iniciar el proceso iterativo. La condición inicial del movimiento se toma en la **postura neutral**, definida en la Tabla 3.1; no debe confundirse con el inicio de cada fase de apoyo, cuya posición viene dada por el final del vuelo anterior.

Tabla 3.1: Coordenadas de la postura neutral (${}^{B}\boldsymbol{p}_{neu}$) configuradas para cada extremidad en el sistema de referencia del cuerpo.

Pata	**X [mm]**	**Y [mm]**	**Z [mm]**
Izquierda Delantera (LT)	+180	+170	−100
Izquierda Media (LM)	0	+240	−100
Izquierda Trasera (LB)	−180	+170	−100
Derecha Trasera (RB)	−180	−170	−100
Derecha Media (RM)	0	−240	−100
Derecha Delantera (RT)	+180	−170	−100

Fase de transferencia (*swing*) y continuidad del ciclo

Durante la fase de transferencia (*swing*), el pie se eleva y se desplaza hasta el siguiente punto de apoyo. Este movimiento se describe en el sistema de trayectoria $\{tr\}$, cuya posición y orientación respecto a $\{G\}$ vienen dadas por $\mathbf{T}_{tr,i}$. El punto de despegue ${}^{tr}\boldsymbol{p}^{-}$ se obtiene expresando en este sistema la posición calculada del pie al finalizar el apoyo.

El paso $\boldsymbol{s}_{step}$ es el desplazamiento horizontal del cuerpo durante una fase de apoyo (Subsección 3.4.2). Si el pie aterriza medio paso por delante de la postura neutral, el avance del cuerpo durante el apoyo lo sitúa, respecto al chasis, medio paso por detrás de ella. Así, el recorrido queda centrado en la postura neutral, como muestra la Figura 3.6.

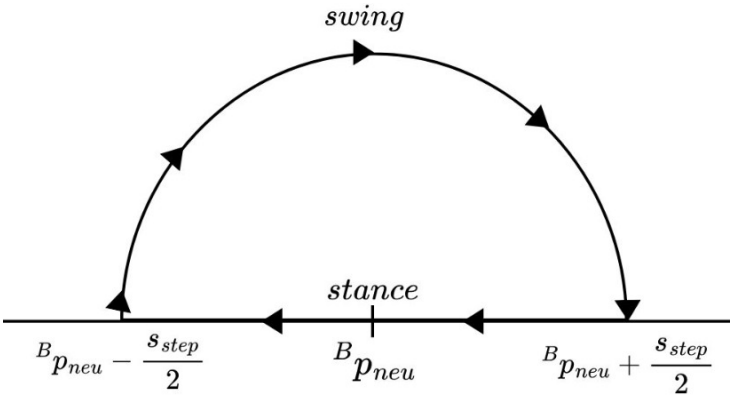


Figura 3.6: Ciclo del pie respecto al cuerpo con traslación constante.
Fuente: Elaboración propia.

Cuando la marcha combina traslación y giro, el destino se calcula como ${}^{tr}\boldsymbol{p}^{+} = \mathbf{R}_{gait}\mathbf{T}_{step}\,{}^{B}\boldsymbol{p}_{neu}$. La transformación $\mathbf{T}_{step}$ desplaza la posición neutral $\boldsymbol{s}_{step}/2$, y $\mathbf{R}_{gait}$ aplica a continuación una rotación alrededor del eje vertical igual a la mitad del giro previsto durante el apoyo. Las coordenadas de despegue y aterrizaje se almacenan en ese instante y se mantienen constantes durante el vuelo. Entre ellas se define la trayectoria:

$$ {}^{tr}\boldsymbol{p}_{traj}(u) = {}^{tr}\boldsymbol{p}^{-} + f_{xy}(u)\left({}^{tr}\boldsymbol{p}^{+} - {}^{tr}\boldsymbol{p}^{-}\right), \qquad u \in [0,1]. \tag{3.26} $$

La variable u representa el progreso del vuelo, desde el despegue ($u = 0$) hasta el aterrizaje ($u = 1$), y $f_{xy}(u)$ es la función de interpolación descrita en la Subsección 3.5.1. En cada iteración i, esta trayectoria se transforma al sistema global y se le añade la elevación vertical $\Delta z_{step}(u)$:

$$ {}^{G}\boldsymbol{p}_{swing,i} = \mathbf{T}_{tr,i}\,{}^{tr}\boldsymbol{p}_{traj}(u_i) + \Delta z_{step}(u_i)\,\hat{Z}_G. \tag{3.27} $$

El término $\Delta z_{step}(u)$, definido en la Subsección 3.5.1, proporciona la elevación del pie en la dirección vertical global $\hat{Z}_G$, anulándose al despegar y al aterrizar. Un futuro mapa de elevación $h({}^{G}p_x, {}^{G}p_y)$ permitiría ajustar la cota de aterrizaje y el perfil vertical de vuelo a la geometría del terreno.

Finalmente, la referencia se expresa en el sistema del cuerpo $\{B\}$:

$$ {}^{B}\boldsymbol{p}_{swing,i} = \left({}^{G}\mathbf{T}_{B,i}\right)^{-1}\,{}^{G}\boldsymbol{p}_{swing,i}. \tag{3.28} $$

La continuidad de posición entre fases se establece tomando como despegue el final del apoyo anterior y como inicio del siguiente apoyo el final del vuelo. Si la consigna de desplazamiento cambia durante la transferencia, los extremos almacenados en $\{tr\}$ no se recalculan, ya que el nuevo valor se incorpora al planificar el destino del siguiente vuelo.

3.3. Modelado cinemático inverso

3.3.1. Concepto de cinemática inversa

La cinemática inversa (IK, por sus siglas en inglés, *Inverse Kinematics*) aborda el problema de determinar la configuración articular $\boldsymbol{q}$ necesaria para alcanzar un objetivo cartesiano. En este TFG, el efector final se especifica únicamente por su posición $\boldsymbol{P}$ en el espacio. El proceso se denota como $\boldsymbol{q} = f^{-1}(\boldsymbol{P})$. A diferencia de la cinemática directa, este cálculo es no lineal, pudiendo presentar múltiples soluciones para un mismo punto objetivo [29, 1].

Si bien en manipuladores complejos es frecuente recurrir a métodos numéricos iterativos, estos resultan inadecuados para sistemas embebidos de tiempo real debido a su alto coste computacional y problemas de convergencia. Sin embargo, para cadenas cinemáticas de pocos grados de libertad, como la extremidad de este hexápodo (3-DOF), es posible obtener soluciones analíticas exactas mediante dos enfoques:

1. **Solución Algebraica:** Se basa en seguir la convención Denavit-Hartenberg, para despejar las variables articulares. Aunque riguroso, suele generar expresiones complejas y operaciones matriciales costosas.
2. **Solución Geométrica:** Aprovecha la implementación física del mecanismo para descomponer el problema espacial 3D en problemas planos 2D más sencillos, resolviéndolos mediante trigonometría. En este TFG se ha optado por este enfoque.

3.3.2. Parámetros geométricos

La pata se modela como una cadena cinemática abierta de tres eslabones rígidos (coxa, fémur y tibia), de longitudes constantes L_c, L_f y L_t. Para formular la cinemática inversa se define un sistema de referencia local de pata $\{L\}$ en la articulación de la coxa. La convención adoptada mantiene X_L alineado con X_B, sitúa Y_L perpendicular a X_L en el plano horizontal y con sentido opuesto a Y_B, y define Z_L positivo hacia abajo, en sentido opuesto a Z_B. Por tanto, Y_L no coincide necesariamente con la dirección radial de cada pata; su orientación particular respecto al chasis se incorpora mediante el ángulo fijo de montaje ε_i en el cálculo de la coxa. Esta configuración de 3 Grados de Libertad permite posicionar el efector final dentro del espacio de trabajo de la extremidad. La Figura 3.7 resume esta definición junto con las dimensiones nominales de los eslabones.

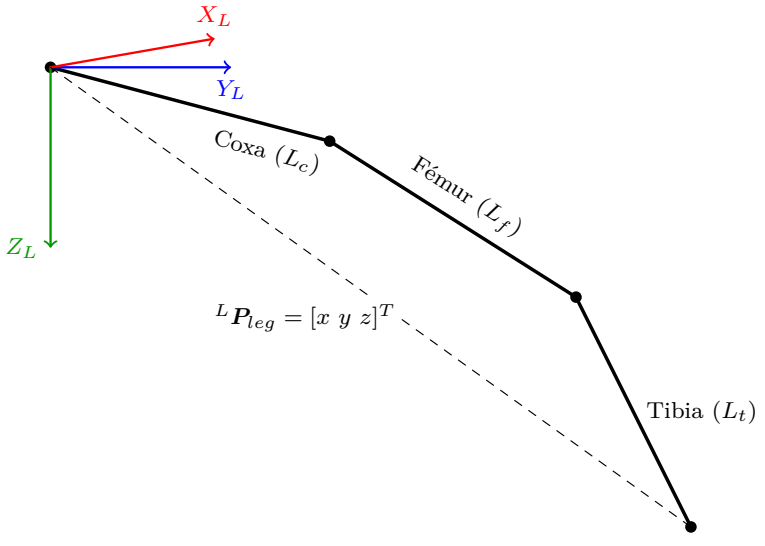


Figura 3.7: Sistema local de pata $\{L\}$ y longitudes L_c, L_f y L_t.

Fuente: Elaboración propia.

Longitudes nominales

Eslabón	L (mm)
Coxa	52
Fémur	66
Tibia	135

3.3.3. Resolución de la cinemática inversa (IK)

El objetivo de la Cinemática Inversa consiste en determinar el vector de variables articulares:

$$\boldsymbol{q} = \begin{bmatrix} \theta_{\text{coxa}} \\ \theta_{\text{femur}} \\ \theta_{\text{tibia}} \end{bmatrix} \tag{3.29}$$

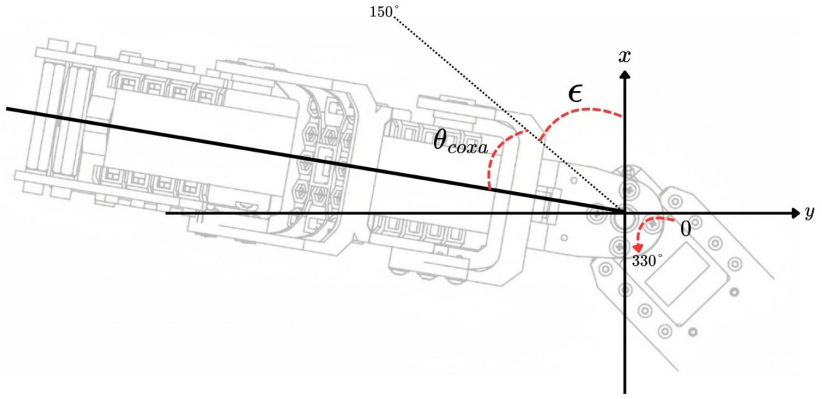
de manera que la punta de la pata alcance la coordenada objetivo ${}^{L}\boldsymbol{P}_{leg}$. Dado que los ejes de rotación del fémur y la tibia son paralelos entre sí, y ortogonales al eje de la coxa, el problema espacial 3D puede desacoplarse en: (i) un problema plano en el plano horizontal para la coxa, y (ii) un problema plano de dos eslabones en el plano vertical.

A continuación se presentan (y se deducen) las ecuaciones para la pata *Left-Top*. El resto de patas se obtienen aplicando la misma formulación y ajustando los offsets de montaje (traslaciones y el offset angular ε_i) propios de cada anclaje.

1) Cálculo del ángulo de Coxa (θ_{coxa}):

Sea el objetivo cartesiano expresado en el sistema local de pata $\{L\}$ como ${}^{L}\boldsymbol{P}_{leg} = [x\ y\ z]^T$. La orientación de la coxa viene determinada por la proyección de dicho objetivo sobre el plano horizontal $X_L Y_L$. Adoptando la convención de signos de la Figura 3.8, el ángulo se obtiene como:

$$\theta_{\text{coxa}} = -\operatorname{atan2}(y, x) - \varepsilon_i \tag{3.30}$$



Valores de ε_i

Pata	ε_i
LT	$+45°$
LM	$+90°$
LB	$+135°$
RT	$-45°$
RM	$-90°$
RB	$-135°$

Figura 3.8: Geometría usada para el cálculo del ángulo de coxa en el plano $X_L Y_L$

Fuente: Elaboración propia.

donde ε_i representa el **offset angular de montaje** de la pata, medido desde el eje longitudinal X_B del chasis y positivo en sentido antihorario visto desde arriba, es decir, hacia $+Y_B$. Se trata de un ángulo constante en cada pata; sus valores se recogen de forma resumida junto a la figura y, junto con los signos de giro y ajustes de montaje, en la Tabla 3.2. A diferencia de $\arctan\left(\frac{y}{x}\right)$, la función $\operatorname{atan2}(y, x)$ resuelve correctamente la ambigüedad de cuadrantes y cubre el rango completo $(-\pi, \pi]$, por lo que es la forma recomendada en la implementación.

Después de aplicar el offset ε_i, el resultado se normaliza de nuevo al intervalo $[-\pi, \pi]$ mediante atan2($\sin\theta_{\text{coxa}}, \cos\theta_{\text{coxa}}$). Esta operación no modifica la orientación de la articulación, pero evita representaciones equivalentes con vueltas completas, como 310° en lugar de −50°, que producirían una consigna incorrecta al añadir el offset del servomotor.

2) Cálculo de los ángulos de fémur y tibia (mecanismo plano fémur–tibia):

Una vez determinada la orientación de la coxa (θ_{coxa}), dado que los ejes de rotación del fémur y la tibia son paralelos entre sí y perpendiculares al plano de movimiento definido por la coxa, el sistema puede reducirse a un **mecanismo plano de dos eslabones**, como se muestra en la Figura 3.9:

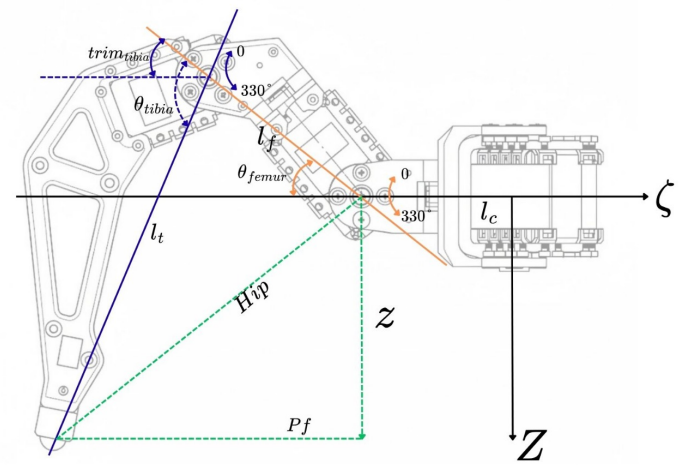


Figura 3.9: Geometría del mecanismo plano fémur–tibia.

Fuente: Elaboración propia.

La distancia horizontal desde la articulación del fémur hasta la proyección del efector final, denominada P_f, se obtiene restando la longitud de la coxa (L_c) a la magnitud radial total en el plano XY. Asimismo, se define la distancia directa (hipotenusa virtual) que conecta la articulación del hombro (fémur) con la punta de la pata, denotada como Hip:

$$P_f = \sqrt{x^2 + y^2} - L_c \qquad (3.31) \qquad\qquad Hip = \sqrt{P_f^2 + z^2} \qquad (3.32)$$

Conocidas las tres longitudes del triángulo (L_f del fémur, L_t de la tibia y la distancia virtual Hip), es posible obtener los ángulos articulares por el teorema del coseno.

Ángulo del fémur (θ_{femur}) : El ángulo objetivo del fémur se compone de dos partes: la elevación del vector Hip respecto a la horizontal y el ángulo interno del triángulo (α_1) adyacente a la articulación.

$$\theta_{\text{femur}} = \frac{\pi}{2} - \text{atan2}(P_f,\, z) - \arccos\left(\frac{L_f^2 + Hip^2 - L_t^2}{2 \cdot L_f \cdot Hip}\right) \qquad (3.33)$$

Ángulo de la tibia (θ_{tibia}) : El ángulo de la tibia determina la flexión de la rodilla y se deriva del ángulo interno (β) opuesto al lado Hip.

$$\theta_{\text{tibia}} = \pi - \arccos\left(\frac{L_f^2 + L_t^2 - Hip^2}{2 \cdot L_f \cdot L_t}\right) \qquad (3.34)$$

La tibia requiere además un ajuste propio: a diferencia de un modelo ideal, donde el ángulo cero corresponde a una extensión perfecta, el servo se monta de forma no colineal con el eje del eslabón, por lo que se introduce un offset angular de montaje que se incorpora en la Ec. 3.36.

Ajuste y transformación a grados

La IK geométrica proporciona ángulos articulares en radianes, referenciados al cero geométrico de cada articulación. Sin embargo, estos valores no pueden enviarse directamente a los servomotores: deben transformarse al sistema angular del actuador incorporando el sentido efectivo de giro y los ajustes de montaje. De forma esquemática, el proceso que conduce desde la posición objetivo del pie hasta la consigna articular final puede resumirse como:

$$ {}^{B}\boldsymbol{p}_{foot,i} \xrightarrow{\text{BODY}\to\text{LEG}} {}^{L_i}\boldsymbol{p}_{foot,i} \xrightarrow{\text{IK}} \begin{bmatrix} \theta_{\text{coxa}} \\ \theta_{\text{femur}} \\ \theta_{\text{tibia}} \end{bmatrix}_i \xrightarrow{\text{Ec. 3,36}} \begin{bmatrix} \theta_{cmd,c} \\ \theta_{cmd,f} \\ \theta_{cmd,t} \end{bmatrix}_i \tag{3.35} $$

De manera general, la transformación final puede escribirse como:

$$ \theta_{cmd} = \theta_0 + \text{trim} + s\,\theta_{IK} \tag{3.36} $$

donde θ_0 representa la referencia angular del actuador, $s \in \{+1, -1\}$ introduce el sentido efectivo de giro y trim compensa los desajustes derivados del montaje. En la configuración adoptada, coxa y fémur no requieren corrección adicional, mientras que la tibia incorpora un ajuste fijo de $\pm 45°$ asociado a su geometría de anclaje. No se establecen límites mecánicos particulares por articulación, ya que estos deben obtenerse experimentalmente para el montaje construido. La Figura 3.10 muestra el hexápodo completo con los sistemas de coordenadas definidos en todas las patas, y los valores concretos empleados en cada una se resumen en la Tabla 3.2.

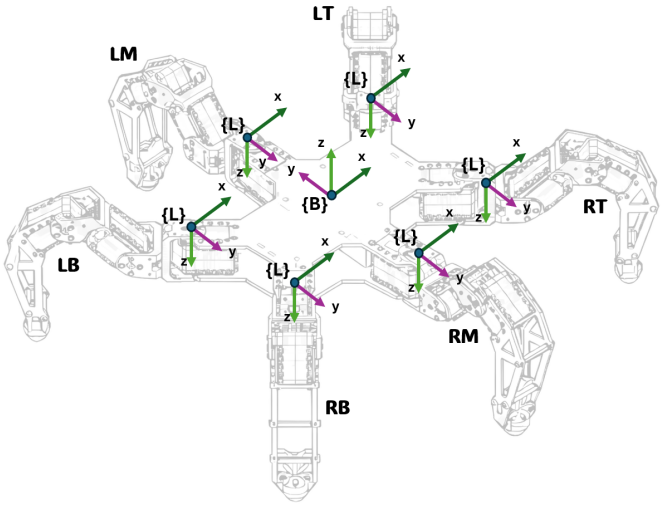


Figura 3.10: Esquema del hexápodo con los sistemas de coordenadas definidos en todas las patas.

Fuente: Elaboración propia.

Patas izquierdas

Pata	ε_i	signos	trim_t
LT	$+45°$	$(+1,+1,+1)$	$-45°$
LM	$+90°$	$(+1,+1,+1)$	$-45°$
LB	$+135°$	$(+1,+1,+1)$	$-45°$

Patas derechas

Pata	ε_i	signos	trim_t
RT	$-45°$	$(+1,-1,-1)$	$+45°$
RM	$-90°$	$(+1,-1,-1)$	$+45°$
RB	$-135°$	$(+1,-1,-1)$	$+45°$

Tabla 3.2: Parámetros geométricos y de montaje.

3.4. Algoritmos de marcha y locomoción

La locomoción mediante extremidades articuladas se fundamenta en la capacidad de levantar y reposicionar las patas en nuevas ubicaciones de manera coordinada. A esta secuencia ordenada de eventos de levantamiento y apoyo, diseñada para garantizar estabilidad, propulsión y eficiencia, se le denomina marcha o *gait*. Ciertas marchas son muy comunes en el imaginario diario (andar, trotar o galopar, en caballos, por ejemplo). Sin embargo, los insectos también representan una fuente muy amplia de patrones de marcha, y su estudio ha sido muy provechoso en la generación de marchas. Para un robot hexápodo, el espacio de posibles secuencias de eventos es inmenso, por lo que en la práctica se seleccionan patrones específicos inspirados en la naturaleza o diseñados matemáticamente para optimizar la estabilidad estática y la velocidad [1].

Las marchas se dividen generalmente en dos categorías principales según la regularidad de sus patrones: (i) **marchas periódicas**, en las que la secuencia de eventos se repite cíclicamente, y (ii) **marchas no periódicas** (*free gaits*), empleadas cuando es necesario adaptar la secuencia de apoyos al terreno o a maniobras complejas [1, 34].

3.4.1. Estabilidad de la marcha

El estudio de la estabilidad en robots andadores se apoya en los trabajos clásicos de McGhee y Frank [5], que establecieron una base matemática para analizar marchas periódicas y comparar su margen de estabilidad. En un hexápodo, disponer de al menos tres apoyos simultáneos constituye la condición geométrica mínima para definir un polígono de soporte. Sin embargo, esta condición por sí sola no basta, ya que, para que exista estabilidad, la proyección del centro de masas debe caer dentro de dicho polígono [1].

De forma más precisa, el **criterio de estabilidad estática** establece que el robot será estable mientras la proyección vertical de su centro de gravedad (P_{CoG}) quede contenida en el polígono convexo formado por las patas que se encuentran en fase de apoyo (véase la Figura 3.11). Esta formulación resulta especialmente útil porque permite evaluar la estabilidad de una marcha a partir de una interpretación puramente geométrica.

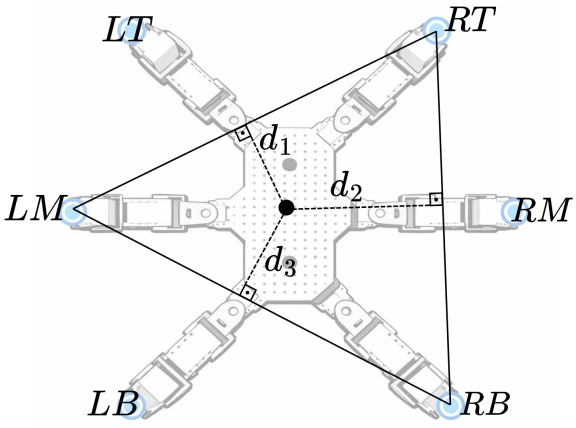


Figura 3.11: Ejemplo de polígono de soporte
Fuente: Elaboración propia.

En la configuración trípode mostrada, el margen de estabilidad puede expresarse como $S_m = \text{mín}(d_1, d_2, d_3)$, donde cada d_i representa la distancia desde la proyección del centro de masas a uno de los lados del polígono de soporte. Con menos de tres apoyos no alineados, el polígono degenera en un segmento o en un punto y no encierra ningún área.

Esta propiedad explica una de las ventajas fundamentales del hexápodo frente a configuraciones con menos patas. Mientras que un cuadrúpedo pasa a apoyarse sobre el triángulo mínimo al levantar una sola extremidad, un hexápodo conserva un número mayor de combinaciones estables de apoyo.

3.4.2. Parámetros fundamentales de la marcha

Para caracterizar matemáticamente y diseñar una marcha, se emplea la formulación estándar introducida por McGhee y Frank y recogida posteriormente en la literatura de robótica con patas [5, 1]. Esta define un conjunto de parámetros temporales y cinemáticos interdependientes:

- **Tiempo de Ciclo (T):** Es la duración de un ciclo completo de locomoción, definido como el intervalo temporal necesario para que todas las patas del robot completen exactamente una fase de apoyo y una de transferencia.
- **Factor de servicio o *Duty Factor* (β_i):** Representa la fracción del tiempo total del ciclo durante la cual la pata i se encuentra en contacto con el suelo (fase de apoyo o *stance*). Se define como:

$$\beta_i = \frac{t_{stance}}{T} \tag{3.37}$$

  En las marchas periódicas simétricas consideradas en este trabajo se asumirá que $\beta_i = \beta$ para todas las patas. Bajo esta formulación, si $\beta \geq 0{,}5$ la locomoción se clasifica como marcha: no aparecen las fases de vuelo sin soporte completo, propias de los regímenes dinámicos o de carrera ($\beta < 0{,}5$).
- **Fase relativa (ϕ_i):** Es el desfase temporal con que la pata i inicia su apoyo respecto a una pata elegida arbitrariamente como referencia, a la que se asigna $\phi_i = 0$ (LT en las marchas trípode y de onda de la Subsección 3.4.3), normalizado por el tiempo de ciclo T. Su valor oscila en el intervalo $[0, 1)$.
- **Velocidad de Avance (V):** Este parámetro describe el avance medio del cuerpo a lo largo de un ciclo completo y se define globalmente como:

$$V = \frac{R_{stride}}{T} \tag{3.38}$$

  donde R_{stride} (*Stride Length*) representa la distancia de avance del cuerpo a lo largo de un ciclo completo.

  Desde la perspectiva de una pata individual, durante su fase de apoyo el cuerpo avanza una distancia $s_{step} = \beta R_{stride} = \beta V T$. Esa misma distancia debe recuperarse durante la fase de transferencia, como ilustra la Figura 3.12. Si U (*Return Speed*) denota la velocidad media de retorno de la pata en *swing*, resulta:

$$U = \frac{s_{step}}{(1-\beta)T} \quad \Longleftrightarrow \quad V = \frac{1-\beta}{\beta} U \tag{3.39}$$

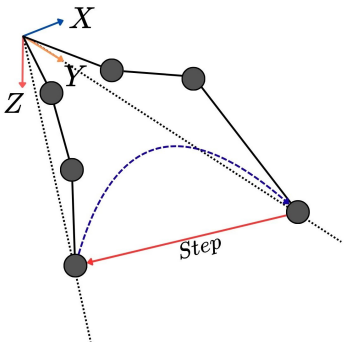


Figura 3.12: Trayectoria de la pata en un ciclo de marcha.

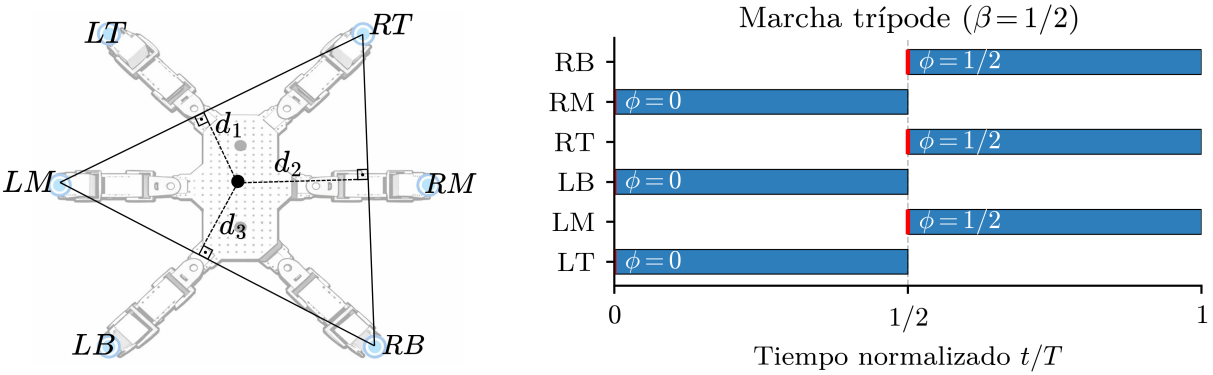
Fuente: Elaboración propia.

  Esta relación establece el vínculo entre la velocidad global del cuerpo y la velocidad local que deben desarrollar las patas durante *swing*. Así, para una marcha muy estable ($\beta = 0{,}75$), la velocidad del robot queda limitada aproximadamente a un tercio de la velocidad de retorno de la pata ($V \approx 0{,}33U$), mientras que en marcha trípode ($\beta = 0{,}5$) ambas coinciden ($V = U$).

3.4.3. Patrones de marcha periódica

Dentro de la categoría de marchas periódicas, se han seleccionado tres patrones representativos y recurrentes en la literatura. En conjunto, estos patrones ilustran distintos compromisos entre velocidad, margen de estabilidad y continuidad del apoyo [1].

Marcha Trípode (*Tripod Gait*, $\beta = 1/2$): Es la marcha periódica estáticamente estable más rápida en un hexápodo. Las patas se organizan en dos trípodes alternos (generalmente {LT, LB, RM} y {RT, RB, LM}) que operan en contrafase, de modo que mientras un grupo permanece en apoyo el otro ejecuta la fase de transferencia.



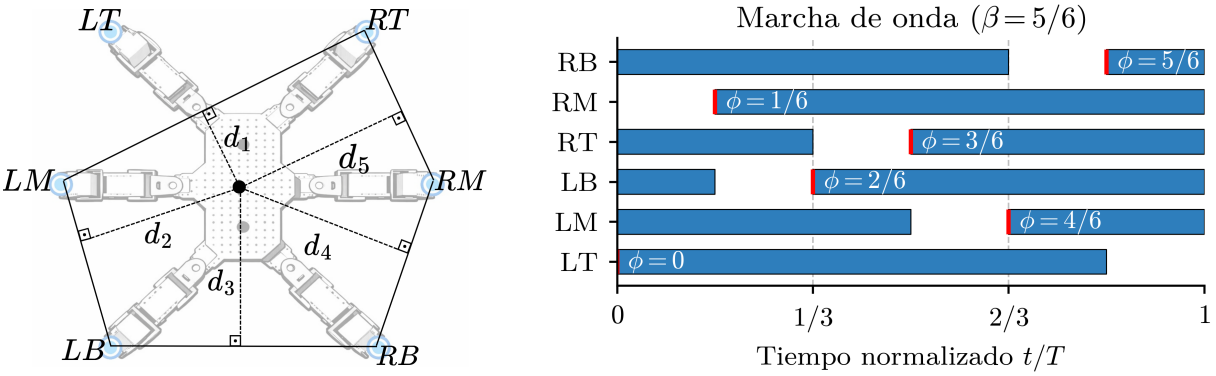
(a) Polígono de soporte

(b) Diagrama de marcha

Figura 3.13: Marcha trípode: (a) polígono de soporte y (b) diagrama de marcha.
Fuente: Elaboración propia.

Como se observa en la Figura 3.13, al tener un factor de servicio $\beta = 0{,}5$, la velocidad de avance del robot iguala a la velocidad de retorno de la pata durante la fase *swing* ($V = U$), según la Ec. 3.39. La marcha trípode mantiene tres apoyos simultáneos y presenta estabilidad estática mientras la proyección del centro de masas permanezca en el interior del triángulo de soporte. Es la marcha con menor margen frente a perturbaciones.

Marcha de Onda (*Wave/Ripple Gait*, $\beta = 5/6$): En esta configuración, las patas se levantan secuencialmente una a una siguiendo un patrón de ola, de modo que en cualquier instante cinco patas se encuentran en fase de apoyo y solo una en fase de transferencia.



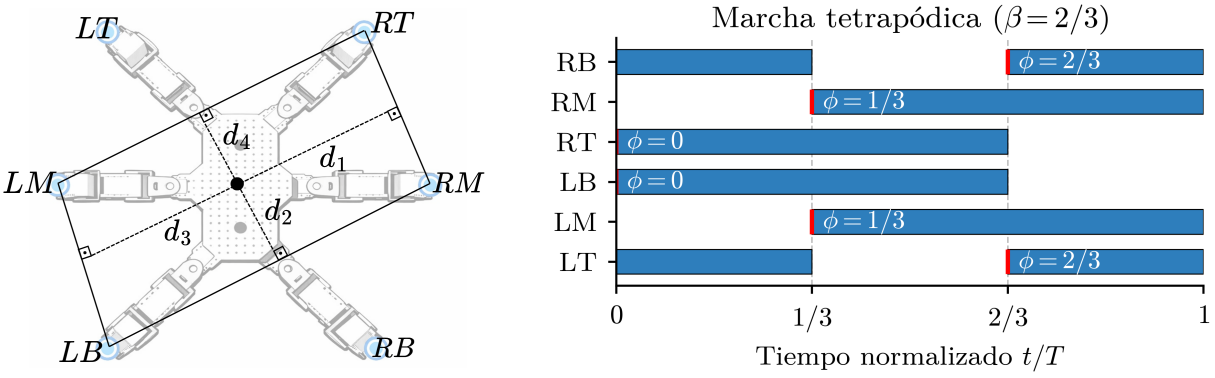
(a) Polígono de soporte.

(b) Diagrama de marcha.

Figura 3.14: Marcha de onda: (a) polígono de soporte y (b) diagrama de marcha.
Fuente: Elaboración propia.

Como muestra la Figura 3.14, esto da lugar a un polígono de soporte pentagonal, maximizando el margen de estabilidad estática y haciendo esta marcha especialmente adecuada para terrenos irregulares o maniobras de precisión. Como contrapartida, su velocidad teórica es reducida ($V = 0{,}2 \cdot U$).

Marcha Tetrapódica (*Tetrapod/Amble Gait*, $\beta = 2/3$): Representa un compromiso intermedio entre rapidez y estabilidad. Las patas se coordinan por pares, de manera que en régimen periódico permanecen cuatro patas en contacto con el suelo.



(a) Polígono de soporte.

(b) Diagrama de marcha

Figura 3.15: Marcha tetrapódica: (a) polígono de soporte y (b) diagrama de marcha.
Fuente: Elaboración propia.

Como se aprecia en la Figura 3.15, con un factor de servicio de $\beta \approx 0{,}67$, ofrece mayor estabilidad que la marcha trípode y mayor velocidad que la marcha de onda ($V = 0{,}5 \cdot U$), proporcionando una redundancia de apoyo útil cuando la tracción del suelo no es perfecta.

3.5. Planificación de trayectorias

La generación de movimiento en el hexápodo requiere definir la evolución temporal de las coordenadas, ya que aplicar cambios instantáneos a los servomotores provocaría aceleraciones teóricamente infinitas. Una vez fijado el patrón temporal de la marcha, el siguiente problema consiste en determinar cómo evoluciona espacialmente cada pata.

3.5.1. Perfil de trayectoria de vuelo (swing phase)

La trayectoria de la pata se parametriza mediante una fase normalizada $u \in [0, 1]$, donde $u = 0$ corresponde al instante de despegue y $u = 1$ al de aterrizaje. Bajo esta variable, la planificación se desacopla en una componente vertical, que garantiza el despeje respecto al suelo, y una componente horizontal, que recoloca la pata en el siguiente punto de apoyo.

Elevación vertical (Z): El término $\Delta z_{step}(u)$ introducido en la componente vertical de la Ec. 3.27 se modela mediante un perfil sinusoidal cuadrático. Esta elección conserva la sencillez geométrica de una ley senoidal y, al mismo tiempo, proporciona un comportamiento suave en los instantes de despegue y aterrizaje:

$$\Delta z_{step}(u) = H_{step} \, \sin^2(\pi u) \tag{3.40}$$

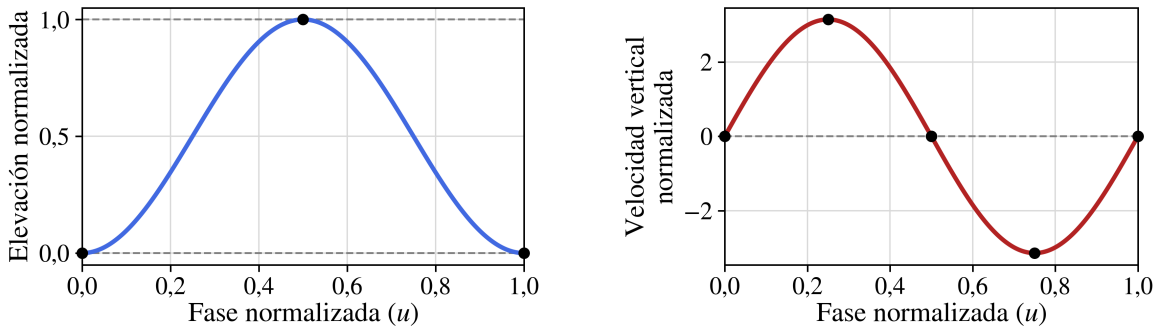
Con esta expresión se satisfacen las condiciones geométricas deseadas para la fase de elevación: $\Delta z_{step}(0) = 0$ y $\Delta z_{step}(1) = 0$, de modo que el pie despega desde la cota de referencia y vuelve a ella al finalizar el *swing*. Además, $\Delta z_{step}\left(\frac{1}{2}\right) = H_{step}$, por lo que la altura máxima se alcanza en el punto medio del recorrido. Para estudiar la suavidad del perfil, se deriva respecto a la variable normalizada u:

$$\frac{d\,\Delta z_{step}}{du} = 2\pi H_{step} \sin(\pi u)\cos(\pi u) = \pi H_{step} \sin(2\pi u) \tag{3.41}$$

De la Ec. 3.41 se deduce que los puntos críticos aparecen en $u = 0$, $u = \frac{1}{2}$ y $u = 1$. En particular, evaluando la derivada en los extremos:

$$\left.\frac{d\,\Delta z_{step}}{du}\right|_{u=0} = \left.\frac{d\,\Delta z_{step}}{du}\right|_{u=1} = 0 \tag{3.42}$$

por lo que la trayectoria presenta pendiente nula tanto al inicio como al final de la fase de vuelo. Esto implica que el pie inicia y finaliza la elevación sin cambios bruscos en la componente vertical, lo que se traduce en un movimiento más suave y controlado, como se observa en la Figura 3.16.



(a) $\Delta z_{step}(u)/H_{step} = \sin^2(\pi u)$. (b) $\frac{1}{H_{step}}\frac{d\,\Delta z_{step}}{du} = \pi \sin(2\pi u)$.

Figura 3.16: Perfil vertical sinusoidal cuadrático: evolución de la posición y de la velocidad normalizadas durante la fase de *swing*.

Fuente: Elaboración propia mediante MATLAB [32].

Avance horizontal (XY): El desplazamiento en el plano, encargado de recolocar la pata en el siguiente punto de apoyo, se modela mediante una interpolación polinómica cúbica sobre esta misma fase normalizada u. Esta elección permite imponer de forma directa las condiciones de contorno del movimiento y, al mismo tiempo, garantiza que la pata inicie y finalice el *swing* sin velocidades horizontales bruscas. Para ello, se parte del polinomio general:

$$f(u) = c_3u^3 + c_2u^2 + c_1u + c_0 \tag{3.43}$$

Sobre este polinomio se imponen las condiciones geométricas deseadas para la fase de avance: el pie debe recorrer el paso completo, $f(0) = 0$ y $f(1) = 1$, partiendo y llegando con velocidad nula, $f'(0) = f'(1) = 0$. Estas restricciones se resumen en la Ec. 3.44. Al aplicarlas en $u = 0$ se obtiene directamente $c_0 = c_1 = 0$, mientras que las restantes conducen al sistema de la Ec. 3.45, cuya solución es $c_2 = 3$ y $c_3 = -2$.

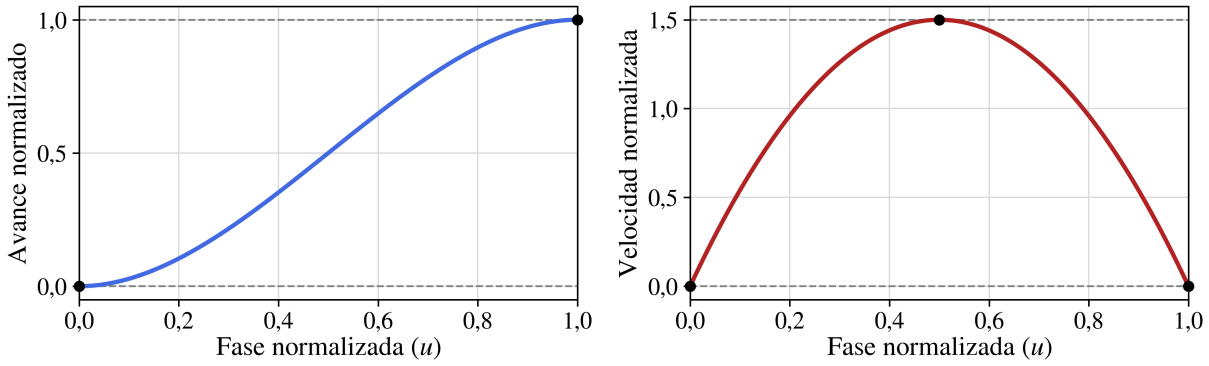
$$\begin{cases} f(0) = 0, & f(1) = 1 \\ f'(0) = 0, & f'(1) = 0 \end{cases} \tag{3.44}$$

$$\begin{cases} c_3 + c_2 = 1 \\ 3c_3 + 2c_2 = 0 \end{cases} \tag{3.45}$$

Sustituyendo estos valores, se obtiene finalmente la ley de interpolación horizontal:

$$f_{xy}(u) = 3u^2 - 2u^3 \tag{3.46}$$

En este caso, la suavidad en los extremos no requiere verificación adicional, puesto que la velocidad nula en el despegue y el aterrizaje forma parte de las condiciones impuestas en la Ec. 3.44, tal como se aprecia en la Figura 3.17.



(a) $f_{xy}(u) = 3u^2 - 2u^3$. (b) $\frac{df_{xy}}{du} = 6u(1-u)$.

Figura 3.17: Perfil horizontal cúbico: evolución de la posición y de la velocidad normalizadas durante la fase de *swing*.

Fuente: Elaboración propia mediante MATLAB [32].

En el caso nominal de paso constante, sin giro y con postura local nula, esta ley puede escribirse en su forma centrada:

$$a(u) = f_{xy}(u) - \frac{1}{2} \tag{3.47}$$

Con ello, la trayectoria horizontal queda referida a la postura neutral y el pie recorre simétricamente el intervalo $\left[{}^{B}\boldsymbol{p}_{neu} - \boldsymbol{s}_{step}/2,\ {}^{B}\boldsymbol{p}_{neu} + \boldsymbol{s}_{step}/2\right]$. Esta forma centrada es algebraicamente equivalente a la interpolación de la Ec. 3.26 cuando los extremos coinciden con los nominales.

3.5.2. Suavizado de postura

Para las transiciones de postura del cuerpo (cambios de altura, inclinación o rotación), se requiere una suavidad superior a la de la trayectoria de la pata. Mientras que el perfil cúbico garantiza velocidad nula en los extremos, permite saltos instantáneos en la aceleración, lo que puede inducir vibraciones estructurales. Para mitigar esto, se implementa un interpolador de quinto orden [29]. Para ello, se parte del polinomio general:

$$S(t) = c_5t^5 + c_4t^4 + c_3t^3 + c_2t^2 + c_1t + c_0 \tag{3.48}$$

Sobre este polinomio se imponen seis condiciones de contorno: el perfil debe recorrer la transición completa, $S(0) = 0$ y $S(1) = 1$, con velocidad y aceleración nulas en ambos extremos, $S'(0) = S'(1) = 0$ y $S''(0) = S''(1) = 0$. Estas restricciones se resumen en la Ec. 3.49. Al aplicarlas en $t = 0$ se obtiene directamente $c_0 = c_1 = c_2 = 0$, mientras que las restantes conducen al sistema de la Ec. 3.50, cuya solución es $c_3 = 10$, $c_4 = -15$ y $c_5 = 6$.

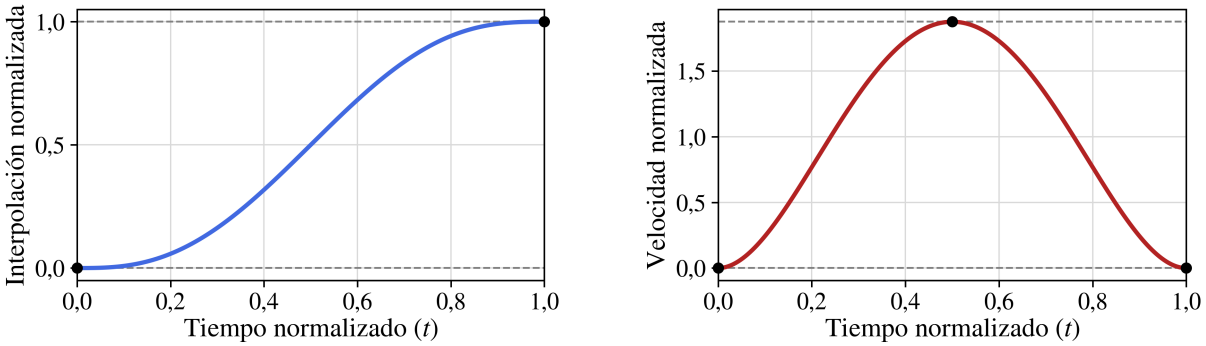
$$\begin{cases} S(0) = 0, & S(1) = 1 \\ S'(0) = 0, & S'(1) = 0 \\ S''(0) = 0, & S''(1) = 0 \end{cases} \tag{3.49}$$

$$\begin{cases} c_5 + c_4 + c_3 = 1 \\ 5c_5 + 4c_4 + 3c_3 = 0 \\ 20c_5 + 12c_4 + 6c_3 = 0 \end{cases} \tag{3.50}$$

Sustituyendo estos valores, se obtiene finalmente la función de mezcla de quinto orden:

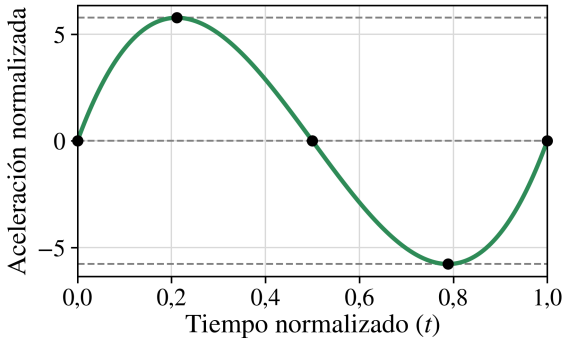
$$S(t) = 6t^5 - 15t^4 + 10t^3 \tag{3.51}$$

La suavidad queda de nuevo garantizada por construcción, con la diferencia de que ahora también la aceleración se anula en los extremos, tal como se aprecia en la Figura 3.18.



(a) $S(t) = 6t^5 - 15t^4 + 10t^3$.

(b) $\frac{dS}{dt} = 30t^2(1-t)^2$.



(c) $\frac{d^2S}{dt^2} = 60t(2t^2 - 3t + 1)$.

Figura 3.18: Perfil quíntico de suavizado: evolución de la posición, la velocidad y la aceleración durante la transición de postura.

Fuente: Elaboración propia mediante MATLAB [32].

4. Plataforma hardware e implementación

Este capítulo describe la plataforma hardware y el código desarrollado para lograr el funcionamiento del hexápodo. El código fuente completo, la documentación generada mediante Doxygen y los archivos de configuración se encuentran disponibles en el repositorio público del proyecto [35]. El diseño del sistema se ha guiado por tres criterios:

i) Partición funcional entre la distribución de potencia y la electrónica de control,

ii) Modularidad de los subsistemas físicos y lógicos,

iii) Compatibilidad con una arquitectura de alto nivel basada en ROS 2.

4.1. Arquitectura general del sistema

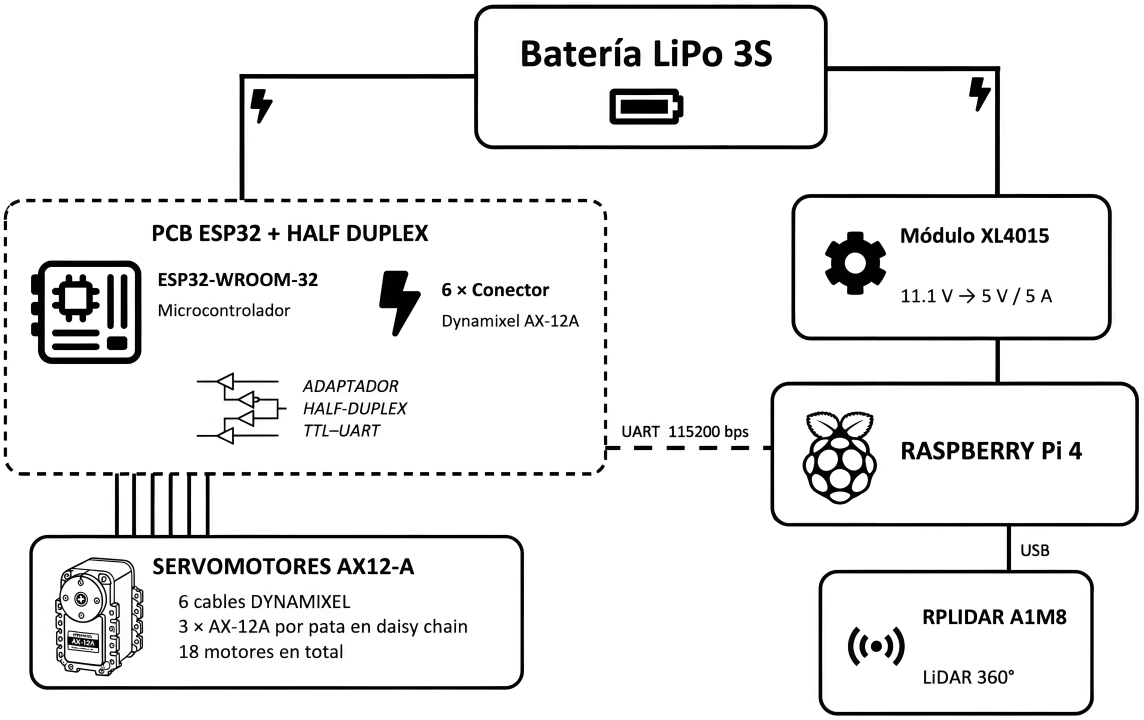


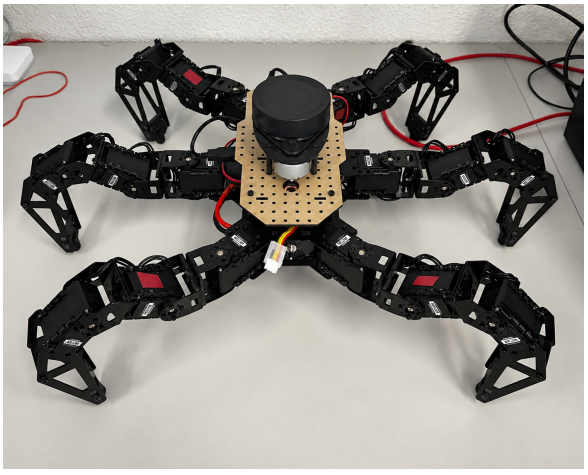
Figura 4.1: Arquitectura general de la plataforma hardware.
Fuente: Elaboración propia.

La Figura 4.1 presenta la arquitectura general de la plataforma y las interconexiones entre sus subsistemas. Desde un punto de vista funcional, el sistema se organiza en dos ramas claramente diferenciadas. La de alto nivel, que se ejecuta sobre la Raspberry Pi 4 [36], donde residen los nodos ROS 2 y el subsistema de percepción (Sección 4.4). Y la rama de bajo nivel, que se implementa sobre el microcontrolador ESP32 [37] (Secciones 4.2 y 4.3), responsable de la comunicación con los actuadores y de la ejecución de las órdenes de movimiento hacia los servos DYNAMIXEL AX-12A [17].

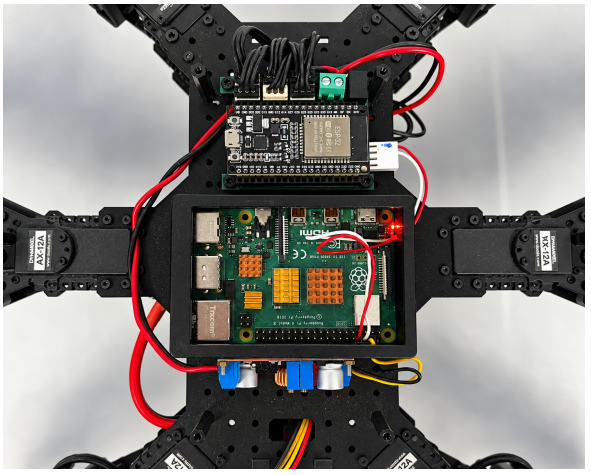
La alimentación global parte de una batería de polímero de litio (LiPo) de tres celdas (3S) y se distribuye en dos ramas. La primera alimenta la PCB principal (Subsección 4.2.4 y Anexo C), que integra en un único soporte físico la electrónica de control basada en el ESP32 y la distribución de potencia hacia los 18 servomotores. Aunque ambas funciones conviven en la misma placa, el diseño se ha planteado con una separación funcional entre la etapa de potencia asociada a los motores y la lógica de control, con el fin de reducir interferencias y facilitar la organización del sistema. Desde esta PCB se reparten alimentación y datos mediante seis cables, uno por pata (Figura 4.4).

La segunda rama incorpora un módulo comercial reductor de corriente continua (DC-DC) basado en el integrado XL4015 [38], que adapta la tensión nominal de la batería (11,1 V) a 5 V para alimentar la Raspberry Pi 4. Este módulo se seleccionó por su capacidad para suministrar hasta 5 A, suficiente para cubrir con margen la demanda de corriente del subsistema de alto nivel. El sensor LiDAR RPLIDAR A1M8 [39], conectado por USB a la Raspberry Pi 4, proporciona la medida del entorno con cobertura angular de 360° (Subsección 4.4.3). La comunicación entre la Raspberry Pi 4 y el ESP32 se realiza mediante un enlace UART (*Universal Asynchronous Receiver-Transmitter*) bidireccional a 115200 bps (Subsecciones 4.3.6 y 4.4.1), lo que permite desacoplar la toma de decisiones de alto nivel del control de bajo nivel de los actuadores.

La Figura 4.2 muestra el montaje del robot y la disposición de la electrónica de control y alimentación sobre el chasis.



(a) Vista general del montaje hardware del robot.



(b) Detalle de la electrónica de control y alimentación.

Figura 4.2: Montaje hardware del hexápodo: vista general y detalle de la electrónica integrada.
Fuente: Elaboración propia.

4.2. Hardware de bajo nivel y control de actuadores

El subsistema de bajo nivel concentra los elementos responsables del control directo del robot: los servomotores, el microcontrolador, la electrónica de adaptación y el firmware ejecutado sobre el ESP32. Es conveniente presentarlo antes que el subsistema de alto nivel, ya que constituye la base funcional sobre la que se apoya la capa ROS 2.

4.2.1. Servomotores DYNAMIXEL AX-12A

En este TFG se emplean servomotores DYNAMIXEL AX-12A [17] (Figura 4.3), que integran en un único módulo el motor DC con reductora, la electrónica de potencia, el control interno y la sensórica necesaria para proporcionar realimentación del estado del actuador. Desde el punto de vista mecánico, el actuador incorpora una reductora de relación 254:1 que le permite entregar un par de parada de 1,5 N·m alimentado a 12 V, con una velocidad máxima en vacío de 59 rpm. Su sensor interno discretiza el giro en 1024 niveles. Además, eléctricamente admite un rango de operación de 9 a 12 V.



Figura 4.3: Actuador DYNAMIXEL AX-12A.
Fuente: e-Manual de ROBOTIS [17].

La conexión de cada actuador consiste en un cable de tres hilos que agrupa la alimentación (VDD), la masa (GND) y la línea de datos, de manera que potencia y comunicación comparten el mismo mazo de cable.

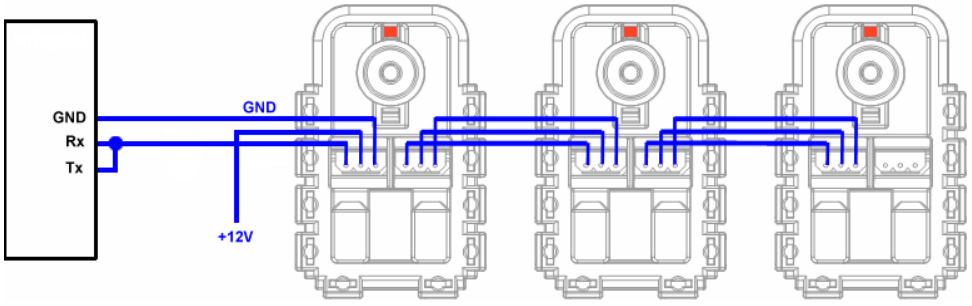


Figura 4.4: Topología de conexión en cadena (*daisy-chain*).
Fuente: Elaboración propia.

Tal y como muestra la Figura 4.4, desde el concentrador de seis puertos de la PCB los 18 actuadores se distribuyen en seis ramas, una por pata, con tres servos conectados en topología *daisy-chain* en cada una de ellas. Dado que todos comparten físicamente la misma línea de datos, resulta necesario asignar un identificador único a cada actuador. La Tabla 4.1 recoge esta asignación en función de la pata y de la articulación correspondiente:

Pata	Coxa	Fémur	Tibia	Pata	Coxa	Fémur	Tibia
Izq. delantera (LT)	1	3	5	Der. delantera (RT)	2	4	6
Izq. media (LM)	13	15	17	Der. media (RM)	14	16	18
Izq. trasera (LB)	7	9	11	Der. trasera (RB)	8	10	12

Tabla 4.1: Mapa de asignación de IDs de los actuadores.

4.2.2. Bus DYNAMIXEL y protocolo de comunicación

La comunicación con los servomotores se realiza a través de un bus serie propietario asíncrono *half-duplex* sobre una única línea de datos. Esta arquitectura permite compartir alimentación, masa y canal de comunicaciones entre múltiples dispositivos, reduciendo de forma significativa el cableado del robot. Sobre este medio físico se emplea el protocolo DYNAMIXEL 1.0 [16], que define el formato de las tramas serie intercambiadas entre el controlador y los actuadores. Aunque existe una versión 2.0, los AX-12A utilizados en este TFG solo son compatibles con la versión 1.0, de modo que su uso viene impuesto por el propio hardware. La comunicación se articula en dos tipos de paquetes: el paquete de instrucción, enviado por el controlador, y el paquete de estado, devuelto por el actuador como respuesta.

Paquete de instrucción El paquete de instrucción tiene la siguiente estructura:

$$[\underbrace{\texttt{0xFF 0xFF}}_{\text{Header}} | \underbrace{\texttt{ID}}_{\text{ID}} | \underbrace{\texttt{LEN}}_{\text{Longitud}} | \underbrace{\texttt{INST}}_{\text{Instrucción}} | \underbrace{\texttt{P}_1 \cdots \texttt{P}_N}_{\text{Parámetros}} | \underbrace{\texttt{CSUM}}_{\text{Checksum}}] \tag{4.1}$$

- Header (`0xFF 0xFF`): dos bytes fijos que marcan el inicio de toda trama.
- ID: identificador del actuador destinatario (0–253). El valor especial `0xFE` es la dirección de *broadcast*, que permite dirigirse a todos los actuadores simultáneamente.
- LEN: número de bytes restantes en el paquete, calculado como $N_{\text{parámetros}} + 2$.
- INST: código de la instrucción a ejecutar. La Tabla 4.2 recoge las principales.
- Parámetros: datos asociados a la instrucción.
- CSUM: suma de verificación del paquete, calculada como:

$$\texttt{CSUM} = \overline{\texttt{ID} + \texttt{LEN} + \texttt{INST} + \texttt{P}_1 + \cdots + \texttt{P}_N}$$

Código	Nombre	Descripción
`0x01`	Ping	Comprueba la presencia de un actuador en el bus.
`0x02`	Read Data	Lee uno o varios bytes consecutivos del mapa de memoria.
`0x03`	Write Data	Escribe uno o varios bytes en el mapa de memoria.
`0x04`	Reg Write	Registra una escritura pendiente, ejecutada al recibir Action.
`0x05`	Action	Ejecuta la escritura registrada previamente con Reg Write.
`0x06`	Reset	Restaura la configuración del actuador a sus valores de fábrica.
`0x83`	Sync Write	Escribe simultáneamente en múltiples actuadores en una sola trama.

Tabla 4.2: Instrucciones principales del protocolo DYNAMIXEL 1.0.

Sync Write (`0x83`) resulta especialmente relevante en este TFG. Su finalidad es escribir, en una única trama, un mismo bloque de registros sobre múltiples actuadores. Para ello se emplea la dirección de *broadcast* (`0xFE`) y se indica la dirección inicial del registro, la longitud del bloque que se desea escribir y, a continuación, una secuencia formada por el identificador de cada servo y sus datos asociados. Esto permite actualizar simultáneamente los 18 actuadores sin que estos generen paquetes de estado en respuesta, al tratarse de una transmisión en *broadcast*.

Paquete de estado Tras recibir y procesar una instrucción, el actuador devuelve un paquete de estado con la siguiente estructura:

$$[\underbrace{\texttt{0xFF 0xFF}}_{\text{Header}} | \underbrace{\texttt{ID}}_{\text{ID}} | \underbrace{\texttt{LEN}}_{\text{Longitud}} | \underbrace{\texttt{ERR}}_{\text{Error}} | \underbrace{\texttt{P}_1 \cdots \texttt{P}_N}_{\text{Parámetros}} | \underbrace{\texttt{CSUM}}_{\text{Checksum}}] \tag{4.2}$$

El byte de error permite detectar condiciones anómalas en tiempo real sin necesidad de sensores externos. Cada bit corresponde a un tipo de error independiente (Figura 4.5).

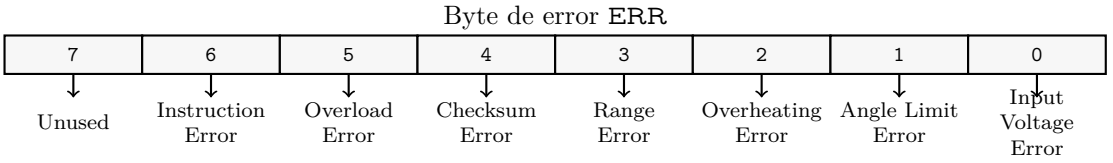


Figura 4.5: Distribución de bits del byte de error del DYNAMIXEL AX-12A.

4.2.3. Mapa de memoria del AX-12A y registros utilizados

El AX-12A dispone de una memoria interna organizada en dos áreas con propiedades distintas. Por un lado, la EEPROM, de carácter no volátil, almacena la configuración del servo, como su identificador, la velocidad de transmisión, los límites angulares o la temperatura máxima permitida. Por otro lado, la RAM (*Random Access Memory*), de carácter volátil, refleja el estado en tiempo real e incluye la posición objetivo, la velocidad de movimiento, la posición actual del eje, la dirección y magnitud estimadas de la carga o la temperatura interna, entre otros registros. La carga indicada por el actuador se infiere a partir de su salida interna y, por tanto, no constituye una medida directa de par.

La Tabla 4.3 recoge los registros más relevantes de ambas áreas, indicando su dirección, tamaño, rango y tipo de acceso.

Área	Dir.	Bytes	Registro	Rango	Acceso
Memoria no volátil (EEPROM) — configuración estática					
	0x03	1	ID	0 – 253	R/W
	0x04	1	Baud Rate	0 – 254	R/W
	0x06	2	CW Angle Limit	0 – 1023	R/W
	0x08	2	CCW Angle Limit	0 – 1023	R/W
	0x0B	1	Temperature Limit	0 – 100 °C	R/W
	0x11	1	Alarm LED	0 – 127	R/W
	0x12	1	Shutdown	0 – 127	R/W
Memoria volátil (RAM) — estado dinámico en tiempo real					
	0x18	1	Torque Enable	0 – 1	R/W
	0x1A	1	CW Compliance Margin	0 – 255	R/W
	0x1B	1	CCW Compliance Margin	0 – 255	R/W
	0x1C	1	CW Compliance Slope	0 – 254	R/W
	0x1D	1	CCW Compliance Slope	0 – 254	R/W
	0x1E	2	Goal Position	0 – 1023	R/W
	0x20	2	Moving Speed	0 – 1023	R/W
	0x24	2	Present Position	0 – 1023	R
	0x26	2	Present Speed	0 – 2047	R
	0x28	2	Present Load	0 – 2047	R
	0x2A	1	Present Voltage	50 – 160	R
	0x2B	1	Present Temperature	0 – 100	R

Tabla 4.3: Mapa de memoria del DYNAMIXEL AX-12A (registros principales).

Conviene detenerse primero en los registros de posición, ya que fijan las unidades con las que trabaja el resto del mapa de memoria. Como muestra la Figura 4.6, en modo articulación el AX-12A dispone de un recorrido útil de 300°, codificado mediante valores entre 0 y 1023. Esto proporciona una resolución de $300°/1023 \approx 0{,}293°$ por unidad. Los registros CW Angle Limit (0x06) y CCW Angle Limit (0x08) permiten restringir dicho intervalo en el propio actuador, de modo que las consignas situadas fuera de los límites configurados no produzcan movimientos contra los topes mecánicos.

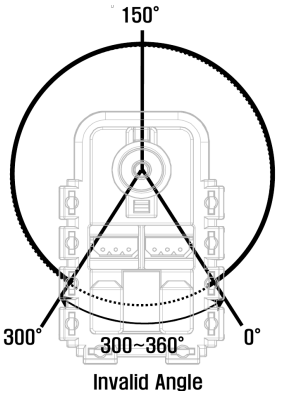


Figura 4.6: Rango del AX-12A.
Fuente: e-Manual de ROBOTIS [17].

Resultan también especialmente relevantes los parámetros Compliance Margin y Compliance Slope, ambos definidos con un registro independiente para cada sentido de giro. La Figura 4.7 ilustra el par de salida en función del error de posición.

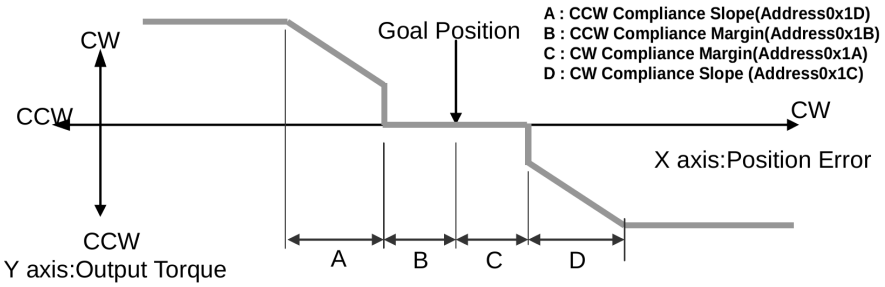


Figura 4.7: Perfil de par de salida en función del error de posición.
Fuente: e-Manual de ROBOTIS [17].

El Compliance Margin define una zona muerta angular alrededor de la posición objetivo, dentro de la cual el servo no genera par de corrección. El Compliance Slope determina cómo crece dicho par una vez superada esa zona y admite siete niveles discretos asociados a potencias de dos. Un margen demasiado estrecho favorece las oscilaciones, el consumo y el calentamiento, mientras que uno excesivo reduce la precisión del apoyo. De forma análoga, una pendiente abrupta transmite los impactos a la estructura y una demasiado suave reduce la capacidad de reacción frente al peso del robot. Atendiendo a este equilibrio, se adoptaron empíricamente los valores 4 y 64. El primero equivale aproximadamente a 1,17° en cada sentido y produce una zona muerta total de 2,35°. El segundo sitúa la pendiente en el penúltimo de los siete escalones disponibles, proporcionando una respuesta más flexible cerca de la consigna.

En la EEPROM también se configuran los registros Alarm LED y Shutdown con los valores 0x25 y 0x04, respectivamente. El indicador luminoso advierte así de errores de tensión, temperatura y sobrecarga, mientras que la desconexión automática del par queda reservada al sobrecalentamiento. De este modo, una incidencia transitoria de alimentación o esfuerzo no provoca la pérdida simultánea de los apoyos del robot.

Además, se ha desarrollado una herramienta de supervisión de los servomotores, que se describirá posteriormente en la capa de alto nivel. Su base de bajo nivel consiste en una única instrucción Read Data sobre el bloque de cuatro bytes comprendido entre Present Load (0x28) y Present Temperature (0x2B): dos para la carga estimada, uno para la tensión y otro para la temperatura. El paquete de estado devuelto en esa misma transacción contiene además el byte de estado ERR. Así se obtienen las variables de operación y las alarmas activas sin añadir otra lectura al bus.

4.2.4. Electrónica de adaptación al bus *half-duplex*

La UART2 del ESP32 [37] dispone de líneas independientes de transmisión y recepción y trabaja con niveles lógicos de 3,3 V. En cambio, los actuadores AX-12A comparten una única línea de datos TTL (*Transistor-Transistor Logic*) de 5 V y emplean una comunicación *half-duplex* [17]. La interfaz debe resolver, por tanto, dos funciones: adaptar los niveles de tensión y conectar alternativamente el transmisor o el receptor al bus de los servos. Esta adaptación se integra en la PCB, que también distribuye conjuntamente la alimentación y los datos, de forma similar a la ArbotiX que se ha sustituido [3].

Para entender el funcionamiento de la adaptación lógica y del *buffer*, conviene comenzar por las tres señales que utiliza el microcontrolador: `TX` (`GPIO16`), que transporta las instrucciones; `RX` (`GPIO17`), que recibe los paquetes de estado; y `DIR` (`GPIO2`), una salida digital que selecciona el sentido de la comunicación. En el esquema de la Figura 4.8, estas redes se identifican como `TX3V3`, `RX3V3` y `DIR3V3`.

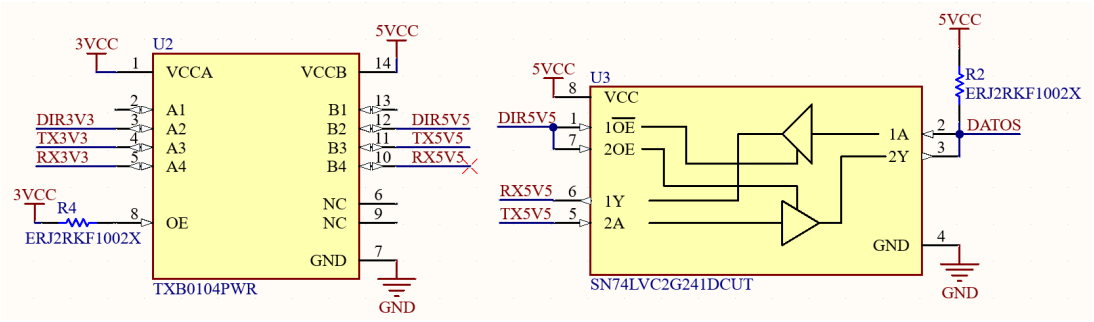


Figura 4.8: Esquema de la interfaz de adaptación al bus *half-duplex*.
Fuente: Elaboración propia mediante Altium Designer [40].

La primera etapa del circuito es un traductor bidireccional TXB0104PWR con detección automática del sentido [41]. Su puerto A se alimenta a 3,3 V y el B a 5 V. Tres de sus cuatro canales convierten las señales anteriores en `DIR5V5`, `TX5V5` y `RX5V5`. La resistencia R4 de 10 kΩ mantiene activa la entrada `OE`, por lo que el traductor permanece habilitado de forma permanente.

Una vez adaptados los niveles, el buffer triestado SN74LVC2G241 convierte las líneas independientes de transmisión y recepción en una única línea de datos [42]. Como muestra la Figura 4.8, uno de sus canales conduce `TX5V5` hacia `DATOS`, mientras que el otro conduce `DATOS` hacia `RX5V5`. Ambos canales tienen habilitaciones de polaridad opuesta gobernadas por `DIR5V5`. Con un nivel alto se habilita la transmisión y el receptor queda en alta impedancia; con un nivel bajo se produce la situación inversa.

La Tabla 4.4 resume esta selección, donde Z representa el estado de alta impedancia de la salida deshabilitada. Además, la resistencia R2 de 10 kΩ actúa como *pull-up* y mantiene `DATOS` a 5 V cuando ningún dispositivo conduce el bus. El estado de reposo programado es la recepción, con `DIR` a nivel bajo. Para enviar una instrucción, el firmware eleva esta señal, escribe el paquete en la UART y espera a que el último bit haya salido del periférico. Solo entonces devuelve `DIR` a nivel bajo y habilita la recepción del paquete de estado. Esta gestión del bus se implementa en el controlador de servomotores, descrito en la Subsección 4.2.5.

Tabla 4.4: Lógica de salidas del buffer.

DIR5V5 ($\overline{1OE} = 2OE$)	1Y	2Y
0	1A (DATOS)	Z
1	Z	2A (TX5V5)

La Figura 4.9 muestra la integración física del circuito. El ESP32 se instala sobre dos tiras de zócalos para poder retirarlo sin intervenir sobre los componentes soldados. El traductor de nivel y el buffer triestado se sitúan entre ambas tiras, próximos a la cabecera P1, que permite medir DATOS, DIR, TX y RX. La línea de datos, la alimentación de los actuadores y la masa se conducen hasta seis conectores Molex de tres vías, uno por pata.

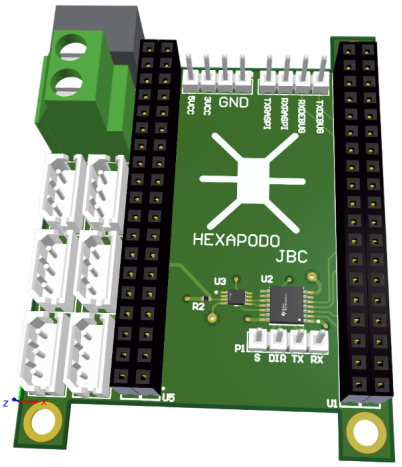
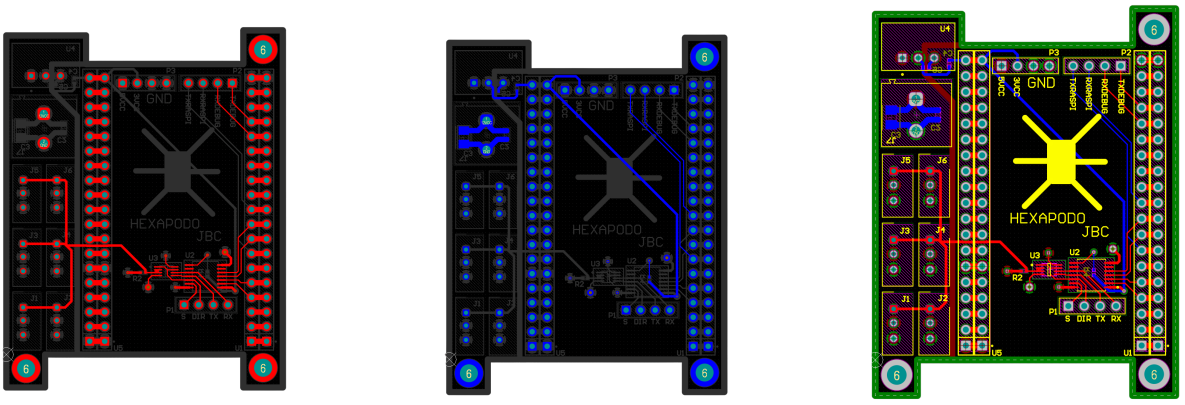


Figura 4.9: PCB en 3D.
Fuente: Altium Designer [40].

La placa se ha diseñado con un apilado de cuatro capas: capa superior, plano interno de masa, plano interno de alimentación y capa inferior. El plano de masa proporciona una referencia común y un retorno de baja impedancia para las señales. El plano de alimentación se divide en dos regiones aisladas: 12VCC, nombre asignado a la tensión no regulada de la batería LiPo 3S (11,1 V), y 5VCC, generada por el TSR 1-2450 [43]. La primera distribuye potencia a los actuadores y la segunda alimenta la lógica. Las capas externas alojan los componentes, las señales y las conexiones locales con los planos internos. La Figura 4.10 muestra ambos trazados y su superposición.



(a) Trazado superior. (b) Trazado inferior. (c) Superposición 2D.

Figura 4.10: Encaminado en las capas externas de la PCB.
Fuente: Altium Designer [40].

La Figura 4.11 muestra una captura de osciloscopio. La señal DATOS se representa en el canal 1 (amarillo) y la señal DIR, en el canal 2 (cian), con escalas de 20 μs/div y 1 V/div, con un desplazamiento vertical para facilitar su identificación. Se observa cómo DIR pasa a nivel alto antes del inicio de la trama de lectura y permanece en dicho estado durante toda la transmisión. Tras el último bit, retorna a nivel bajo, conmutando el circuito al estado de escucha para recibir la respuesta del servomotor al que se dirige la instrucción.

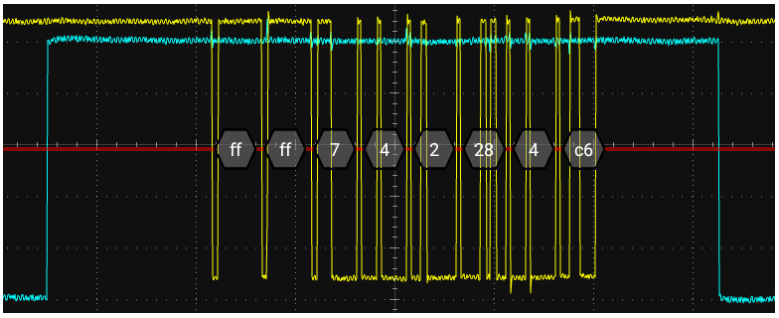


Figura 4.11: Captura de instrucción Read.
Fuente: Elaboración propia mediante osciloscopio RIGOL [44].

4.2.5. Módulo de acceso a los servomotores (`Firmware AX12A`)

Una vez descritos los actuadores, el protocolo DYNAMIXEL y la electrónica de adaptación *half-duplex*, queda establecer la forma en que el firmware del ESP32 accede al bus físico. Para ello se ha desarrollado el módulo `ax12`, que reúne en una única interfaz las operaciones necesarias para utilizar los 18 servomotores del hexápodo. El módulo constituye así la frontera entre el hardware de actuación y los componentes software que se presentan en los apartados posteriores.

La implementación se apoya en *Dynamixel2Espressif* [45], un *port* para ESP-IDF publicado por hansj66 y derivado de la biblioteca *Dynamixel2Arduino* [46], incorporado al proyecto como copia local dentro del propio componente. Sobre esta base se define una interfaz específica para la plataforma, de modo que el resto del firmware pueda solicitar operaciones sobre los actuadores sin gestionar directamente la construcción de paquetes DYNAMIXEL, la configuración de la UART o la conmutación de la línea de comunicación. La Figura 4.12 resume su posición y responsabilidades:

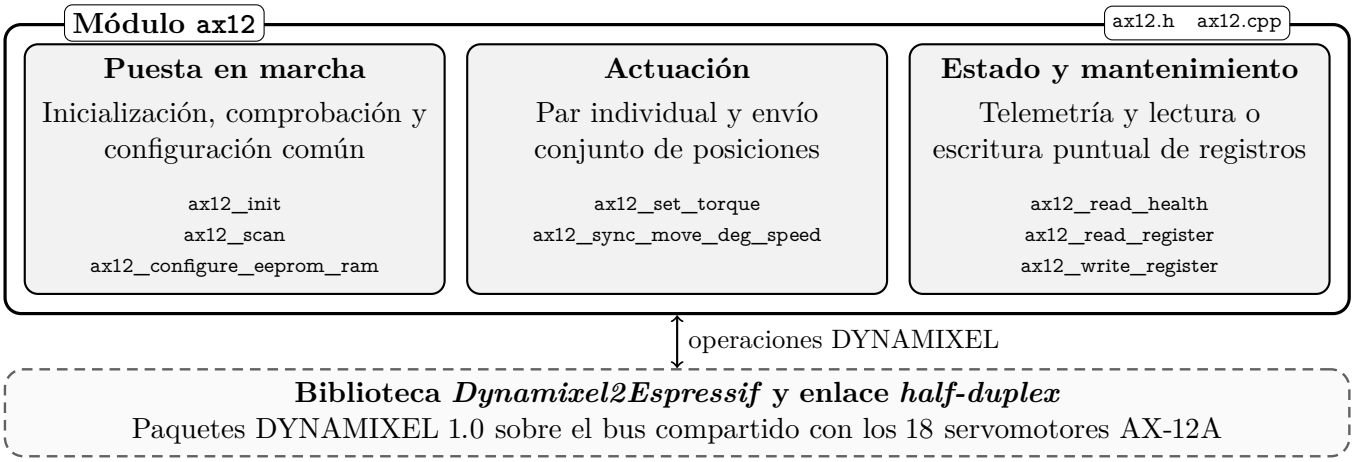


Figura 4.12: Posición y responsabilidades del módulo `ax12` en el acceso a los actuadores.
El código fuente y la documentación detallada de la API (*Application Programming Interface*), generada mediante Doxygen [47], se encuentran disponibles en el repositorio del proyecto [35].

El módulo concentra en una única interfaz todo el ciclo de uso de los actuadores, de modo que ningún otro componente del firmware necesita acceder directamente al bus. Para el movimiento coordinado agrupa en una sola trama, mediante la instrucción *Sync Write* (Tabla 4.2), las consignas de los actuadores seleccionados junto con una velocidad común, evitando una transacción independiente por servomotor sobre un enlace que todos comparten. Además, las posiciones expresadas en grados se convierten a los valores entre 0 y 1023 con que el AX-12A codifica su recorrido (Tabla 4.3).

Por último, cabe destacar que el conocimiento específico del AX-12A queda confinado aquí, en el único punto del firmware que incluye la biblioteca DYNAMIXEL y que manipula valores de registro. Sustituir los actuadores por un modelo distinto obligaría, por tanto, a reescribir la implementación de `ax12`, pero no la interfaz que utiliza el resto del firmware, que seguiría solicitando posiciones en grados.

4.3. Implementación del firmware cinemático

Una vez se ha descrito el acceso al bus mediante el módulo `ax12` y su funcionamiento lógico y eléctrico, esta sección presenta el firmware que transforma las consignas de ROS 2 en referencias para los 18 servomotores. La implementación combina módulos cinemáticos, un estado compartido sincronizado y tres tareas de FreeRTOS [48]. La Figura 4.13 recorre este flujo hasta los AX-12A.

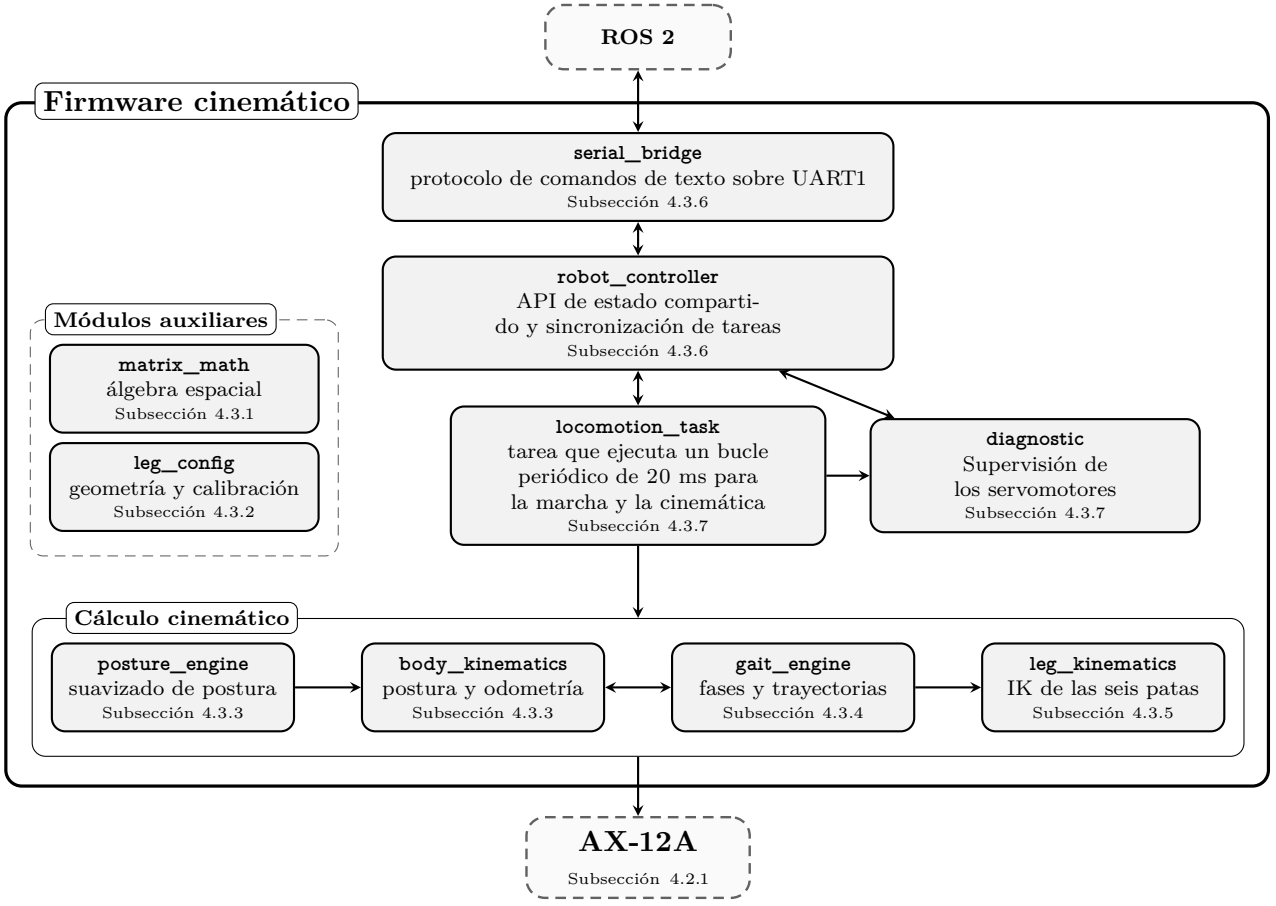


Figura 4.13: Módulos y flujos principales del firmware del hexápodo. Las flechas simples representan el flujo de datos; las dobles, un intercambio bidireccional.

Las órdenes recibidas por `serial_bridge` se trasladan a `robot_controller`, que conserva el estado compartido y sincroniza su acceso. En sentido inverso, el puente devuelve a ROS 2 la odometría y la telemetría. Cada 20 ms, `locomotion_task` obtiene los objetivos vigentes y ejecuta sucesivamente `posture_engine`, `body_kinematics`, `gait_engine` y `leg_kinematics`. El cálculo se realiza sobre una copia provisional del estado. Si las seis soluciones resultan válidas, el ciclo se confirma y las 18 referencias articulares se envían mediante una única instrucción *Sync Write*; de lo contrario, se conserva el último estado válido. Los módulos `matrix_math` y `leg_config` actúan como auxiliares y aportan, respectivamente, las operaciones de álgebra espacial y los parámetros de geometría.

Tras cada ciclo, `locomotion_task` activa `diagnostic` para consultar la telemetría o acceder a los registros de los actuadores, tareas necesarias por los esfuerzos electromecánicos a los que están sometidos los motores. Su menor prioridad reduce la interferencia con el control periódico. La comunicación serie se ejecuta en el núcleo 0 del ESP32, mientras que la locomoción y el diagnóstico se ejecutan en el núcleo 1.

4.3.1. Biblioteca de transformaciones espaciales (matrix_math)

El módulo **matrix_math** centraliza el álgebra tridimensional empleada por la cinemática del cuerpo y de las patas y por el generador de marcha. Así, todos ellos comparten representaciones y convenciones, se evita duplicar operaciones y se facilitan implementaciones eficientes. La interfaz sigue la Sección 3.1 y se resume en la Figura 4.14.

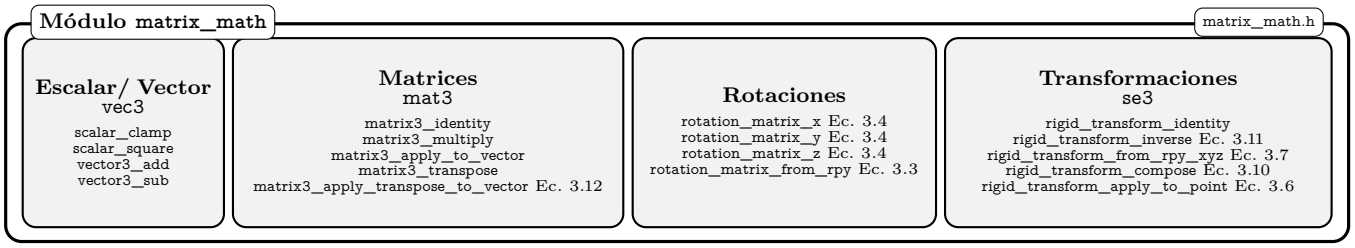


Figura 4.14: Tipos y familias de operaciones de la biblioteca **matrix_math**.

Aunque no se describen aquí todas las funciones de la biblioteca, su funcionamiento, sus parámetros y sus valores de retorno se detallan junto al código mediante Doxygen [47]. Cabe destacar que, para matrices de rotación, **matrix3_transpose** obtiene la inversa mediante la propiedad $R^{-1} = R^T$ de la Ec. 3.12. La función **matrix3_apply_transpose_to_vector** la aplica directamente a un vector. La función **rotation_matrix_from_rpy** compone las rotaciones elementales según la Ec. 3.3. Las transformaciones rígidas se almacenan con el tipo **se3**, formado por una matriz de rotación R y un vector de traslación t. Así se evita almacenar la última fila constante de la matriz homogénea de la Ec. 3.7.

4.3.2. Configuración de las patas (leg_config)

El módulo **leg_config** reúne los parámetros que diferencian las seis patas: los identificadores de los actuadores, la geometría y el anclaje al chasis, los ajustes de montaje y los límites software. La configuración se almacena en el array constante **g_leg_cfgs**, formado por seis estructuras **leg_cfg_t**. Cada entrada contiene a su vez una estructura **leg_params** con las magnitudes geométricas. La Figura 4.15 resume esta organización.

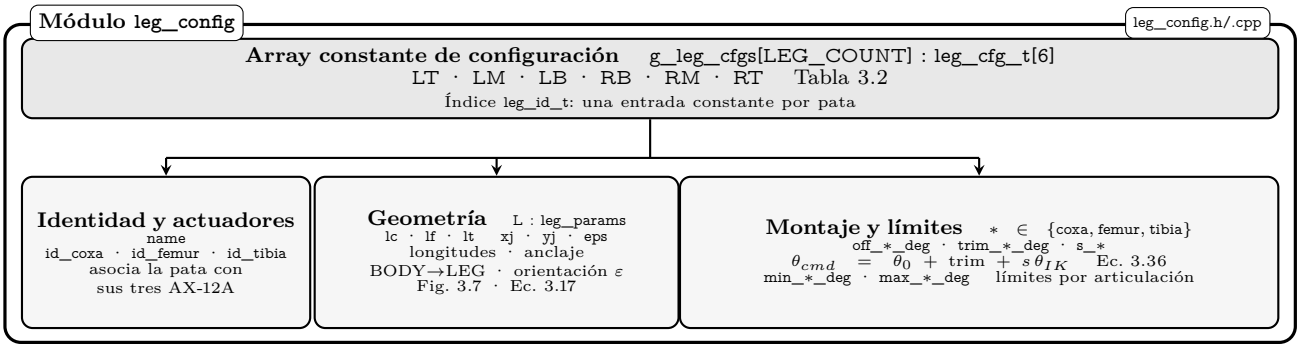


Figura 4.15: El índice **leg_id_t** selecciona una entrada **leg_cfg_t** del array constante. Cada entrada reúne los identificadores de los actuadores, la geometría **leg_params**, los ajustes de montaje y los límites articulares.

El módulo no expone funciones ni mantiene estado durante la ejecución: se limita a definir los tipos y la tabla constante empleada por la cinemática de las patas, la locomoción y el diagnóstico. Se trata, por tanto, de un módulo transversal, ya que proporciona a todos ellos una configuración común.

4.3.3. Cinemática del cuerpo (`body_kinematics`/`posture_engine`)

El tratamiento de la postura del cuerpo del robot se reparte entre dos módulos complementarios. `posture_engine` es un interpolador temporal que convierte las seis consignas de postura en trayectorias continuas en el tiempo. `body_kinematics` es el núcleo cinemático del chasis: construye su posición respecto al mundo, proporciona los cambios de coordenadas que necesitan la marcha y la cinemática inversa, y mantiene la odometría. El primero alimenta al segundo en cada ciclo de control (Figura 4.16). Sus funciones se documentan junto al código mediante Doxygen [47].

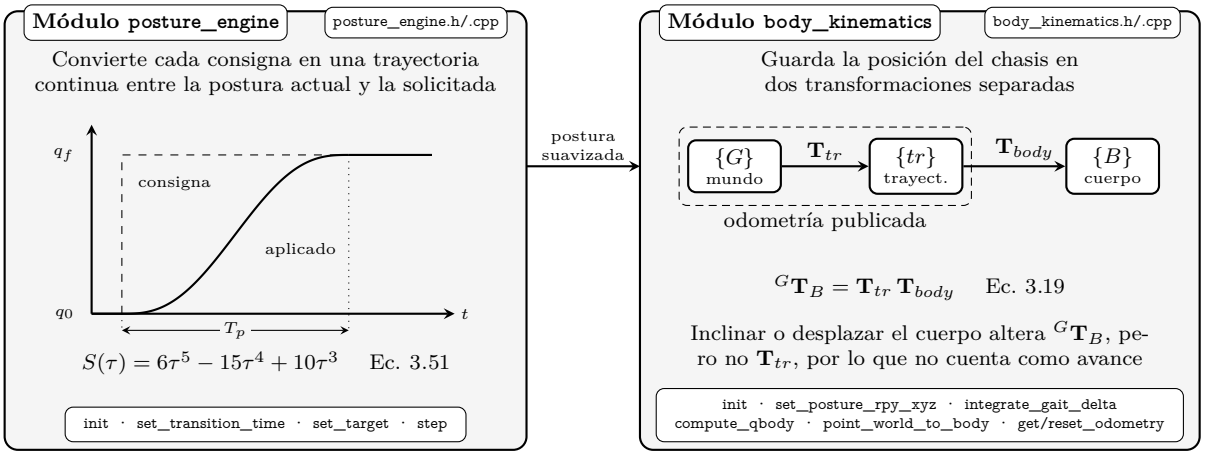


Figura 4.16: Perfil temporal aplicado a cada consigna de postura y cadena de transformaciones de la cinemática del cuerpo.

Las órdenes de postura llegan como valores aislados y su tiempo de aplicación se configura por separado, de modo que `posture_engine` interpone una transición temporal entre la consigna y el valor que realmente se aplica. Cada grado de libertad dispone de su propia estructura `poly_transition_t`, que almacena el valor inicial, el objetivo, el valor actual, el tiempo transcurrido y la duración. Las seis transiciones avanzan de forma independiente con el perfil quíntico deducido en la Subsección 3.5.2, aunque comparten una misma duración, expresada en segundos. Las rotaciones *roll*, *pitch* y *yaw* se expresan en radianes y los desplazamientos x, y y z, en milímetros.

`body_kinematics` implementa la formulación de la Sección 3.2 mediante dos transformaciones rígidas separadas. `T_tr` representa la localización odométrica: la posición en el plano, la orientación *yaw* y la altura base del cuerpo. Por su parte, `T_body` contiene la postura local instantánea, que se incorpora mediante `body_kinematics_set_posture_rpy_xyz`. La posición completa del cuerpo respecto al mundo se obtiene componiendo ambas transformaciones, de acuerdo con la Ec. 3.19.

`body_kinematics_compute_qbody` calcula, a partir de dos estados consecutivos, el movimiento relativo $\mathbf{Q}_{body,i}$ de la Ec. 3.23, con el que el generador de marcha mantiene fijos respecto al terreno los pies en apoyo, y `body_kinematics_point_world_to_body` le permite expresar en el sistema del cuerpo los objetivos de la transferencia. El módulo calcula esas transformaciones, pero no genera el movimiento articular: de eso se ocupan `gait_engine` y `leg_kinematics`. La odometría se extrae exclusivamente de `T_tr`, por lo que la postura local de `T_body` no se interpreta como avance.

4.3.4. Generador de marcha (gait_engine)

`gait_engine` decide, en cada iteración, en qué fase se encuentra cada pata y dónde debe situarse su pie, implementando la planificación de la Sección 3.5. Parte para ello del estado cinemático del cuerpo y de una consigna de desplazamiento por ciclo, e integra además la localización odométrica. Su salida es `foot_body`: seis posiciones cartesianas ${}^{B}\boldsymbol{p}_{foot}$, una por pata, en el sistema del cuerpo y en milímetros. Ese vector es la frontera con la cinemática inversa, que `locomotion_task` resuelve después para obtener los ángulos articulares (Figura 4.17).

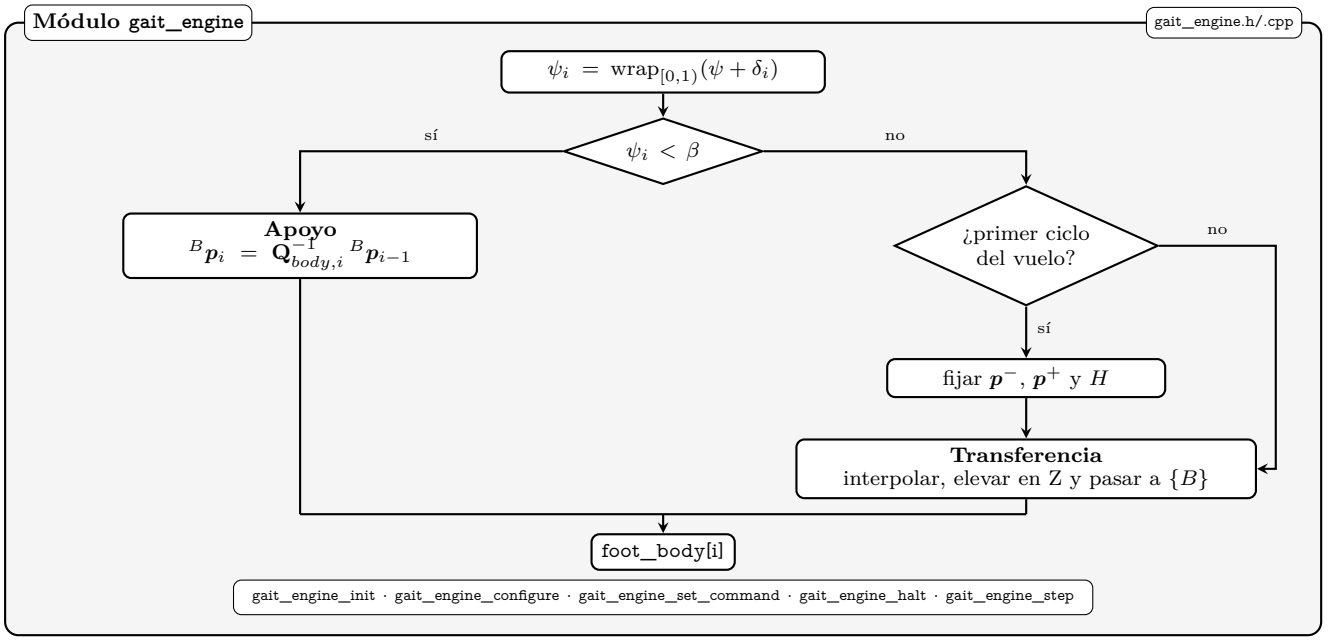


Figura 4.17: Flujo ejecutado por `gait_engine` en cada ciclo de control para determinar la fase de una pata y calcular su objetivo cartesiano durante el apoyo o la transferencia.

El flujo de la Figura 4.17 se ejecuta para cada una de las seis patas en cada ciclo de control. El módulo avanza una fase global ψ y calcula la fase local ψ_i sumándole el desfase δ_i asignado a la pata. Estos desfases se almacenan en una tabla constante, con $\delta_i = 1 - \phi_i$, para reproducir las marchas trípode, *ripple* y *amble* descritas en la Subsección 3.4.3. La comparación de ψ_i con el factor de servicio β, definido en la Ec. 3.37, determina cuál de las dos ramas se ejecuta.

Si $\psi_i < \beta$, la pata se encuentra en apoyo. En este caso, `gait_engine` aplica a la posición anterior del pie la inversa del movimiento relativo del cuerpo, $\mathbf{Q}_{body,i}^{-1}$. Esta operación implementa directamente la actualización de apoyo de la Ec. 3.25. Si $\psi_i \geq \beta$, la pata se encuentra en transferencia. Durante el primer ciclo de esta fase se almacenan la posición real de despegue $\boldsymbol{p}^-$, la posición de aterrizaje $\boldsymbol{p}^+$ y la altura H. Estos valores permanecen fijos hasta finalizar el vuelo, tal como se establece en la condición de continuidad de la Ec. 3.26. En los ciclos siguientes, el módulo interpola entre ambos extremos y añade la elevación en Z definida por la Ec. 3.40. El resultado se transforma después al sistema del cuerpo según la Ec. 3.28.

Ambas ramas terminan actualizando `foot_body[i]`, que contiene el objetivo cartesiano de la pata. Una vez procesadas las seis extremidades, estos objetivos se envían a la cinemática inversa para obtener las consignas articulares. Todos los detalles de la implementación se documentan junto al código mediante Doxygen [47].

4.3.5. Cinemática inversa de las patas (leg_kinematics)

leg_kinematics implementa la cadena que convierte el objetivo cartesiano de un pie en las tres consignas angulares de sus servomotores (Figura 4.18). La función leg_compute_deg toma de leg_config (Subsección 4.3.2) los datos particulares de cada pata, por lo que el mismo algoritmo sirve para las seis. Las funciones están documentadas con Doxygen [47].

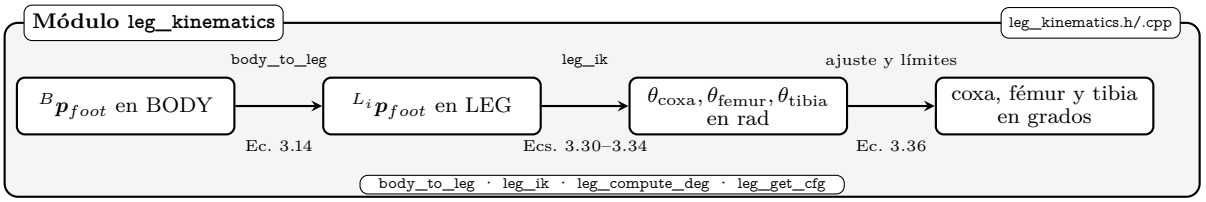


Figura 4.18: Etapas que convierten el objetivo cartesiano en consignas angulares.

El recorrido comienza con body_to_leg, que expresa el objetivo en el sistema local de la pata mediante la Ec. 3.14. A continuación, leg_ik aplica la solución geométrica de la Subsección 3.3.3 y selecciona la rama compatible con el montaje. Antes de resolverla comprueba la condición $|L_f - L_t| \leq Hip \leq L_f + L_t$; si el objetivo no es alcanzable, devuelve false. Finalmente, leg_compute_deg convierte la solución a grados y aplica el ajuste de la Ec. 3.36.

4.3.6. Estado y comunicación (robot_controller/serial_bridge)

Estos módulos forman la interfaz entre la comunicación externa y las tareas de locomoción y diagnóstico. serial_bridge traduce los comandos de texto en operaciones sobre robot_controller, que centraliza el intercambio de consignas, odometría y telemetría. De este modo, la comunicación queda desacoplada (Figura 4.19).

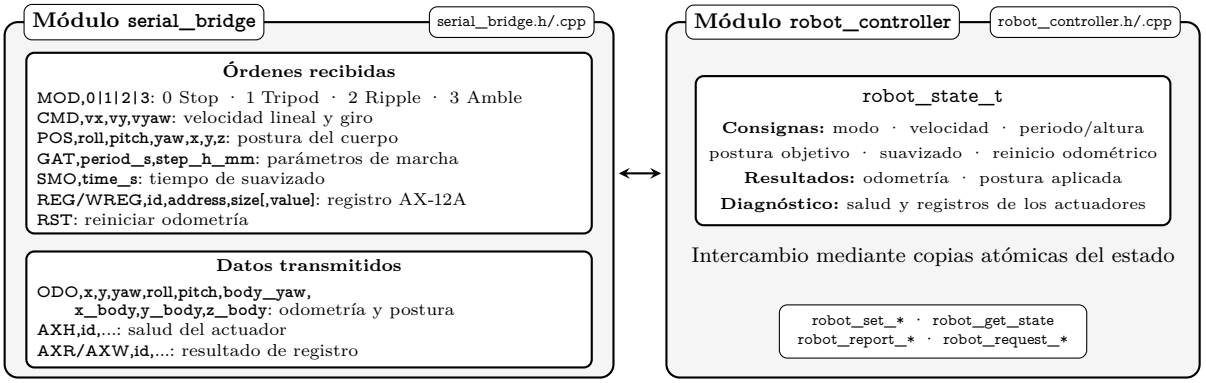


Figura 4.19: Protocolo propio de comandos de texto y acceso al estado compartido. Las funciones están documentadas con Doxygen [47].

robot_controller mantiene el estado compartido del robot, que reúne las consignas recibidas y los resultados publicados por las tareas internas. Un mutex protege las operaciones de lectura y escritura. Cada lectura devuelve una instantánea completa, por lo que las tareas pueden trabajar con datos coherentes sin mantener el bloqueo.

serial_bridge se fija al núcleo 0 y ejecuta un bucle sin periodo estricto. En cada vuelta espera datos de UART durante un máximo de 10 ms, tiempo durante el cual el núcleo atiende otras tareas. Por ello, sin tráfico el bucle se repite aproximadamente cada 10 ms, aunque su periodo no es exacto. Tras procesar una orden, consulta el estado: ODO se envía cada segundo y AXH, AXR y AXW solo cuando hay datos nuevos.

4.3.7. Locomoción y diagnóstico (`locomotion_task`/`diagnostic`)

`locomotion_task` integra los módulos anteriores en el bucle que genera las consignas de los 18 actuadores. Esta tarea inicializa y coordina la cadena cinemática completa, está fijada al núcleo 1 con prioridad 10 y se activa cada 20 ms (Figura 4.20).

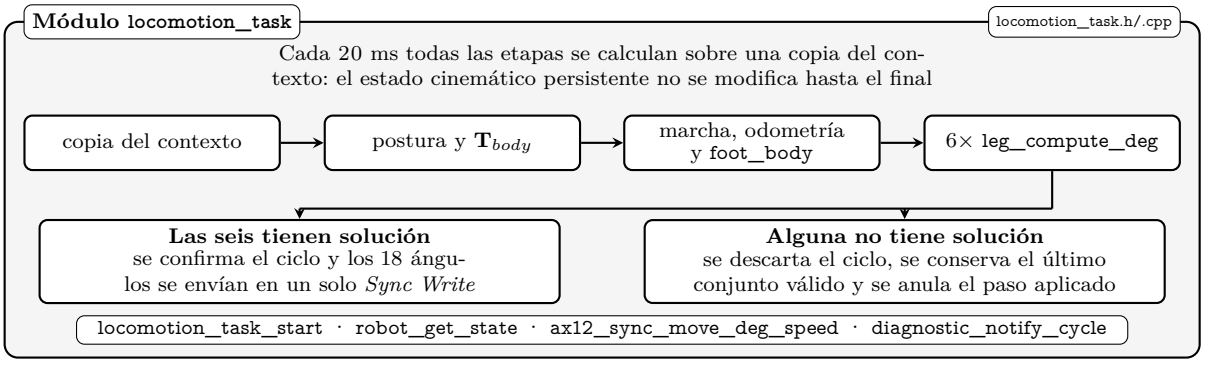


Figura 4.20: Etapas de un ciclo de 20 ms y confirmación conjunta de las seis soluciones de cinemática inversa.

Antes del bucle, la tarea inicializa el bus DYNAMIXEL a 1 Mbit/s, configura los actuadores y activa su par. También inicializa el modelo del cuerpo a una altura de 100 mm, las posiciones neutrales de la Tabla 3.1 y el patrón trípode; una orden `STOP` conserva este patrón con desplazamiento objetivo nulo.

En cada ciclo obtiene una copia atómica de las consignas y otra provisional del contexto cinemático, sobre la que actualiza la postura, la marcha, la odometría y las seis IK. Solo si todas son válidas confirma el contexto y envía los 18 ángulos. En caso contrario, conserva el último conjunto válido y anula el paso aplicado con `gait_engine_halt`, para que una consigna nula permita reanudar la marcha. Un cambio de patrón mantiene el objetivo a cero durante dos periodos antes de activar el nuevo modo. De esta manera se asegura una transición suave. Finalmente, la tarea notifica a `diagnostic` con independencia del resultado de la IK, para que se ejecute en la ventana de tiempo restante.

`diagnostic`, fijada al núcleo 1 con prioridad 9, ejecuta fuera del control periódico las operaciones auxiliares sobre los servomotores (Figura 4.21).

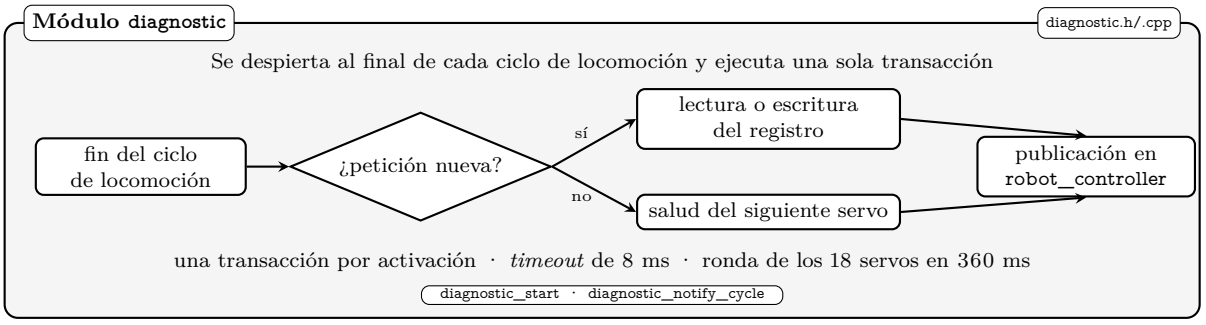


Figura 4.21: Selección de una única operación auxiliar después de cada ciclo de locomoción.

La tarea permanece bloqueada hasta la notificación del final de cada ciclo. En cada activación atiende primero una petición `REG`/`WREG`, si esta no existe, consulta la tensión, la temperatura, la carga y el error del siguiente servo de la ronda. Ejecuta como máximo una transacción con un timeout de 8 ms y publica el resultado en `robot_controller`. El mutex interno de `ax12` serializa el acceso al bus con la locomoción.

4.4. Subsistema de alto nivel e integración ROS 2

Una vez resuelto el control en tiempo real, el subsistema de alto nivel se plantea como una capa de coordinación, percepción y supervisión. Su cometido no es ejecutar el control temporal estricto del robot, sino organizar los módulos de mayor nivel de abstracción y comunicarse de forma desacoplada con el firmware de bajo nivel. Esta capa se ha desarrollado siguiendo el modelo de nodos y comunicaciones de ROS 2 [2]. Su extensa comunidad y su ecosistema de paquetes facilitan la integración de sensores, algoritmos de planificación y control, y herramientas de visualización, además de la modularidad, que permite desarrollar y desplegar cada componente de manera independiente.

4.4.1. Interfaz ROS 2 del puente serie

El primer nodo desarrollado es `/esp32_bridge`, incluido en el paquete `hexapod_bridge`. Este nodo constituye la frontera entre dos modelos de ejecución: hacia ROS 2 ofrece una interfaz basada en mensajes y tópicos, mientras que hacia el ESP32 mantiene una comunicación UART basada en las órdenes de texto descritas en la Subsección 4.3.6. Se trata, en esencia, de un traductor entre las interfaces de alto nivel y el protocolo del firmware. La interfaz pública del robot se agrupa bajo el espacio de nombres `/hexapod`, en el que se exponen los tópicos recogidos en la Tabla 4.5; su implementación completa está disponible en el repositorio del proyecto [35].

Tópico	Mensaje	Sentido	Función
/hexapod/cmd_vel	geometry_msgs/msg/Twist	**Entrada**	Consigna de velocidad lineal en los ejes x e y y velocidad angular alrededor de z.
/hexapod/robot_command	std_msgs/msg/String	**Entrada**	Configuración de marcha y postura, reinicio de odometría y acceso a registros AX-12A.
/hexapod/odom_model	nav_msgs/msg/Odometry	**Salida**	Odometría teórica calculada a partir del modelo de marcha.
/hexapod/body_pose	geometry_msgs/msg/Pose	**Salida**	Orientación y desplazamiento de cuerpo aplicados por el firmware.
/hexapod/servo_health	std_msgs/msg/String	**Salida**	Tensión, temperatura, carga y alarmas de los 18 actuadores.
/hexapod/servo_register	std_msgs/msg/String	**Salida**	Resultado de una lectura o escritura del mapa de registros AX-12A.

Tabla 4.5: Interfaz ROS 2 expuesta por el nodo `/esp32_bridge`.

1. **Control continuo del movimiento.** Las consignas se reciben mediante `/hexapod/cmd_vel`. El mensaje `Twist` expresa las velocidades lineales en m/s y la angular en rad/s, conforme a las convenciones de ROS 2 [2]. El puente convierte las primeras a mm/s y calcula las velocidades máximas admisibles a partir del recorrido permitido en cada paso y de la duración de la fase de apoyo. Si la petición supera los límites calculados, las tres componentes se reducen mediante un factor común.

   Por ejemplo, en marcha trípode con un periodo de 2 s, la fase de apoyo ocupa la mitad del ciclo y dura $2 \cdot (1/2) = 1$ s, de acuerdo con el patrón descrito en la Subsección 3.4.3. Con recorridos máximos de 120 mm en x, 50 mm en y y 0,35 rad alrededor de z, se obtienen los límites:

   $$v_{x,\text{máx}} = 120 \text{ mm/s}, \qquad v_{y,\text{máx}} = 50 \text{ mm/s}, \qquad \omega_{z,\text{máx}} = 0{,}35 \text{ rad/s}.$$

   Para $(v_x, v_y) = (0{,}12, 0{,}05)$ m/s, la demanda normalizada vale $120/120 + 50/50 = 2$ y el factor $1/2$ produce $(v_x, v_y) = (60, 25)$ mm/s, conservando la dirección.

2. **Configuración y diagnóstico.** Las operaciones discretas se reciben mediante `/hexapod/robot_command`. Este tópico transporta cambios de modo (MOD), periodo y altura de paso (GAT), postura (POS), suavizado (SMO), reinicio de odometría y accesos a los registros de los actuadores (REG/WREG). Las órdenes CMD se descartan expresamente en este canal, de modo que el movimiento solo pueda introducirse mediante la interfaz estándar `Twist`. Los cambios de marcha se conservan localmente porque condicionan los límites aplicados a las consignas posteriores.

3. **Publicación de telemetría.** En el sentido ESP32–ROS 2, el puente lee continuamente la UART y clasifica cada línea por su prefijo. Una trama ODO origina dos publicaciones: `/hexapod/odom_model` contiene la traslación planar y la orientación estimadas, mientras que `/hexapod/body_pose` representa la postura aplicada al cuerpo. Las posiciones se convierten de milímetros a metros y los ángulos se expresan mediante cuaterniones. Las tramas AXH se publican como estado de los actuadores, y las respuestas AXR/AXW, como resultados del acceso a sus registros.

La interfaz no fija qué nodos deben utilizarla: cualquier publicador que respete los tipos establecidos puede enviar consignas y cualquier suscriptor puede consumir la telemetría. Como ejemplo, la Figura 4.22 muestra `/teleop_twist_keyboard` [49] publicando mensajes `geometry_msgs/msg/Twist` en `/hexapod/cmd_vel`.

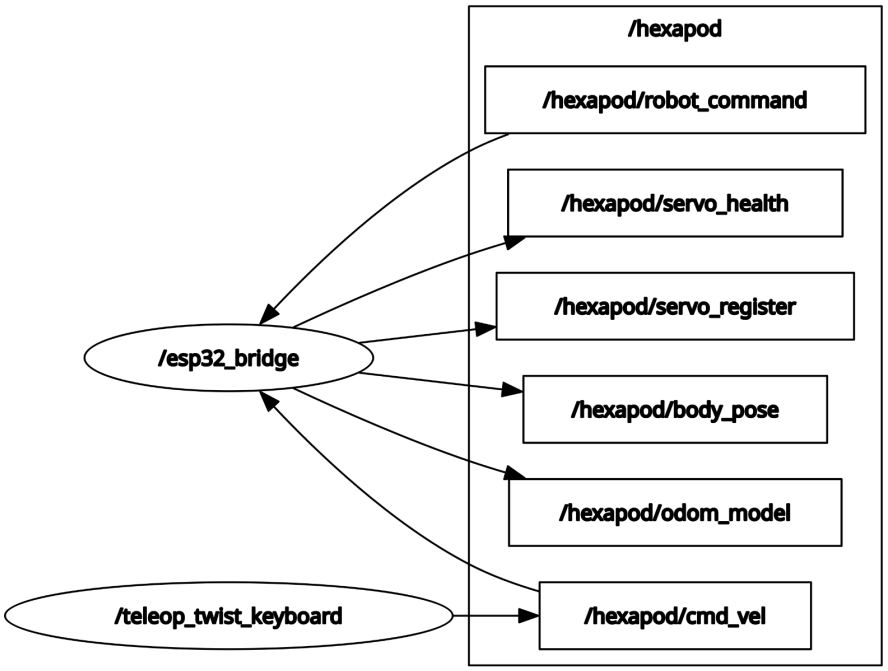


Figura 4.22: Conexión de `/teleop_twist_keyboard` con `/esp32_bridge` mediante `/hexapod/cmd_vel`, visualizada con `rqt_graph`.
Fuente: Elaboración propia.

Otra integración habitual en este tipo de soluciones consiste en conectar periféricos externos mediante paquetes de adaptación. En el caso de un mando de videojuegos, el paquete `joy` [50] publica su estado mediante mensajes `sensor_msgs/msg/Joy`, lo que ofrece otra posible vía de integración. Sin embargo, la necesidad de disponer de una interfaz que se adaptase al conjunto de tópicos y que, a su vez, sirviera como herramienta de supervisión, llevó a desarrollar un panel web específico, que se describe en la Subsección 4.4.2.

4.4.2. Panel web como cliente de la interfaz del hexápodo

El panel web se desarrolló como un cliente de la interfaz proporcionada por `/esp32_bridge`. Su nodo `/web_panel` convierte las acciones del navegador en publicaciones ROS 2 y presenta la telemetría recibida mediante sus suscripciones. De este modo, reúne el control, la configuración y la supervisión sin modificar la interfaz del puente. La relación entre ambos nodos se muestra en la Figura 4.23, y sus vistas se recogen a mayor tamaño en el Anexo E.

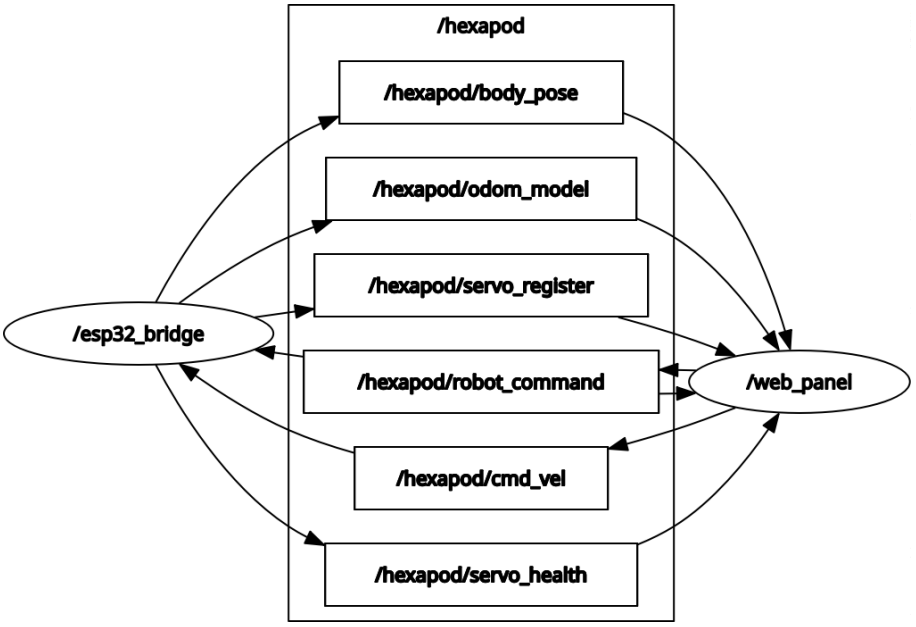


Figura 4.23: Conexión del panel web con la interfaz de `/esp32_bridge`.
Fuente: Elaboración propia.

1. **Control del movimiento y configuración de la marcha.** El bloque de teleoperación de la Figura E.1 ofrece un área de control bidimensional para las velocidades v_x y v_y, un deslizador independiente para el giro y campos numéricos con los que introducir consignas precisas. Desde ella se seleccionan también los modos *tripod*, *ripple* y *amble*, el periodo y la altura de paso. Mientras existe un modo de marcha activo, `/web_panel` publica la velocidad en `/hexapod/cmd_vel` a 10 Hz. Esa renovación periódica actúa además como señal de actividad: si el puente no recibe una consigna nueva antes de un segundo, la considera caducada y ordena velocidad nula, de modo que el robot se detiene si el navegador deja de responder. Los cambios discretos de modo y de parámetros, en cambio, se envían una sola vez mediante `/hexapod/robot_command`.

2. **Postura y supervisión del movimiento.** Esa misma vista permite ajustar los tres ángulos de orientación y los tres desplazamientos del cuerpo, junto con el tiempo de suavizado de postura. Para cada componente se presentan conjuntamente el objetivo solicitado y el valor aplicado que el firmware publica en `/hexapod/body_pose`.

3. **Diagnóstico de los actuadores.** El panel de la Figura E.2 distribuye los 18 actuadores según su posición en el robot y presenta, para cada uno, su identificador, articulación, temperatura, carga y tensión. Al seleccionar uno se abre la tabla de registros AX-12A de la Figura E.3, desde la que pueden consultarse todos ellos y escribirse únicamente los que la aplicación habilita. Las solicitudes utilizan `/hexapod/robot_command` y las respuestas, `/hexapod/servo_register`.

4.4.3. Percepción y navegación reactiva

Dotar a la plataforma de percepción del entorno es el primer paso hacia un sistema de navegación. Se ha optado por un LiDAR porque permite obtener resultados inmediatos, como una navegación reactiva basada en la detección de obstáculos.

El sensor es el RPLIDAR A1M8 [39], presentado en la Sección 4.1, que mide distancias por triangulación láser. Su integración en ROS 2 corre a cargo de `sllidar_ros2` [51], el controlador que mantiene el propio fabricante, cuyo nodo publica el tópico `/scan` con mensajes estándar `sensor_msgs/msg/LaserScan` [2]. El ángulo cero de `/scan` corresponde al eje X de la Figura 4.24, que apunta hacia el motor y es solidario a la carcasa, por lo que el sensor se ha atornillado con ese eje alineado con el eje X del sistema del cuerpo $\{B\}$, es decir, con el frente del robot. La Figura 4.25 muestra un barrido obtenido con este montaje.

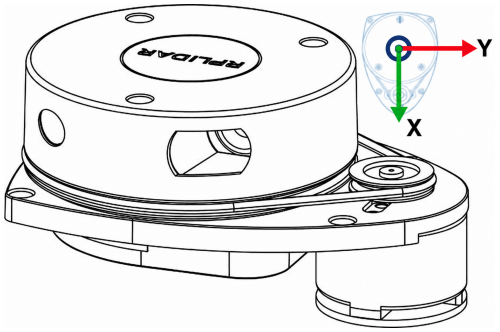


Figura 4.24: LiDAR RPLIDAR A1M8 y su sistema de referencia.
Fuente: SLAMTEC [39] y `sllidar_ros2` [51].

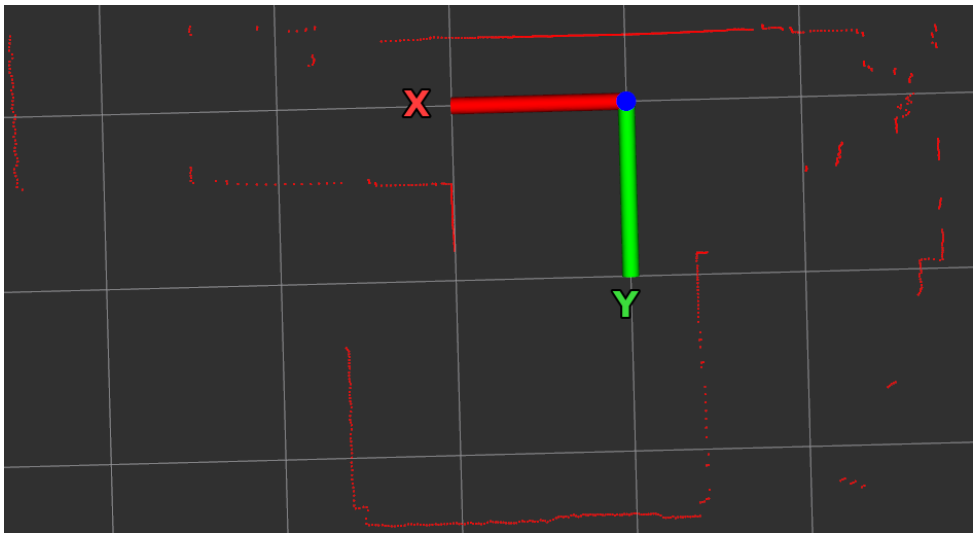


Figura 4.25: Nube de puntos publicada en `/scan` y visualizada en RViz 2 [24]
Fuente: Elaboración propia.

Como demostración se ha desarrollado un evitador puramente reactivo en dos etapas (Figura 4.26): un filtrado de cada barrido y un nodo que decide el movimiento.

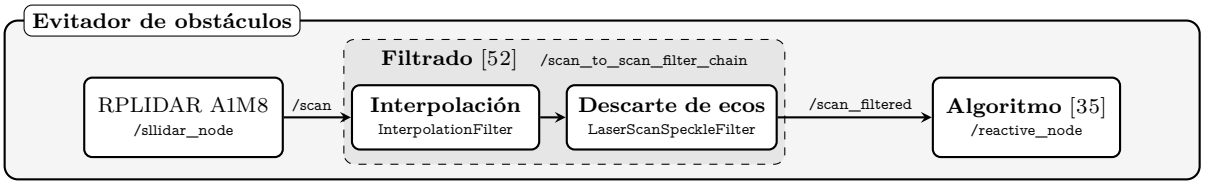


Figura 4.26: Arquitectura en dos etapas del evitador de obstáculos.

El filtrado lo realiza el nodo `/scan_to_scan_filter_chain` de `laser_filters` [52], que publica `/scan_filtered`. El primer filtro rellena cada tramo sin eco con la media de sus extremos, de modo que un cristal o un objeto oscuro se percibe como obstáculo, y el segundo descarta las medidas aisladas a menos de 1 m, propias de retornos espurios.

El algoritmo sigue el esquema de tres sectores habitual en los evitadores reactivos: no construye mapas ni planifica trayectorias, sino que toma una única decisión por barrido. El robot se modela como un círculo de 0,35 m de radio centrado en el LiDAR, que contiene los pies durante la marcha, y de cada barrido se extraen cuatro distancias (Figura 4.27): la libre en el pasillo frontal de 0,7 m de ancho que el robot barrería al avanzar (d_{f}), la mínima en cada sector lateral, entre 35° y 120° (d_{izq} y d_{der}), y la mínima en todo el entorno (d_{min}). Un sector sin medidas válidas se considera ocupado. Con ellas se aplican tres reglas:

1. **Avance.** Con al menos 0,5 m libres en el pasillo, el robot avanza a 0,08 m/s y, sin cambiar el rumbo, se aparta lateralmente de las paredes situadas a menos de 0,8 m, a una velocidad proporcional a la diferencia entre ambos lados y limitada a 0,015 m/s. Aprovecha así la capacidad omnidireccional del hexápodo.

2. **Giro.** Si el pasillo está ocupado pero el círculo del robot está libre, gira sobre sí mismo a 0,25 rad/s hacia el lateral más despejado. El sentido se mantiene hasta reanudar el avance, que exige 0,6 m libres en lugar de 0,5 m; esta histéresis evita que pequeñas variaciones entre barridos lo hagan oscilar entre avanzar y girar.

3. **Parada.** Si alguna medida cae dentro del círculo del robot, se ordena velocidad nula, ya que un giro podría llevar una pata contra el obstáculo.

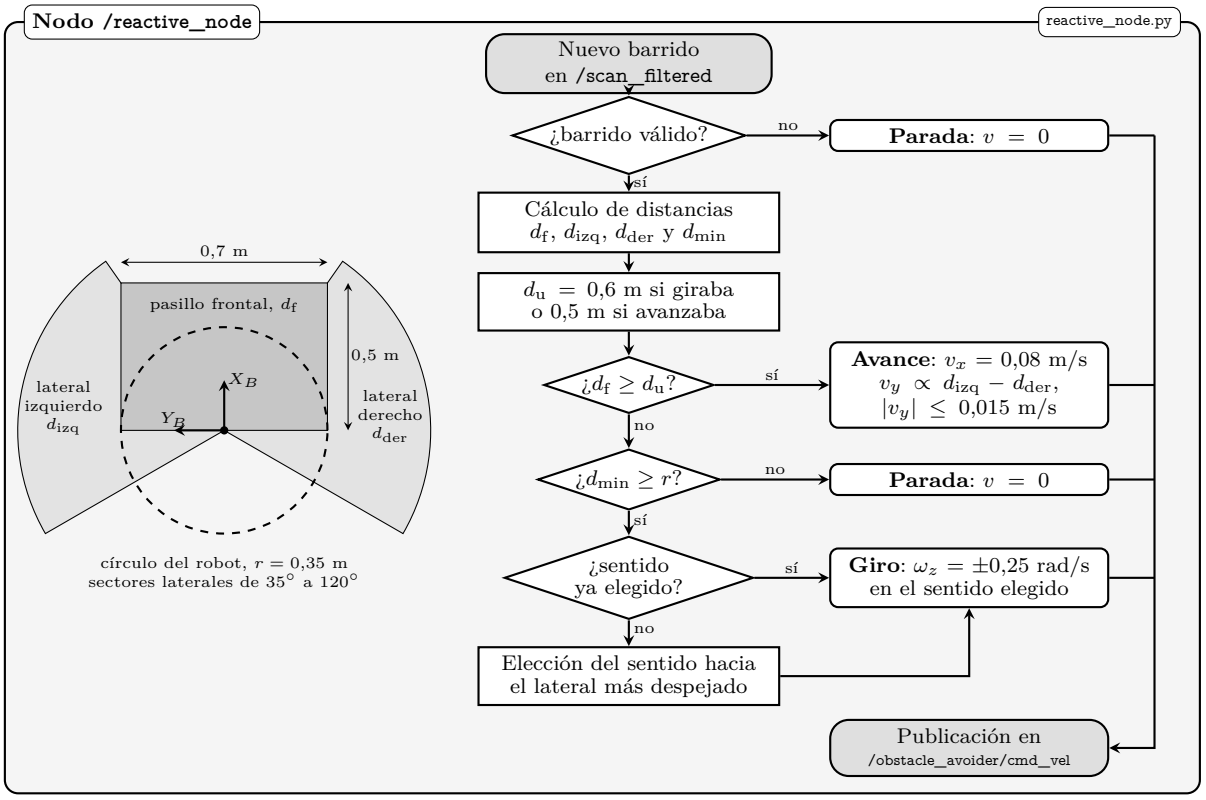


Figura 4.27: Zonas de medida en el sistema del cuerpo $\{B\}$ y diagrama de flujo aplicado en cada barrido.

5. Validación, conclusiones y líneas futuras

5.1. Validación funcional

...

5.2. Conclusiones

...

5.3. Líneas futuras

...

Bibliografía

[1] Bruno Siciliano and Oussama Khatib, editors. *Springer Handbook of Robotics*. Springer International Publishing, Cham, Switzerland, 2 edition, 2016.

[2] Open Robotics. ROS 2 documentation: Jazzy. https://docs.ros.org/en/jazzy/, 2024. Accedido: 2026-05-11.

[3] Interbotix / Trossen Robotics. ArbotiX-M Robocontroller. https://www.interbotix.com/arbotix-robocontroller, s. f. Accedido: 2026-06-14.

[4] Vanadium Labs. PyPose and NUKE (Nearly Universal Kinematics Engine). https://github.com/vanadiumlabs/pypose, 2014. Accedido: 2026-10-07.

[5] Robert B. McGhee and Andrew A. Frank. On the stability properties of quadruped creeping gaits. *Mathematical Biosciences*, 3(3-4):331–351, 1968.

[6] Germán Fernández-Villalobos. Neurobiología del control motor en insectos. *Archivos de Neurociencias*, 24(1):17–27, 2019. https://doi.org/10.31157/an.v24i1.171.

[7] M. Russell. Odex I: The first functionoid. *Robotics Age*, 5(5):12–18, 1983.

[8] Kenneth J. Waldron and Robert B. McGhee. The Adaptive Suspension Vehicle. *IEEE Control Systems Magazine*, 6(6):7–12, 1986.

[9] Rodney A. Brooks. A robot that walks; emergent behaviors from a carefully evolved network. *Neural Computation*, 1(2):253–262, 1989.

[10] Lynxmotion. SES-V1 legged robot platforms. https://www.lynxmotion.com/ses-v1-legged-robot/, 2022.

[11] Freenove Creative Technology. Freenove Hexapod Robot Kit. https://github.com/Freenove/Freenove_Hexapod_Robot_Kit, 2023.

[12] Interbotix / Trossen Robotics. PhantomX Hexapod MK-IV features. https://www.youtube.com/watch?v=0fMXJBiTIVo, 2020.

[13] ROBOTIS Co., Ltd. DYNAMIXEL X-Series e-Manual. https://emanual.robotis.com/docs/en/dxl/x/, 2022.

[14] Max.K. ZeroBug: DIY hexapod robot. Hackaday.io. https://hackaday.io/project/180534-zerobug-diy-hexapod-robot, 2021. Accedido: 2026-10-07.

[15] Hiwonder. miniHexa AI hexapod robot. https://www.hiwonder.com/collections/hexapod-robot, 2024. Accedido: 2026-05-07.

[16] ROBOTIS. DYNAMIXEL Protocol 1.0. https://emanual.robotis.com/docs/en/dxl/protocol1/, 2024. Accedido: 2026-02-02.

[17] ROBOTIS. DYNAMIXEL AX-12A e-Manual. https://emanual.robotis.com/docs/en/dxl/ax/ax-12a/, 2024. Accedido: 2026-02-02.

[18] Morgan Quigley, Ken Conley, Brian Gerkey, Josh Faust, Tully Foote, Jeremy Leibs, Rob Wheeler, and Andrew Y. Ng. ROS: An open-source robot operating system. In *ICRA Workshop on Open Source Software*, 2009.

[19] ROS Wiki. ROS technical overview. https://wiki.ros.org/ROS/Technical_Overview, 2024. Accedido: 2026-10-08.

[20] Object Management Group. Data Distribution Service (DDS) specification. Specification formal/15-04-10, Object Management Group, 2015. Versión 1.4. https://www.omg.org/spec/DDS/1.4/PDF. Accedido: 2026-05-11.

[21] Open Navigation LLC. Nav2 documentation. https://docs.nav2.org, 2024. Accedido: 2026-05-11.

[22] Steve Macenski and Ivona Jambrecic. SLAM Toolbox: SLAM for the dynamic world. *Journal of Open Source Software*, 6(61):2783, 2021.

[23] Thomas Moore and Daniel Stouch. A generalized extended Kalman filter implementation for the Robot Operating System. In *Intelligent Autonomous Systems 13*, volume 302 of *Advances in Intelligent Systems and Computing*, pages 335–348. Springer, 2016.

[24] Open Robotics. RViz2 documentation. https://docs.ros.org/en/jazzy/p/rviz2/, 2024. Accedido: 2026-05-11.

[25] Open Robotics. Gazebo simulator. https://gazebosim.org/, 2024. Accedido: 2026-05-11.

[26] Vanadium Labs. BioloidController: Arduino library for Bioloid robot control. https://github.com/vanadiumlabs/arbotix, 2011. Accedido: 2026-05-11.

[27] Espressif Systems. ESP-IDF: Espressif IoT Development Framework. https://docs.espressif.com/projects/esp-idf/en/stable/esp32/, 2024. Accedido: 2026-05-11.

[28] Auke Jan Ijspeert. Central pattern generators for locomotion control in animals and robots: A review. *Neural Networks*, 21(4):642–653, 2008.

[29] John J. Craig. *Introduction to Robotics: Mechanics and Control*. Pearson, New York, NY, USA, 4 edition, 2018.

[30] Benjamin Tam, Fletcher Talbot, Ryan Steindl, Alberto Elfes, and Navinda Kottege. OpenSHC: A versatile multilegged robot controller. *IEEE Access*, 8:188908–188926, 2020.

[31] Aldebaran Robotics. Roll, pitch and yaw. Documentación NAO/Pepper OS 2.4. http://doc.aldebaran.com/2-4/_images/rollPitchYaw.png, 2016. Adaptado a escala de grises. Accedido: 2026-09-28.

[32] The MathWorks, Inc. MATLAB. https://www.mathworks.com/products/matlab.html. Accedido: 2026-07-22.

[33] Stefano Arrigoni, Marco Zangrandi, Giovanni Bianchi, and Francesco Braghin. Control of a hexapod robot considering terrain interaction. *Robotics*, 13(10):142, 2024.

[34] Xingji Duan, Weihai Chen, Shouqian Yu, and Jingmeng Liu. Tripod gaits planning and kinematics analysis of a hexapod robot. In *2009 IEEE International Conference on Control and Automation*, pages 1850–1855, 2009.

[35] Josué Bohórquez Cristóbal. hexapod-ros2: código fuente y documentación del proyecto. https://github.com/josuebcc/hexapod-ros2, 2026. Accedido: 2026-08-02.

[36] Raspberry Pi Ltd. Raspberry Pi 4 Model B Datasheet. https://pip.raspberrypi.com/documents/RP-008341-DS-raspberry-pi-4-datasheet.pdf, 2025. Accedido: 2026-07-07.

[37] Espressif Systems. ESP32 series datasheet. https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf, 2024. Accedido: 2026-05-11.

[38] XLSEMI. XL4015: 5A 180KHz 36V Buck DC to DC Converter. https://www.xlsemi.com/datasheet/XL4015-EN.pdf, s. f. Accedido: 2026-07-07.

[39] SLAMTEC. RPLIDAR A1: especificaciones técnicas. https://www.slamtec.com/en/lidar/a1spec, s. f. Accedido: 2026-10-08.

[40] Altium Limited. Altium Designer. https://www.altium.com/altium-designer. Accedido: 2026-07-22.

[41] Texas Instruments. TXB0104: 4-bit bidirectional voltage-level translator with automatic direction sensing and ±15-kV ESD protection. https://www.ti.com/lit/ds/symlink/txb0104.pdf, 2025. Accedido: 2026-07-21.

[42] Texas Instruments. SN74LVC2G241: Dual buffer/driver with 3-state outputs. https://www.ti.com/lit/ds/symlink/sn74lvc2g241.pdf, 2019. Accedido: 2026-07-21.

[43] Traco Power. TSR 1 Series: 1 A step-down switching regulators. https://www.tracopower.com/tsr1-datasheet, 2024. Accedido: 2026-07-21.

[44] RIGOL Technologies. DHO800 Series: Digital oscilloscope. https://www.rigolna.com/products/digital-oscilloscopes/dho800/, s. f. Accedido: 2026-10-08.

[45] hansj66. dynamixel2espressif: Dynamixel protocol library for ESP32. https://components.espressif.com/components/hansj66/dynamixel2espressif, 2024.

[46] ROBOTIS. Dynamixel2Arduino: DYNAMIXEL protocol library for Arduino. https://github.com/ROBOTIS-GIT/Dynamixel2Arduino. Repositorio oficial. Accedido: 2026-08-01.

[47] Dimitri van Heesch. Doxygen: Source code documentation generator. https://www.doxygen.nl/, s. f. Accedido: 2026-07-13.

[48] Amazon Web Services. FreeRTOS — market leading RTOS. https://www.freertos.org, 2024. Accedido: 2026-05-11.

[49] Austin Hendrix and Graylin Trevor Jay. teleop_twist_keyboard: teleoperación mediante teclado para ROS 2. https://docs.ros.org/en/jazzy/p/teleop_twist_keyboard/___README.html, s. f. Versión 2.4.1 para ROS 2 Jazzy. Accedido: 2026-09-01.

[50] ROS Drivers. joy: controlador para joysticks y mandos genéricos en ROS 2. https://github.com/ros-drivers/joystick_drivers/tree/ros2/joy, s. f. Versión 3.3.0. Accedido: 2026-09-01.

[51] SLAMTEC. sllidar_ros2: ROS 2 driver for SLAMTEC RPLIDAR. https://github.com/Slamtec/sllidar_ros2, 2024. Accedido: 2026-09-13.

[52] Tully Foote and Jon Binney. laser_filters: filters for ROS 2 LaserScan messages, version 2.0.10. https://github.com/ros-perception/laser_filters. Accedido: 2026-10-14.

Anexo A: Aspectos éticos, económicos, sociales y ambientales

El apartado "Requisitos de las acreditaciones internacionales EUR-ACE y ABET" de la Normativa de TFT de la ETSIT-UPM establece que "*La memoria del TFT del GITST, GIB y MUIT, y en general la de aquellas titulaciones que hayan obtenido o para las que se desee solicitar una acreditación internacional EUR-ACE o ABET, debe mostrar conciencia de la responsabilidad de la aplicación práctica de la ingeniería, el impacto social y ambiental, el compromiso con la ética profesional, la responsabilidad y las normas de la aplicación práctica de la ingeniería, así como sobre las prácticas de gestión de proyectos, gestión, y control de riesgos, entendiendo sus limitaciones*".

Este anexo obligatorio del TFT tendrá un carácter sintético con los siguientes apartados:

A1. Introducción

Breve descripción del contexto del proyecto, objetivos, necesidades que pretende cubrir o problemas que pretende resolver, centrándose en su relación con los temas sociales, económicos, éticos, legales y/o ambientales que se hayan identificado.

A2. Descripción de impactos relevantes relacionados con el proyecto

Síntesis del trabajo realizado en la fase 3, de selección y descripción de impactos. Presentar y justificar las conclusiones a las que se haya llegado sobre cuáles son los asuntos más relevantes relacionados con la sostenibilidad social, económica o ambiental, así como los principales grupos de interés identificados y que se han considerado en los análisis posteriores.

A3. Análisis detallado de alguno de los impactos

Síntesis del trabajo de análisis realizado.

A4. Conclusiones

Valorar el proyecto desde un punto de vista ético, social, económico y medioambiental y justificar si el uso de criterios de sostenibilidad ha aportado o puede aportar valor añadido al proyecto.

Anexo B: Presupuesto económico

El apartado "Requisitos de las acreditaciones internacionales EUR-ACE y ABET" de la Normativa de TFT de la ETSIT-UPM establece que "*La memoria del TFT del GITST, GIB y MUIT, y en general la de aquellas titulaciones que hayan obtenido o para las que se desee solicitar una acreditación internacional EUR-ACE o ABET, ... debe incluir un presupuesto económico*". A modo de ejemplo, la tabla del presupuesto de un proyecto podría ser la siguiente:

COSTE DE MANO DE OBRA (coste directo)	**Horas**	**Precio/hora**	**Total**
	300	15 €	**4.500 €**

COSTE DE RECURSOS MATERIALES (coste directo)	**Precio de compra**	**Uso en meses**	**Amortización (en años)**	**Total**
Ordenador personal (Software incluido).....	1.500,00 €	6	5	150,00 €
Impresora láser	500,00 €	6	5	50,00 €
Otro equipamiento				

COSTE TOTAL DE RECURSOS MATERIALES	**200,00 €**

GASTOS GENERALES (costes indirectos)	15 %	sobre CD	**705,00 €**
BENEFICIO INDUSTRIAL	6 %	sobre CD + CI	**324,30 €**

MATERIAL FUNGIBLE	
Impresión	**100,00 €**
Encuadernación	**300,00 €**

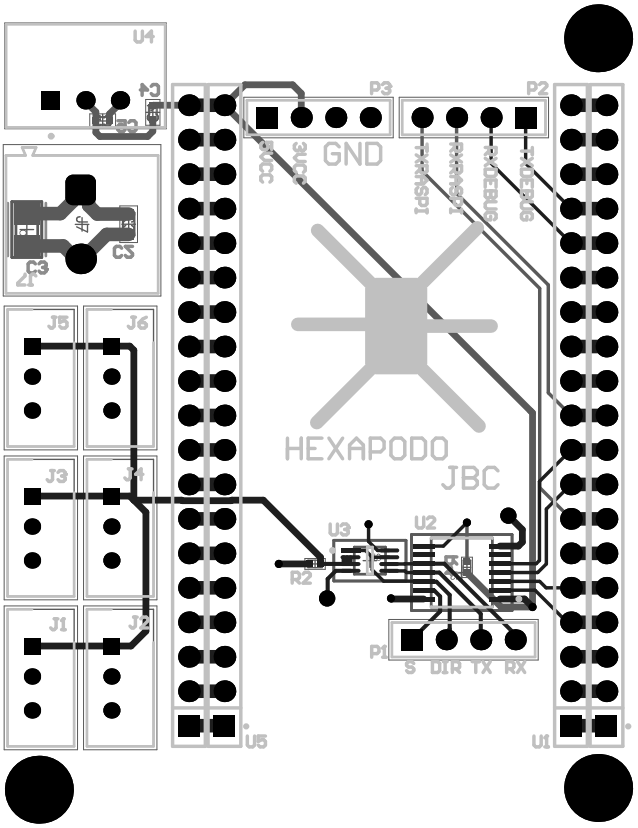
SUBTOTAL PRESUPUESTO		**6.129,30 €**
IVA APLICABLE	21 %	**1.287,15 €**

TOTAL PRESUPUESTO	**7.416,45 €**

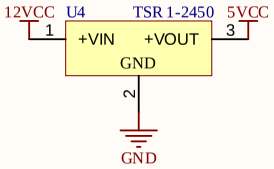
Tabla B.1: Presupuesto económico

Anexo C: Documentación técnica de la PCB principal

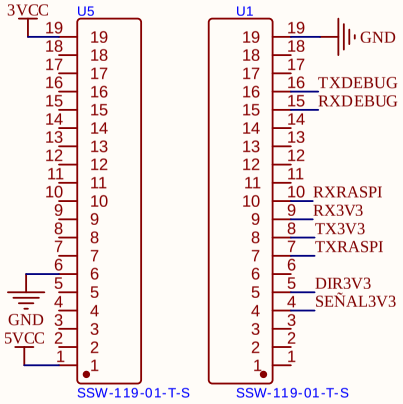
Este anexo recoge el diseño electrónico de la PCB principal del hexápodo, desarrollado en Altium Designer, y complementa la descripción funcional del Capítulo 4.



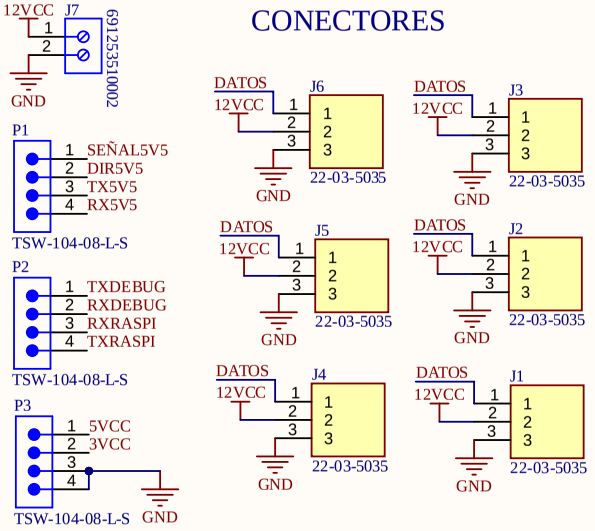
ALIMENTACION



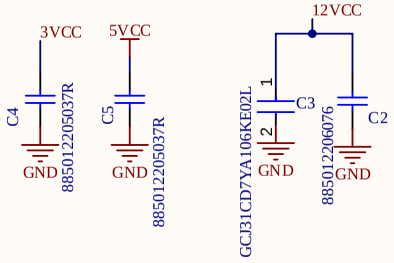
ZÓCALO ESP32



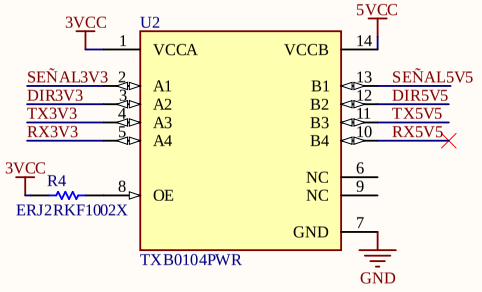
CONECTORES



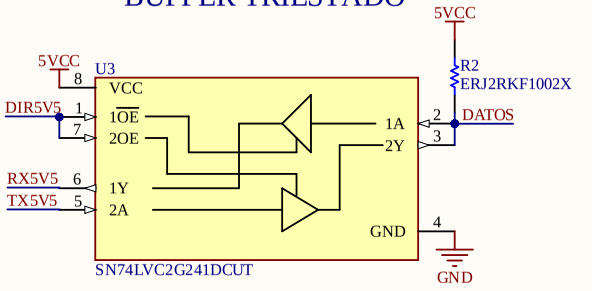
CONDENSADORES



CONVERSOR DE NIVEL



BUFFER TRIESTADO



	TITLE: =Prj **ESQUEMÁTICO HEXAPODO**	DESIGNER: **Josué Bohórquez Cristóbal**	PRODUCT NUMBER: **1.0**	SHEET VERSION: **1.0** LAST MODIFIED: **20/07/2026**	SHEET 1 OF 1

Anexo D: Manual de puesta en marcha del sistema

Este anexo describe el procedimiento necesario para compilar, cargar y poner en funcionamiento el firmware de bajo nivel sobre el hexápodo, así como para instalar, compilar y lanzar el subsistema ROS 2 que se ejecuta en la Raspberry Pi. El objetivo no es repetir la arquitectura desarrollada en el Capítulo 4, sino proporcionar una secuencia reproducible de preparación, arranque, comprobación y parada para ambos subsistemas.

D.1. Requisitos previos y seguridad

Para cargar el firmware se necesita un equipo con ESP-IDF 5.5.1, un cable USB de programación y acceso al directorio del proyecto `firmware_esp32`. En el robot, los 18 AX-12A deben estar configurados a 1 Mbit/s y con los identificadores indicados en la Tabla D.1. La PCB debe proporcionar la adaptación *half-duplex* y la alimentación descritas en las Subsecciones 4.2.4 y 4.2.5.

Precaución. Durante la primera puesta en marcha el hexápodo debe permanecer elevado, con todas las patas libres y sin personas u objetos dentro de su recorrido. Al arrancar, el firmware activa el par de los actuadores y envía la postura neutral, aunque el modo de marcha inicial sea `STOP`. Todas las conexiones deben realizarse con la alimentación de los actuadores desconectada.

Pata	Coxa	Fémur	Tibia
Izquierda delantera	1	3	5
Izquierda media	13	15	17
Izquierda trasera	7	9	11
Derecha trasera	8	10	12
Derecha media	14	16	18
Derecha delantera	2	4	6

Tabla D.1: Identificadores de los AX-12A utilizados por el firmware.

D.2. Conexiones utilizadas por el firmware

El ESP32 emplea tres UART diferentes. La UART0 queda reservada para la carga y los mensajes de depuración; la UART1 recibe los comandos de control; y la UART2 se conecta

al circuito de adaptación del bus DYNAMIXEL. La configuración utilizada se resume en la Tabla D.2.

Interfaz	Señales	Velocidad	Uso
UART0	USB/programación	115200 bit/s	Carga del firmware y registro de arranque.
UART1	TX: GPIO4; RX: GPIO5	115200 bit/s	Comandos y telemetría.
UART2	TX: GPIO16; RX: GPIO17; DIR: GPIO2	1 Mbit/s	Bus *half-duplex* de los AX-12A.

Tabla D.2: Interfaces serie empleadas en el ESP32.

Para una prueba directa de la UART1 puede utilizarse un adaptador USB–UART de 3,3 V. Su RX debe conectarse al GPIO4 del ESP32, su TX al GPIO5 y ambas placas deben compartir GND. No debe aplicarse una señal serie de 5 V al ESP32. Cada comando se transmite como una línea de texto terminada en salto de línea. La consola USB de programación no sustituye esta conexión, ya que corresponde a la UART0.

D.3. Compilación y carga

Con el entorno de ESP-IDF inicializado, el firmware se compila y carga mediante:

```
cd /ruta/al/proyecto/firmware_esp32
idf.py build
idf.py -p /dev/ttyUSB0 flash monitor
```

El nombre del puerto puede ser distinto, por ejemplo `/dev/ttyACM0`. La orden `monitor` muestra la salida de depuración de la UART0 y se abandona con `Ctrl+]`.

La carga puede realizarse sin alimentar los actuadores. Sin embargo, antes de la puesta en marcha real debe estar disponible el bus completo y debe reiniciarse el ESP32, puesto que el escaneo y la activación de par se ejecutan una sola vez durante el arranque.

D.4. Primera puesta en marcha

Con el robot elevado y las conexiones revisadas, el procedimiento recomendado es el siguiente:

1. Mantener inicialmente desconectada la alimentación de potencia y comprobar la polaridad, la masa común y los seis ramales del bus.
2. Conectar la interfaz de depuración y, si se desea operar sin ROS 2, un terminal serie independiente a la UART1 a 115200 bit/s, 8N1.
3. Alimentar el sistema o reiniciar el ESP32 con los AX-12A ya disponibles en el bus.
4. Verificar en la consola que el escaneo detecta los servos con IDs 1–18 y finaliza con el mensaje `18/18 respuestas`.

5. Confirmar que las patas adoptan la postura neutral sin colisiones, vibraciones ni movimientos en sentido contrario al esperado.

El estado inicial del firmware es `STOP`, con periodo de marcha de 2 s, altura de paso de 40 mm, postura local nula y tiempo de suavizado de 1 s. Cuando se utiliza ROS 2, el puente modifica esta configuración al conectarse (Sección D.12). El modo `STOP` evita el avance, pero mantiene el par de los actuadores.

D.5. Prueba funcional mediante UART1

Antes de apoyar el robot en el suelo debe realizarse una prueba a baja velocidad. La siguiente secuencia restablece una configuración conocida, activa el patrón trípode y solicita un avance longitudinal reducido:

```
HELP
MOD,0
CMD,0,0,0
POS,0,0,0,0,0,0
SMO,2
GAT,2,20
MOD,1
CMD,20,0,0
```

Tras comprobar varios ciclos, la parada se realiza enviando primero `CMD,0,0,0`. Cuando haya cesado el movimiento, se envía `MOD,0`. Las consignas permanecen almacenadas hasta recibir una nueva orden o reiniciar el sistema; por ello no debe desconectarse el enlace de control mientras exista una velocidad distinta de cero.

Comando	**Función**
`HELP` o `?`	Muestra la ayuda integrada.
`MOD,m`	Selecciona `0`=STOP, `1`=TRIPOD, `2`=RIPPLE o `3`=AMBLE.
`CMD,vx,vy,vyaw`	Fija las velocidades v_x y v_y en mm/s y v_{yaw} en rad/s.
`POS,r,p,y,x,ty,z`	Fija la postura: rotaciones en radianes y desplazamientos en milímetros.
`GAT,T,H`	Fija el periodo T en segundos y la altura H del paso en milímetros.
`SMO,t`	Fija el tiempo de suavizado de la postura en segundos.
`RST`	Reinicia el origen de la odometría.
`REG,id,a,n`	Lee uno o dos bytes de la tabla de control de un AX-12A.
`WREG,id,a,n,v`	Escribe uno o dos bytes en la tabla de control de un AX-12A.

Tabla D.3: Comandos aceptados por la UART1.

El controlador limita individualmente *roll* y *pitch* a ±0,35 rad, el *yaw* local a ±0,52 rad, los desplazamientos x e y a ±40 mm y z a ±50 mm. El periodo de marcha no puede ser inferior a 0,6 s y la altura de paso se mantiene entre 10 y 60 mm. Además, el generador normaliza conjuntamente el desplazamiento efectivo de cada paso, es decir, el que recorre el cuerpo durante una fase de apoyo, con límites de 120 mm en X, 50 mm en Y y 0,35 rad de giro. Estos límites no evitan que una combinación extrema de postura y marcha pueda

producir una cinemática inversa inválida, por lo que la validación inicial debe hacerse con consignas reducidas.

Advertencia sobre `WREG`. Este comando permite modificar cualquier registro admitido por el firmware, incluidos parámetros de EEPROM como el identificador o la velocidad del bus. Debe reservarse para diagnóstico y utilizarse solo conociendo la tabla de control del AX-12A.

D.6. Telemetría y comprobación

La UART1 transmite automáticamente una línea `ODO` cada segundo y publica los resultados del diagnóstico de los actuadores. Las respuestas se interpretan de acuerdo con la Tabla D.4.

Respuesta	Contenido
`ODO,x,y,yaw,roll,pitch,body_yaw,x_body,y_body,z_body`	Odometría plana y postura local confirmada.
`AXH,id,valid,V,T,load,error`	Validez, tensión, temperatura, carga y byte de error de un actuador. La tensión se expresa en décimas de voltio.
`AXR,id,a,n,valid,value,error`	Resultado de una lectura solicitada con `REG`.
`AXW,id,a,n,valid,value,error`	Resultado de una escritura solicitada con `WREG`.

Tabla D.4: Mensajes transmitidos por el firmware.

Una muestra `AXH` no válida o con un byte de error distinto de cero debe investigarse antes de apoyar el robot. La odometría publicada es una estimación obtenida a partir de la consigna de marcha y no sustituye una medida externa del desplazamiento real.

D.7. Parada y resolución de problemas

La parada ordinaria consiste en enviar `CMD,0,0,0`, esperar a que las patas terminen la transición y seleccionar `MOD,0`. Esta orden no desactiva el par; para manipular las articulaciones manualmente debe desconectarse la alimentación de los actuadores. Si se pierde la comunicación con una consigna de velocidad activa, debe cortarse dicha alimentación.

Los fallos más habituales pueden localizarse con las siguientes comprobaciones:

- **No se detecta ningún servo:** comprobar la alimentación del bus, la masa común, la adaptación *half-duplex*, la velocidad de 1 Mbit/s y las señales GPIO16, GPIO17 y GPIO2.
- **El escaneo es incompleto:** revisar el ID, el cableado y la alimentación del ramal correspondiente. No debe haber IDs repetidos.
- **No se reciben comandos:** comprobar que se utiliza la UART1, que TX y RX están cruzados, que el formato es 115200 bit/s 8N1 y que cada orden termina en salto de línea.
- **Se registra una IK inválida:** enviar una velocidad nula, recuperar la postura neutral y reducir la altura, la velocidad o el desplazamiento solicitado antes de reanudar la marcha.
- **Una articulación gira en sentido incorrecto:** detener el sistema y revisar la correspondencia de IDs, los signos y los ajustes de montaje definidos en `leg_config.cpp`.

D.8. Requisitos e instalación de ROS 2 en la Raspberry Pi

El subsistema de alto nivel, descrito en la Sección 4.4, se ejecuta sobre una Raspberry Pi 4 [36] con Ubuntu Server 24.04 LTS de 64 bits, la distribución de base admitida por ROS 2 Jazzy. Sobre esa base se instalan el paquete `ros-jazzy-ros-base`, la herramienta de compilación `colcon` (`python3-colcon-common-extensions`) y el gestor de dependencias `rosdep` (`python3-rosdep`), siguiendo el procedimiento oficial de instalación de ROS 2 [2]. Instalado el entorno, cada sesión debe cargarlo:

```
source /opt/ros/jazzy/setup.bash
```

por lo que conviene añadir esta línea a `~/.bashrc`. La Tabla D.5 resume las dependencias declaradas por cada paquete del proyecto, que `rosdep` resuelve automáticamente en la Sección D.11.

Paquete	Compilación	Dependencias
hexapod_bridge	ament_python	rclpy, geometry_msgs, nav_msgs, std_msgs, tf_transformations, python3-serial
hexapod_web	ament_python	rclpy, geometry_msgs, nav_msgs, std_msgs, tf_transformations, python3-flask
obstacle_avoider	ament_python	rclpy, sensor_msgs, geometry_msgs, python3-numpy
sllidar_ros2	ament_cmake	rclcpp, std_srvs, sensor_msgs
hexapod_bringup	ament_cmake	launch, launch_ros, ament_index_python, laser_filters y los cuatro paquetes anteriores

Tabla D.5: Dependencias declaradas por los paquetes ROS 2 del hexápodo.

D.9. Enlace serie con el ESP32

A diferencia de la prueba directa descrita en la Sección D.2, en funcionamiento normal el puente `/esp32_bridge` se comunica con el ESP32 a través del UART por hardware de la propia Raspberry Pi (`/dev/ttyAMA0`), sin adaptador USB–UART intermedio: el GPIO14 (TX) se conecta al GPIO5 (RX) del ESP32, el GPIO15 (RX) al GPIO4 (TX) del ESP32, y ambas placas comparten masa. Al ser ambos extremos de 3,3 V, no se necesita la adaptación de niveles que sí exige el bus de los AX-12A.

En la Raspberry Pi 4, la interfaz PL011 que corresponde a `/dev/ttyAMA0` está asignada por defecto al controlador Bluetooth, por lo que debe liberarse y habilitarse manualmente. En `/boot/firmware/config.txt` se añade:

```
enable_uart=1
dtoverlay=disable-bt
```

y se deshabilitan los servicios que, de lo contrario, reutilizarían el puerto como consola serie:

```
sudo systemctl disable --now hciuart.service
```

```
sudo systemctl disable --now serial-getty@ttyAMA0.service
sudo usermod -aG dialout $USER
```

Tras reiniciar, `/dev/ttyAMA0` debe existir y ser accesible sin privilegios de superusuario; este es el puerto y la velocidad (115200 bit/s) configurados por defecto en `hexapod_bridge/config/esp32_bridge.yaml`.

Precaución. Un error en `config.txt` puede impedir el arranque gráfico o dejar la Raspberry Pi sin consola serie de emergencia. Conviene conservar una copia del archivo original y disponer de acceso por SSH antes de reiniciar.

D.10. Preparación del LiDAR

El RPLIDAR A1M8 se conecta por USB y se presenta como un adaptador serie CP210x. Para evitar que el nombre del dispositivo cambie según el orden de conexión, `sllidar_ros2` [51] incluye la siguiente regla `udev`, que debe copiarse a `/etc/udev/rules.d/` y recargarse:

```
KERNEL=="ttyUSB*", ATTRS{idVendor}=="10c4", ATTRS{idProduct}==
    "ea60", MODE:="0777", SYMLINK+="rplidar"
```

```
sudo cp src/sllidar_ros2/scripts/rplidar.rules /etc/udev/rules.d/
sudo udevadm control --reload-rules
sudo udevadm trigger
```

A partir de ese momento el sensor queda disponible de forma estable en `/dev/rplidar`, con independencia del puerto `ttyUSB` que le asigne el sistema, y es ese enlace el que conviene indicar al lanzamiento descrito en la Sección D.12.

D.11. Obtención y compilación del espacio de trabajo

Con ROS 2 instalado y `colcon` disponible, el código del proyecto [35] se sitúa dentro de `src` de un espacio de trabajo y se compila:

```
mkdir -p ~/ros2_ws/src
git clone https://github.com/josuebcc/hexapod-ros2 ~/ros2_ws/src
cd ~/ros2_ws
rosdep install --from-paths src --ignore-src -r -y
colcon build --symlink-install
source install/setup.bash
```

El espacio de trabajo contiene cinco paquetes: los tres desarrollados para este proyecto (`hexapod_bridge`, `hexapod_web` y `obstacle_avoider`), el controlador del fabricante `sllidar_ros2` [51] y el paquete de lanzamiento `hexapod_bringup`, que no contiene código propio más allá del archivo de lanzamiento descrito a continuación. Como con el firmware, `source install/setup.bash` debe añadirse a `~/.bashrc` para no repetirlo en cada sesión.

D.12. Lanzamiento del sistema

El lanzamiento conjunto se realiza con un único comando:

```
ros2 launch hexapod_bringup hexapod.launch.py
```

que arranca `/esp32_bridge`, `/web_panel`, `/reactive_node` (el nodo de `obstacle_avoider`), el filtro `/scan_to_scan_filter_chain` de `laser_filters` y el controlador `sllidar_ros2`, en las condiciones resumidas en la Tabla D.6. El puente no incorpora valores por defecto: si se ejecuta de forma aislada con `ros2 run`, debe recibir siempre el archivo `esp32_bridge.yaml`.

Unos dos segundos después de arrancar, `/esp32_bridge` envía una única vez la configuración inicial definida en `esp32_bridge.yaml`. Con los valores por defecto, esta selecciona la marcha trípode (`MOD,1`), un periodo de 2 s con altura de paso de 30 mm (`GAT,2,30`) y un suavizado de 0,5 s (`SMO,0.5`). Por tanto, a diferencia del arranque descrito en la Sección D.4, el robot no permanece en `STOP`, aunque no avanza mientras no reciba una velocidad en `/hexapod/cmd_vel`.

Argumento	Valor por defecto	Función
lidar_port	/dev/ttyUSB0	Puerto serie del LiDAR (usar `/dev/rplidar` tras la Sección D.10).
lidar_baud	115200	Velocidad de transmisión del LiDAR.
start_web	true	Arranca o no el panel web.
start_lidar	true	Arranca o no el controlador del LiDAR.
bridge_params_file	esp32_bridge.yaml	Parámetros del puente ESP32.
web_params_file	web_panel.yaml	Parámetros del panel web.
scan_filters_file	scan_filters.yaml	Filtros aplicados al barrido del LiDAR.

Tabla D.6: Argumentos de `hexapod.launch.py`.

Por ejemplo, para lanzar el sistema con el LiDAR fijado por la regla `udev` y sin el panel web:

```
ros2 launch hexapod_bringup hexapod.launch.py lidar_port:=/dev/
   rplidar start_web:=false
```

El evitador también puede ejecutarse de forma aislada, con `/scan_filtered` como entrada y `/obstacle_avoider/cmd_vel` como salida, siempre que el filtro del LiDAR esté en ejecución:

```
ros2 run obstacle_avoider reactive_node
```

D.13. Uso del panel web

Con `/web_panel` en ejecución, la interfaz queda disponible en el puerto 8080 de la Raspberry Pi, por ejemplo en http://192.168.1.145:8080/ dentro de la red del robot. El panel es el único nodo que publica en `/hexapod/cmd_vel` y arranca siempre en modo manual: en ese modo reenvía sus propios controles, mientras que en modo autónomo reenvía las propuestas recibidas en `/obstacle_avoider/cmd_vel`. La activación del modo autónomo, mediante las rutas `GET/POST /control_mode`, exige una propuesta del evitador

recibida hace menos de 0,5 s; en caso contrario se rechaza. Durante la autonomía el servidor descarta además cualquier velocidad manual con el código HTTP 409, y una parada (`MOD,0`) o el cierre del nodo devuelven el sistema a modo manual. La Tabla D.7 recoge los límites que el panel aplica a la postura del cuerpo, configurables en `web_panel.yaml`.

Parámetro	**Mínimo**	**Máximo**	**Paso**
Roll	$-20°$	$20°$	$1°$
Pitch	$-20°$	$20°$	$1°$
Yaw local	$-25°$	$25°$	$1°$
x	-40 mm	40 mm	1 mm
y	-40 mm	40 mm	1 mm
z	-50 mm	50 mm	1 mm

Tabla D.7: Límites de postura configurados en `web_panel.yaml`.

La vista de servos de la misma interfaz permite leer y escribir registros individuales de los AX-12A mediante los comandos `REG`/`WREG` descritos en la Sección D.5; la escritura no vuelve a leer el registro de forma automática, por lo que la comprobación debe hacerse manualmente con el botón *Leer*.

D.14. Resolución de problemas (Raspberry Pi y ROS 2)

- **`ros2 launch` no encuentra los paquetes:** falta cargar `/opt/ros/jazzy/setup.bash` o `install/setup.bash`, o `colcon build` terminó con errores que no se revisaron.
- **Permiso denegado en `/dev/ttyAMA0`, `/dev/ttyUSB0` o `/dev/rplidar`:** el usuario no pertenece al grupo `dialout`; tras añadirlo es necesario cerrar sesión y volver a entrar.
- **El puente no recibe telemetría del ESP32:** el UART de la Raspberry Pi no está habilitado, el servicio `serial-getty@ttyAMA0` sigue activo, o el puerto y la velocidad de `esp32_bridge.yaml` no coinciden con los de la Sección D.2.
- **El LiDAR no aparece o cambia de nombre:** falta instalar la regla `udev` de la Sección D.10; debe usarse `/dev/rplidar` en lugar de `/dev/ttyUSBn`.
- **El panel web no responde:** comprobar que ningún otro proceso ocupa el puerto 8080 y que la Raspberry Pi es accesible en la red utilizada.
- **No es posible activar el modo autónomo:** `reactive_node` no está en ejecución o no recibe `/scan_filtered` (`start_lidar` desactivado, LiDAR desconectado o filtro detenido), o su última propuesta supera los 0,5 s exigidos.
- **El robot no se detiene al perder la conexión:** verificar que `/esp32_bridge` sigue en ejecución y que `cmd_vel_timeout_s` en `esp32_bridge.yaml` conserva un valor bajo (1 s por defecto).

Anexo E: Capturas de la interfaz web

Este anexo reúne, a mayor tamaño, las vistas de la interfaz web descrita en la Subsección 4.4.2: los controles de teleoperación y configuración, la supervisión conjunta de los 18 servomotores y el acceso a los registros internos de cada AX-12A.

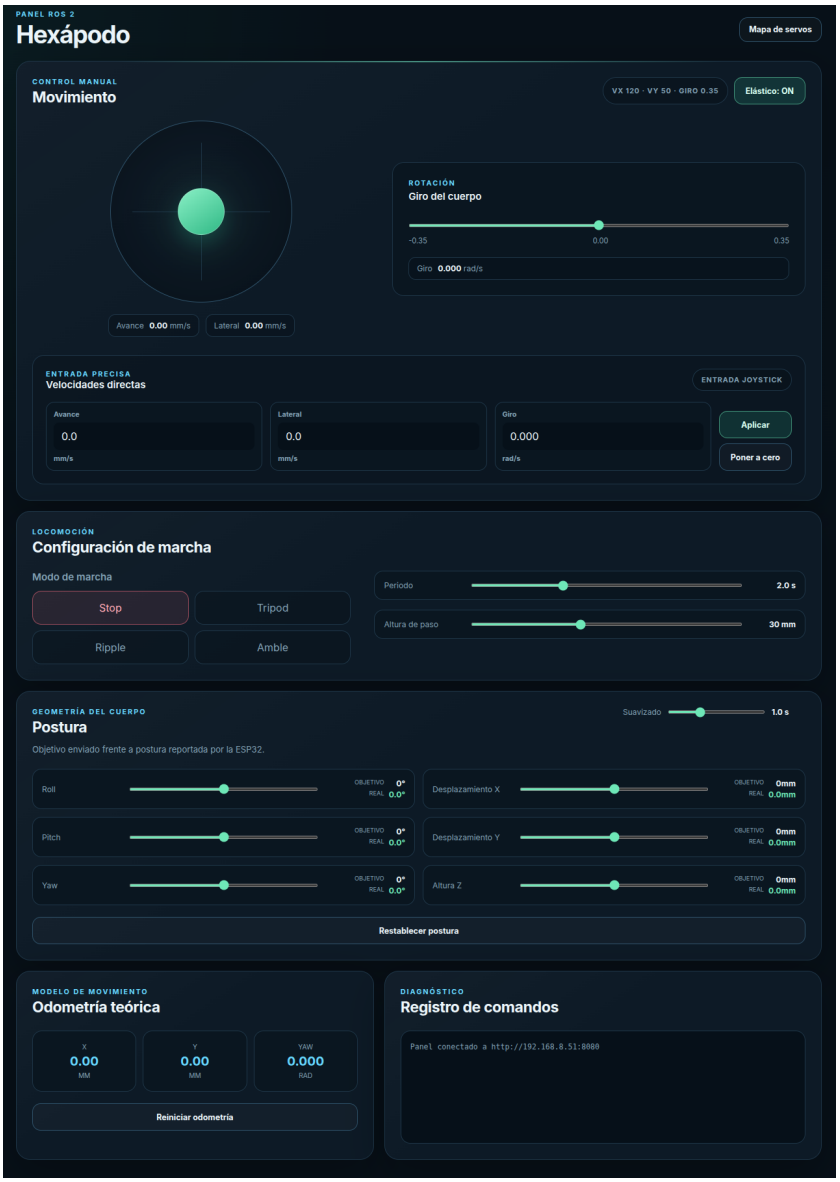


Figura E.1: Controles de movimiento, configuración de la marcha, ajuste de la postura y odometría estimada.
Fuente: elaboración propia.

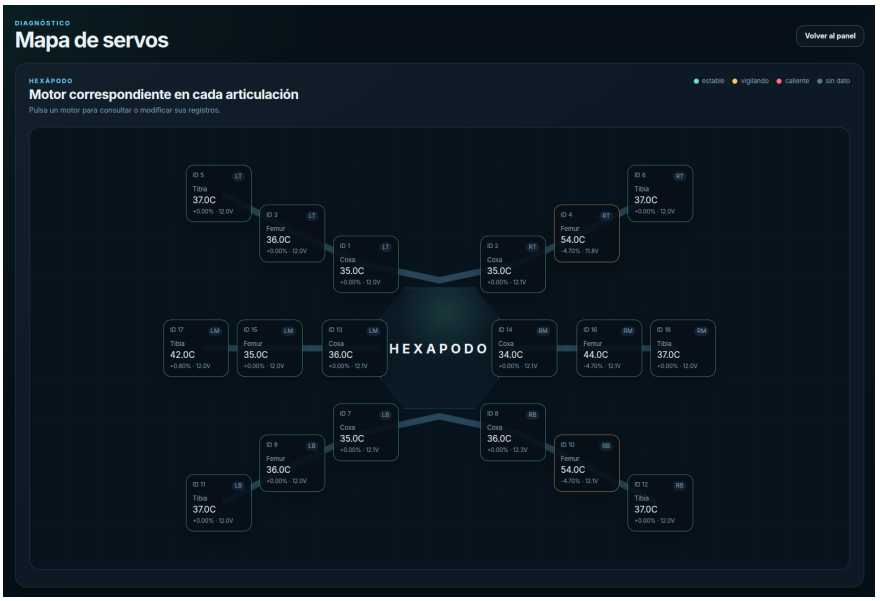


Figura E.2: Distribución de los 18 servomotores con su temperatura, carga y tensión.
Fuente: elaboración propia.

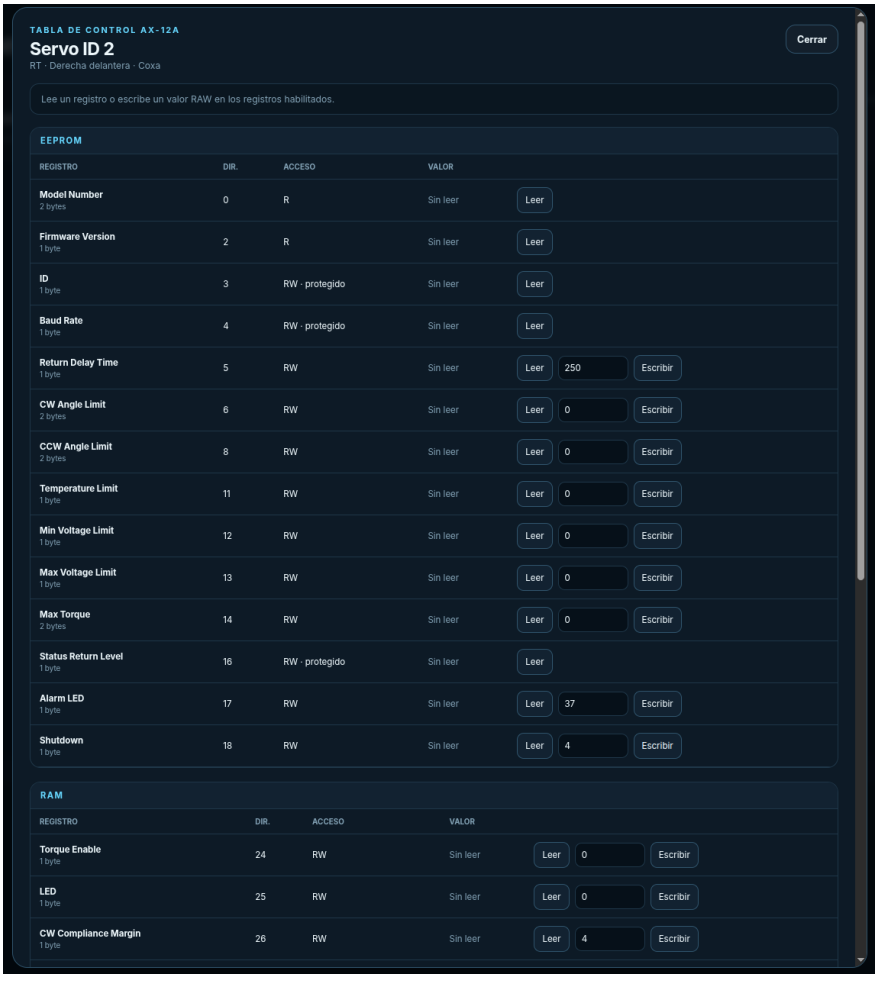


Figura E.3: Lectura y escritura de los registros de un servomotor AX-12A.
Fuente: elaboración propia.